Beretta: An Explainable Cyber-Insurance Underwriting and Continuous Warranty Monitoring Platform

Project Report submitted in the partial fulfilment

of

B.Tech Programme

In

Cybersecurity

by

Harsh Rathod (K080)

Yugandhara Raj Goud (K023)

Krish Mundhra (K041)

Under the supervision of

Prof. Rohit Suryawanshi

SVKM's NMIMS University

(Deemed-to-be University)

SVKM'S NMIMS Deemed-to-be UNIVERSITY

MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING (MPSTME)

Vile Parle (W), Mumbai-56

2026-27

CERTIFICATE

SVKM'S
NMIMS
Deemed-to-be UNIVERSITY

This is to certify that the project entitled **"Automated cyber-insurance engine"** has been done by Krish Mundhra(K041), Harsh Rathod(K080) and Yugandhara Goud(K023) under my guidance and supervision and has been submitted in partial fulfilment of the degree of Bachelor of Technology in Computer Science and Engineering (Cybersecurity) of MPSTME, SVKM's NMIMS (Deemed-to-be University), Mumbai, India.

Project Mentor: Prof. Rohit Suryawanshi
(Internal Guide)

31/10/26
Examiner
Dr. Rupali V.

Date: 31/10/26

Place: Mumbai

Head of Department (HoD)
Dr. Pintu Shah

Mukesh Patel School of Tech. Mgmt. & Engg.
SVKM'S NMIMS MUMBAI 56

ACKNOWLEDGEMENT

We thank our project mentor, Prof. Rohit Suryawanshi, for his guidance throughout this capstone project. His questions about where each number in the premium came from shaped the way the pricing model was built and documented, and his reviews at each stage kept the work on schedule.

We thank the Head of the Department and the Dean of the Mukesh Patel School of Technology Management & Engineering for the facilities and the review process that made this project possible, and the panel members of the project reviews for their comments on the problem statement, the design and the demonstration.

We also thank our families and friends for their support during the year.

Student Name	Roll No.	SAP ID
Harsh Rathod	K080	70102100101
Yugandhara Raj Goud	K023	70102300063
Krish Mundhra	K041	70102300006

ABSTRACT

Cyber insurance remains largely out of reach for Indian small and medium enterprises (SMEs), despite their growing exposure to ransomware, business email compromise and payment fraud. A key barrier lies in how the product is underwritten. Premiums are typically set from static rate tables and unverified self-assessments, the insured has no visibility into how risk was priced, and the security controls that justify a lower premium are seldom verified after the policy is issued. This creates a market where pricing is opaque to the buyer and unreliable for the insurer.

This work proposes Beretta, an explainable cyber-underwriting framework for SMEs that links security evidence directly to premium. Each security control is assessed from multiple sources and weighted according to how strongly it is substantiated, distinguishing what an applicant claims from what can be technically observed. Risk is priced through an actuarial frequency–severity model, in which control effectiveness adjusts expected loss in proportion to its verified strength, so that every premium can be decomposed into traceable, defensible components. Beyond inception, a lightweight endpoint agent continuously validates policy warranties, allowing the insurer to detect lapses and confirm remediation throughout the coverage period.

The prototype demonstrates that cyber insurance pricing can be made transparent to the insured, grounded in verifiable evidence rather than declaration, and sustained by ongoing assurance instead of a one-time assessment. The approach offers a practical path toward fairer, more accessible cyber cover for the SME segment.

Keywords: cyber insurance, explainable underwriting, actuarial risk modelling, security controls, evidence-based assessment, continuous monitoring, SMEs

Table of Contents

Topics	Page
List of Figures	i
List of Tables	v
Abbreviations	vi
Chapter 1 Introduction	1
1.1 Background of the project topic	1
1.2 Motivation and scope of the report	2
1.3 Problem statement	3
1.4 Salient contribution	3
1.5 Organisation of report	4
1.6 Summary	5
Chapter 2 Literature Survey	6
2.1 Introduction to the overall topic	6
2.1.1 Economics of security and insurance	6
2.1.2 Cyber-loss data and distributions	6
2.1.3 How cyber insurance is priced in practice	7
2.1.4 Explainability, standards and regulation	8
2.2 Literature survey of journal and conference papers	8
2.3 Comparison of approaches and research gap	12
2.4 Summary	13
Chapter 3 Methodology and Implementation	14
3.1 Block diagram	14
3.2 Hardware description	16
3.3 Software description, flowcharts and algorithms	18
3.3.1 Design constraints and technology	18
3.3.2 Structure and data model of the platform	19
3.3.3 Engine A: evidence and the evidence ladder	20
3.3.4 Engine B: the pricing model V2	21

3.3.5 Eligibility, binding and the policy life cycle 27

3.3.6 The warranty state machine 28

3.3.7 Part 2: the endpoint agent 29

3.3.8 Part 2: the collector 33

3.3.9 Linking Part 2 to Part 1 37

3.3.10 Roles and access control 39

3.3.11 Security and privacy design 41

3.3.12 User interface 41

3.4 Implementation 47

3.5 Summary 50

Chapter 4 Results and Analysis 50

4.1 Test results 51

4.2 Checks of the pricing model against its sources 53

4.3 Worked example 55

4.4 Sensitivity analysis 56

4.5 Linked demonstration 58

4.6 Role separation 63

4.7 Comparison with the earlier model and with practice 64

4.8 Summary 65

Chapter 5 Advantages, Limitations and Applications 66

5.1 Advantages 66

5.2 Limitations 66

5.3 Applications 68

5.4 Summary 68

Chapter 6 Conclusion and Future Scope 68

6.1 Conclusion 68

6.2 Future scope 69

References 70

Appendix A Soft Code Flowcharts 73

Appendix B Data Sheets 77

Appendix C List of Components 80

ss**Error! Bookmark not defined.**

Appendix D Source Code **Error! Bookmark not defined.**

E.1 Platform (Part 1) 82

E.2 Endpoint agent (Part 2) 97

E.3 Collector (Part 2) 108

List of Figures

Fig No	Name of the figure	Page No
3.1	Block diagram of Beretta: insured endpoints, the collector (Part 2) and the platform (Part 1)	16
3.2	UML component diagram of the whole system	16
3.3	Context data-flow diagram (level 0)	17
3.4	Level-1 data-flow diagram: processes and data stores	17
3.5	UML deployment diagram	18
3.6	Demonstration set-up: the laptop runs both systems and the simulated endpoints; the Windows VM runs the real agent	19
3.7	UML class diagram of the platform (Part 1)	21
3.8	Entity–relationship diagram of the platform database	21
3.9	State diagram of a control's evidence status	23
3.10	Flowchart of a V2 quote (UML activity diagram)	24
3.11	State diagram of a policy	29
3.12	Sequence diagram: quote and bind	30
3.13	Warranty states on the platform and the grace period of each warranty	31
3.14	UML state machine of a platform warranty	31
3.15	Agent layers, from the operating system (1) to the network (10)	32
3.16	UML class diagram of the agent	34
3.17	Sequence diagram: agent enrolment and a signed report	34
3.18	One check from the Windows setting to the platform warranty	35
3.19	Collector overview: agents, ingestion, roll-up, warranties and the dashboard	35
3.20	The ingestion gate: each check must pass before the next runs	36
3.21	The silence rule	36

3.22	Roll-up from a check result to a company control	37
3.23	UML state machine of host liveness	37
3.24	Collector warranty states (stand-alone mode)	38
3.25	UML state machine of a collector warranty (stand-alone mode)	38
3.26	UML class diagram of the collector	38
3.27	Entity–relationship diagram of the collector database	39
3.28	What monitoring does before and after bind	40

Fig No	Name of the figure	Page No
3.29	What happens when EDR stops on one of four machines after bind	40
3.30	UML sequence diagram of the Part 2 → Part 1 roll-up	41
3.31	Use-case diagram for the insured company	42
3.32	Use-case diagram for the insurer	42
3.33	Platform: portfolio dashboard (underwriter)	44
3.34	Platform: company intake, with classification and endpoint monitoring	44
3.35	Platform: security questionnaire with resolved value and evidence per control	45
3.36	Platform: quote summary with premium, eligibility and referral	45
3.37	Platform: classification and the per-peril frequency and payout	46
3.38	Platform: controls and evidence, largest effect first	46
3.39	Platform: from expected loss to premium	47
3.40	Platform: model configuration (administrator), priced controls with their sources	47
3.41	Platform as the insured customer: read-only view of its own company	48
3.42	Platform as the insured customer: premium, eligibility, savings and evidence per control	49
3.43	Deployment of the agent and collector	50

3.44	Demonstration flow with the Windows virtual machine	50
3.45	Collector: overview of companies and hosts	51
3.46	Collector: one company, with the platform's warranty states and the link panel	51
3.47	Collector: one host, with the evidence behind each check	52
4.1	Automated tests per suite	53
4.2	Check on the lognormal shape against the share of SME claims above $500K	55
4.3	Frequency multiplier of each priced control by evidence status	56
4.4	Claims-history multiplier against the number of material incidents	57
4.5	Expected loss by peril, worked example	58
4.6	From expected loss to payable premium, worked example	58
4.7	Premium saving if each control were verified, worked example	59
4.8	Sensitivity of the worked-example premium to each assumption	60
Fig No	**Name of the figure**	**Page No**
4.9	Collector before bind: four simulated endpoints reporting for Kalinga Components	61
4.10	Quote for Kalinga Components: what verification would save, and claims history	62
4.11	Platform Monitoring tab after EDR stopped on one machine: only the EDR warranty moved	63
4.12	After the demonstration's fast-forward: the EDR warranty is breached; the premium is unchanged	64
4.13	Warranty timeline: MET, IN_GRACE, BREACHED, REMEDIATED, MET	64
4.14	UML timing diagram of the EDR warranty during the demonstration	65
4.15	Monitoring tab: endpoints, what the fleet sent, and why	66
A.1	Flowchart: the collector's ingestion of an agent report	77
A.2	Flowchart: the agent's scheduling and reporting loop	78

A.3	Flowchart: end to end, from intake to a breached warranty	79

List of Tables

Table No	Name of the Table	Page No
2.1	Comparison of cyber-insurance pricing and monitoring approaches	12
3.1	Hardware and software used for development and demonstration	18
3.2	Software components and technology	20
3.3	Evidence sources and their strength	22
3.4	Evidence status used by the pricing model	22
3.5	Perils, base frequencies and mean losses (Coalition 2025)	25
3.6	Turnover bands	25
3.7	Frequency controls (Marsh McLennan 2023)	26
3.8	The twelve agent checks	32
3.9	Mapping of collector controls to platform controls	39
3.10	Roles and permissions	41
3.11	Threats and countermeasures	43
3.12	Size of the implementation (lines, including comments)	50
4.1	Automated test suites	53
4.2	What the tests cover	54
4.3	Requirement traceability	54
4.4	Worked example: inputs	57
4.5	Worked example: results	58
4.6	Sensitivity of the worked-example premium (base ₹99,895)	59
4.7	Demonstration run	61
4.8	Customer-role checks	67
4.9	Model V1 and model V2	67
B.1	Global values	81
B.2	Sector frequencies and average losses (Coalition 2025 unless noted)	81

B.3	Loss split by coverage head (shares of each peril's loss)	82
B.4	Agent policy thresholds (policy kyh-endpoint-baseline 2.1.0)	82
B.5	Platform API routes	82
C.1	Platform (Part 1) modules	84
C.2	Monitoring (Part 2) components	84

Abbreviations

Sr. No.	Abbreviation	Full form
1	API	Application Programming Interface
2	BEC	Business Email Compromise
3	BI	Business Interruption
4	CA	Certificate Authority
5	CERT-In	Indian Computer Emergency Response Team
6	CIS	Center for Internet Security
7	CSF	Cybersecurity Framework (NIST)
8	CSP	Content Security Policy
9	CV	Coefficient of Variation
10	CVE	Common Vulnerabilities and Exposures
11	DFD	Data Flow Diagram
12	DMARC	Domain-based Message Authentication, Reporting and Conformance
13	DPDP	Digital Personal Data Protection (Act, 2023)
14	DR	Data Restoration
15	EDR	Endpoint Detection and Response
16	EL	Expected Loss
17	ER	Entity–Relationship
18	FTF	Funds Transfer Fraud
19	GST	Goods and Services Tax

20	HMAC	Hash-based Message Authentication Code
21	HTTP(S)	Hypertext Transfer Protocol (Secure)
22	IdP	Identity Provider
23	IR	Incident Response
24	IRDAI	Insurance Regulatory and Development Authority of India
25	IRIS	Information Risk Insights Study (Cyentia Institute)
26	JSON	JavaScript Object Notation
27	LEV	Limited Expected Value
28	MFA	Multi-Factor Authentication
29	NIC	National Industrial Classification
30	NIST	National Institute of Standards and Technology
Sr. No.	**Abbreviation**	**Full form**
31	NTP	Network Time Protocol
32	OS	Operating System
33	PAM	Privileged Access Management
34	RBI	Reserve Bank of India
35	RDP	Remote Desktop Protocol
36	SEBI	Securities and Exchange Board of India
37	SHA	Secure Hash Algorithm
38	SIEM	Security Information and Event Management
39	SME	Small and Medium Enterprise
40	SOC	Security Operations Centre; also System and Organization Controls (SOC 2)
41	SQL	Structured Query Language
42	TLR	Target Loss Ratio
43	TLS	Transport Layer Security
44	TPL	Third-Party Liability

45	UML	Unified Modeling Language
46	USD / INR	United States Dollar / Indian Rupee
47	VAPT	Vulnerability Assessment and Penetration Testing
48	VM	Virtual Machine

Chapter 1 Introduction

This report describes Beretta, an explainable cyber-insurance underwriting platform for Indian small and medium enterprises (SMEs), and the endpoint-monitoring system linked to it. The platform takes evidence about a company's security controls, prices a policy once at inception with a published-data frequency–severity model, binds the policy, and then watches a small set of warranty controls for the life of the policy. Monitoring can change the state of a warranty, which affects what a claim would pay under the policy wording; it can never change the premium.

This chapter gives the background to the problem, the motivation and scope of the work, the problem statement, the contributions of the project and the organisation of the rest of the report.

1.1 Background of the project topic

A cyber-insurance policy pays for the costs that follow a cyber incident: forensic investigation, restoring data, lost income while systems are down, ransom negotiation, notifying affected people, legal and regulatory costs, and claims from third parties. For a small business the most frequent claims come from a few attack types. Coalition's 2025 claims report, covering its book of mostly small and mid-sized policyholders, finds that business email compromise and funds transfer fraud together account for most claims by count, and ransomware for the largest losses [1].

Underwriting is the insurer's decision about whether to offer cover and at what price. For cyber risk it is harder than for property or motor insurance. Loss data is short (the market is about twenty years old), the threat changes every year, losses are heavy-tailed, and the risk depends on things the insurer cannot see from outside: whether the company patches its systems, whether staff use multi-factor authentication, whether backups can actually be restored. Insurers learn about these controls mainly through a proposal form. Studies of real proposal forms and rate filings find that the questions vary widely between insurers and that the price is set mostly from revenue and industry, with security answers adding a few yes/no modifiers [2], [3].

In India two regulatory changes have made cyber risk a board-level concern for SMEs. The CERT-In Directions of April 2022 require every service provider, intermediary and body corporate to report listed cyber incidents within six hours, keep logs for 180 days

and synchronise clocks with the national time servers [4]. The Digital Personal Data Protection Act, 2023 allows penalties of up to ₹250 crore for failing to protect personal data [5]. Both raise the cost of an incident, and both are reasons for an SME to buy cover.

1.2 Motivation and scope of the report

The project started from three observations.

- **The price is not explained.** An SME that receives a cyber quote usually gets one number. It cannot see which of its answers raised or lowered the premium, or what it could do to lower it.
- **Declared answers are treated as facts.** A "yes" on a proposal form is priced as if the control were in place. Nobody checks it until a claim is made, when a false answer can void the cover.
- **Nothing is watched after bind.** Most policies require some controls (often called warranties) to stay in place during the policy year, but nothing checks them. Commercial insurers such as Coalition and At-Bay scan their policyholders from the internet, but the inside of the network, where patching and EDR status can be seen, is not watched.

Continuous-monitoring-linked cyber insurance exists commercially, so this project does not claim the idea as new. Its contribution is an **open and inspectable** implementation of the whole chain, from evidence through pricing and binding to postbind warranty monitoring, where every number can be traced to a source and every decision is logged. It is meant as a research and teaching prototype and as a design an Indian insurer or managing general agent could study.

The scope of the work and of this report is:

- an underwriting platform (Part 1) with an evidence engine, a pricing engine, binding, a warranty state machine, role-based access for underwriters, administrators and insured customers, and an audit trail;
- a pricing model (model V2) built from published claims data, with every parameter tagged as published, derived or assumed, and a sensitivity analysis of the assumptions;

- a read-only endpoint agent for Windows, Linux and macOS and a collector that receives its signed reports and rolls them up per company (Part 2);
- a signed link through which the collector sends fleet results to the platform, which stays the only authority for warranty states;
- tests for every part and a live demonstration with four simulated endpoints and, optionally, a real Windows virtual machine.

The report does not cover policy wordings, claims handling, reinsurance, or actuarial certification of the rates. The model's frequencies and severities come from USweighted claims studies; adjusting them for India needs Indian claims data that is not yet published (Section 5.2).

1.3 Problem statement

To design and build an explainable cyber-insurance underwriting system for Indian SMEs that (i) gathers evidence about a company's security controls from several sources and records how strong each piece of evidence is, (ii) prices a policy at inception with a frequency–severity model whose every parameter is traceable to a published source or a labelled assumption, and shows the applicant why the premium is what it is and how to lower it, (iii) monitors the insured company's endpoints after bind with a read-only agent whose reports cannot be forged or replayed, and (iv) lets monitoring change warranty states only, never the bound premium.

1.4 Salient contribution

- **An evidence ladder for underwriting.** Every input is an observation with a source, a locator and one of four evidence strengths: DECLARED, DOCUMENTED, TESTED and TECHNICALLY_OBSERVED. For each control the strongest, most recent evidence wins, and disagreeing sources are flagged. The pricing model reads this as VERIFIED, STATED, NOT_WORKING or UNKNOWN (Section 3.3.3).
- **A published-data pricing model.** Model V2 prices five perils separately, each with a frequency from Coalition's 2025 claims data and a lognormal loss whose shape ($\sigma = 2.193$) is fitted to NetDiligence's 2025 SME claims. Control effects come from Marsh McLennan's 2023 study and are centred so that the average firm

pays the base rate. A stated control earns 60% of a verified control's credit, a figure derived from an assumed hollow-declaration rate. Claims history enters through a credibility formula with $\alpha = 4.22$ (Section 3.3.4).

- **Full explainability.** Every quote stores the classification, the per-peril frequency chain, each control's multiplier with its evidence status and source, the expected payout, the loadings, the eligibility and referral reasons, the list of assumptions used, and what verifying each control would save. A stored snapshot reproduces the premium exactly later.
- **A read-only, tamper-evident endpoint agent.** Twelve checks on three operating systems, signed reports (HMAC-SHA256 over the exact bytes), monotonic sequence numbers against replay, a build hash that exposes modified agents, a silence rule that turns an uninstalled agent into a failure, and a 95% fleet roll-up (Section 3.3.7).
- **Monitoring that never re-prices.** After bind, fleet results reach only the warranty state machine (MET, IN_GRACE, BREACHED, REMEDIATED). The engine refuses to price a bound policy, and the tests check this on every run.
- **A role-separated interface.** Underwriters enter intake and price; administrators change the model configuration; the insured customer sees a read-only view of its own company, with its premium, the evidence held for each control and how to lower the premium.

1.5 Organisation of report

Chapter 2 surveys the literature on cyber-insurance economics, cyber-loss data, pricing practice and monitoring, and identifies the gap the project addresses. Chapter 3 describes the methodology and implementation: the block diagram, the hardware used, and the software of both parts, with UML diagrams, the pricing model and its algorithms, and screens of the working system. Chapter 4 presents the results: the test results, checks of the model against its sources, a worked example, a sensitivity analysis, and the linked demonstration. Chapter 5 lists the advantages, limitations and applications of the system, and Chapter 6 concludes and sets out future work. The appendices give the flowcharts, the parameter data sheets, the list of software components, the list of publications and the complete source code.

1.6 Summary

Cyber insurance for Indian SMEs is priced from unexplained rate tables and unverified answers, and nothing is checked after bind. Beretta addresses this with an evidencegraded, published-data pricing model and read-only monitoring that drives warranties but never the premium.

Chapter 2 Literature Survey

This chapter reviews the work on which the project builds. Section 2.1 gives the background: the economics of information security, the data on cyber losses, how cyber insurance is priced in practice, and the standards and regulations that define security controls. Section 2.2 reviews the main journal and conference papers in date order. Section 2.3 compares them and states the research gap that defines the problem.

2.1 Introduction to the overall topic

2.1.1 Economics of security and insurance

Security economics explains why companies under-invest in security. Gordon and Loeb modelled the optimal investment in protecting an information set and showed that it should not exceed about 37% (1/e) of the expected loss, and that the most vulnerable assets are not always the best ones to protect [6]. Anderson and Moore argued that many security failures come from misaligned incentives and externalities rather than missing technology: the party that could prevent a loss is often not the one that bears it [7]. Insurance is one way to re-align these incentives, because an insurer that prices security into the premium gives the insured a financial reason to improve it.

Whether cyber insurance actually improves security is contested. Böhme and Schwartz proposed a unifying framework for cyber-insurance models built on five components: the networked environment, demand side, supply side, information structure and organisational environment [8]. Shetty et al. showed that with information asymmetry, competitive insurance may not raise security at all, because insurers cannot observe the insured's effort [9]. Pal et al. reached a similar conclusion for networks with interdependent risks, unless contracts can discriminate on security [10]. Both results point to the same need: the insurer must be able to observe security, which is what monitoring provides.

2.1.2 Cyber-loss data and distributions

Pricing needs a model of how often losses happen and how large they are. Early studies of breach data found heavy tails. Maillart and Sornette found that the number of records lost per breach follows a power law [11]. Edwards, Hofmeyr and Forrest analysed a decade of breach reports and found breach sizes close to lognormal and daily breach

counts close to negative binomial, with no clear upward trend in either [12]. Wheatley, Maillart and Sornette found that the largest breaches were growing [13]. Biener, Eling and Wirfs studied 994 cyber losses in an operational-risk database and concluded that cyber risk is insurable but differs from other operational risks in its correlation and lack of data [14]. Eling and Wirfs later separated everyday cyber losses from extreme ones and fitted them with separate distributions [15]. Eling and Schnell's review listed the main obstacles to insuring cyber risk: lack of data, accumulation risk and information asymmetry [16].

Industry claims studies now publish the figures an actuary needs. Coalition's 2025 report gives claim frequencies and average losses by peril, revenue band and industry for its policyholders [1]. NetDiligence's 2025 study gives the mean and the tail of SME claim costs [17]. Cyentia's IRIS reports give incident probabilities by firm size and sector [18], [19]. These are the sources of model V2's frequencies, severities and the lognormal shape.

2.1.3 How cyber insurance is priced in practice

Romanosky et al. read 235 cyber-insurance filings made to US state regulators, including their security questionnaires and rate schedules [3]. Most carriers computed the premium as a base rate from revenue or assets, multiplied by industry and a small number of hazard factors; security questions entered, if at all, as broad modifiers. Woods et al. mapped the questions on 24 proposal forms to the CIS security controls and found that forms cover different controls and rarely ask how a control is verified [2]. Woods and Simpson described the ways insurers can influence security, through questionnaires, premiums and post-bind services [20], and Talesh showed that insurers already act as compliance managers for their policyholders [21]. Woods, Moore and Simpson inferred the loss distributions that insurers assume from the prices they file [22]. Marotta et al. surveyed the field as a whole, from the theoretical models to the products on the market [23].

The effect of individual controls is now measured from claims. Marsh McLennan's 2023 study of its clients' claims gives, for each of twelve controls, how much more often firms without it had a claim and what share of firms have it [24]. Sophos measured how much compromised backups raise ransomware costs [25], IBM's breach-cost studies give the cost effect of incident-response planning, training and encryption [26], [27],

and Microsoft's study of its accounts found that MFA reduced the risk of compromise by more than 99% [28]. These give model V2 its control multipliers.

Claims experience needs its own treatment. Credibility theory, set out by Bühlmann, weights a policyholder's own claims history against the collective rate according to how much information the history carries [29]. Model V2's claims multiplier is a credibility formula of this kind.

2.1.4 Explainability, standards and regulation

Doshi-Velez and Kim define interpretability as the ability to explain a decision in terms a person can understand, and argue that it matters most where decisions affect people and must be justified [30]. Underwriting is such a decision. Beretta's model is interpretable by construction: it is a product of named factors, each with a source, rather than a fitted black box.

Security controls are defined by frameworks. The NIST Cybersecurity Framework 2.0 organises controls into six functions [31], the CIS Critical Security Controls v8 list eighteen prioritised controls [32], and ISO/IEC 27001:2022 specifies an information security management system with a statement of applicability [33]. Beretta's fifteen controls are drawn from these, and an ISO 27001 statement of applicability is one of the documents that counts as independent evidence. In India the regulatory context is set by the CERT-In Directions [4], the DPDP Act [5] and, for the insurer's expenses, the IRDAI regulations of 2023 [34]. The model's sector comes from the National Industrial Classification 2008 code [35]. Dambra, Bilge and Balzarotti surveyed the technical side of cyber insurance and set out a roadmap in which system-security research supplies the measurements insurers lack [36]; continuous endpoint monitoring is one of the directions they name.

2.2 Literature survey of journal and conference papers

The papers below are reviewed in date order. For each, the contribution and its relevance to this project are given.

Bühlmann (1967), *Experience rating and credibility* [29]. Bühlmann derived the credibility premium as a weighted average of the policyholder's own experience and the collective mean, with a weight that grows with the amount of experience. The result is the basis of experience rating in general insurance. Model V2's claims multiplier, M =

$(\alpha + N) / (\alpha + \lambda T)$, is the Poisson–gamma form of this credibility weight: with no history ($T = 0$) it is 1, and each material incident moves it by an amount that depends on how many incidents the firm was expected to have.

Gordon and Loeb (2002), *The economics of information security investment* [6]. The authors built a model of the optimal investment in security for an information set with a given vulnerability and potential loss. They showed that investment should not exceed 1/e of the expected loss and that it can be optimal to protect moderately vulnerable assets rather than the most vulnerable. The paper frames security as an economic decision under uncertainty; Beretta's what-if savings give the insured the price side of that decision, the premium saving from verifying each control.

Anderson and Moore (2006), *The economics of information security* [7]. This review in *Science* argued that security failures often come from misaligned incentives, network externalities and information asymmetry rather than from technical weakness. It established security economics as a field. For insurance, the asymmetry is between the insured, who knows its controls, and the insurer, who does not; Beretta's evidence ladder and monitoring reduce that asymmetry.

Böhme and Schwartz (2010), *Modeling cyber-insurance: Towards a unifying framework* [8]. The authors reviewed the analytical models of cyber insurance and organised them into one framework of five components. They noted that most models assume the insurer can observe security or ignore the question. Beretta makes the information structure explicit: every input carries its evidence strength, and the price depends on it.

Shetty, Schwartz, Felegyhazi and Walrand (2010), *Competitive cyber-insurance and Internet security* [9]. Using a model with competitive insurers and unobservable security, the authors showed that insurance can reduce users' security effort (moral hazard) and does not improve overall security. The result depends on the insurer being unable to observe security. It motivates Beretta's two mechanisms for making security observable: technical evidence before bind and warranty monitoring after it.

Maillart and Sornette (2010), *Heavy-tailed distribution of cyber-risks* [11]. Analysing breach records, the authors found that the size of breaches follows a heavy-tailed power-law distribution. Heavy tails mean that the average loss is dominated by rare large events, which is why a pricing model must apply limits and retentions to the full

distribution rather than to an average. Model V2 integrates the payout over the whole lognormal through the retention and the limit.

Pal, Golubchik, Psounis and Hui (2014), *Will cyber-insurance improve network security? A market analysis* [10]. The authors analysed cyber-insurance markets with interdependent security and found that insurance improves network security only when contracts can discriminate between users by their security level. Beretta's control multipliers are such discrimination: two firms in the same sector and turnover band pay different premiums if their evidenced controls differ.

Biener, Eling and Wirfs (2015), *Insurability of cyber risk: An empirical analysis* [14]. From 994 cyber-related operational losses, the authors found that cyber losses differ from other operational risks in their distribution and that insurability is limited by data scarcity, correlated losses and information asymmetry. Model V2 addresses data scarcity by using the large claims datasets published since, and labels every parameter that is still an assumption.

Edwards, Hofmeyr and Forrest (2016), *Hype and heavy tails: A closer look at data breaches* [12]. Using ten years of US breach reports, the authors found that breach sizes are well described by a lognormal distribution and breach counts by a negative binomial, and that neither had grown significantly. Their finding supports V2's lognormal severity, and the negative binomial is the frequency model behind V2's credibility parameter α, which is derived from a negative-binomial fit to IRIS 2022 data.

Eling and Schnell (2016), *What do we know about cyber risk and cyber risk insurance?* [16]. This review summarised the literature on cyber-risk measurement and insurance and listed the open problems: missing data, risk accumulation, information asymmetry and the lack of standard policy terms. Beretta works on two of these: information asymmetry (evidence and monitoring) and transparency of pricing (the tagged parameter file).

Wheatley, Maillart and Sornette (2016), *The extreme risk of personal data breaches and the erosion of privacy* [13]. The authors modelled the largest personal-data breaches with extreme-value methods and found that the frequency of very large breaches was increasing. For an insurer this means the tail matters and may be growing. V2 keeps the tail in the payout calculation and includes σ in its sensitivity analysis; a lower σ in fact raises the premium for the worked example, because more of the loss falls between the retention and the limit (Section 4.4).

Marotta, Martinelli, Nanni, Orlando and Yautsiukhin (2017), *Cyber-insurance survey* [23]. This survey covered the theoretical models of cyber insurance, the riskassessment approaches and the products then on the market. It noted that pricing methods in practice are rarely published. Beretta publishes its whole method and every parameter.

Woods, Agrafiotis, Nurse and Creese (2017), *Mapping the coverage of security controls in cyber insurance proposal forms* [2]. The authors collected 24 proposal forms and mapped their questions to the CIS controls. Coverage of controls varied widely between insurers, and the forms asked whether a control existed, not how it was verified. Beretta's questionnaire covers fifteen controls, asks for existence, currency and testing of the incident-response plan and backups, and treats a questionnaire answer as the weakest evidence.

Woods and Simpson (2017), *Policy measures and cyber insurance: A framework* [20]. The authors described how insurers can act on security at three points: before bind through the application, at bind through the premium and conditions, and after bind through services and monitoring. Beretta implements all three: evidence-graded intake, a priced quote with warranty conditions, and post-bind warranty monitoring.

Doshi-Velez and Kim (2017), *Towards a rigorous science of interpretable machine learning* [30]. The authors defined interpretability and proposed ways to evaluate it, arguing that it is needed where decisions must be justified or audited. Underwriting decisions affect customers and are regulated. V2 is a transparent multiplicative model in which every factor is shown, so no separate explanation method is needed.

Talesh (2018), *Data breach, privacy, and cyber insurance* [21]. Based on fieldwork with insurers, Talesh showed that cyber insurers provide risk-management services and shape how their policyholders comply with privacy law, acting as compliance managers. Beretta's customer view, which shows the insured what each control would save, is a small example of this role.

Romanosky, Ablon, Kuehn and Jones (2019), *Content analysis of cyber insurance policies: How do carriers price cyber risk?* [3]. From 235 regulatory filings, the authors found that most carriers price from revenue or assets with industry modifiers, that security questionnaires focus on general practices, and that few carriers use detailed technical information. This is the practice that Beretta's evidence-graded, control-level model changes.

Eling and Wirfs (2019), *What are the actual costs of cyber risk events?* [15]. The authors separated daily cyber losses from extreme ones and modelled them with different distributions, showing that the cost of cyber risk depends strongly on the tail. V2 uses one lognormal per peril with a shared shape fitted to SME claims, checked against two tail points (Section 4.2).

Dambra, Bilge and Balzarotti (2020), *SoK: Cyber insurance – technical challenges and a system security roadmap* [36]. This systematisation-of-knowledge paper at the IEEE Symposium on Security and Privacy reviewed the technical problems of cyber insurance: collecting reliable data, measuring the security posture of an applicant and monitoring it over time. It proposed that system-security research provide these measurements. Beretta's agent and collector are a working example of measuring and monitoring posture with signed, read-only endpoint checks.

Woods, Moore and Simpson (2021), *The county fair cyber loss distribution* [22]. The authors inferred the loss distributions that insurers assume from the prices in their regulatory filings. Their method treats prices as evidence of the insurer's model. Beretta takes the opposite route, building the price from published loss data, so the assumed loss distribution is stated rather than inferred.

2.3 Comparison of approaches and research gap

Table 2.1 compares the main approaches in the literature and in practice with what this project builds.

Table 2.1 Comparison of cyber-insurance pricing and monitoring approaches

Approach	Price basis	Security information	Verified?	After bind	Explained to insured?
Analytical models [8], [9], [10]	Theoretical	Assumed observable or not	n/a	n/a	n/a
Approach	**Price basis**	**Security information**	**Verified?**	**After bind**	**Explained to insured?**
Filed rate plans [3]	Revenue × industry × hazard factors	Yes/no questionnaire	No	Nothing	No
Proposal forms [2]	Underwriter judgement	Varies by insurer; existence only	No	Nothing	No
Price-implied loss models [22]	Inferred from prices	None	n/a	n/a	n/a

Commercial monitored insurers	Proprietary	Questionnaire + outside-in scan	Outside only	External scans, alerts	Partly
Beretta (this project)	Published claims data, five perils, lognormal payout	15 controls, four evidence strengths, agent	Yes: documents, tests, agent, scans	Signed agent monitoring drives warranties only	Yes: every factor, savings per control

Three gaps follow from the survey:

1. **Pricing is not traceable.** The literature describes pricing as base rates and modifiers, but published models do not connect each factor to a published loss source, and rate filings do not explain the factors to the applicant [3], [23].
2. **The strength of security evidence is ignored.** Proposal forms ask whether a control exists, and the answer is priced the same whether it is a self-declaration or a test result [2]. Economic models show that this asymmetry can make insurance useless for security [9].
3. **Monitoring and pricing are not separated.** Where monitoring exists, it is external and its link to the policy is not published. Re-pricing a policy mid-term from monitoring data would be unfair to the insured and is not how policies work; the right link is through warranty conditions [20], [36].

The problem statement in Section 1.3 follows from these gaps: an open system that grades evidence, prices from published data with every factor shown, and monitors warranties after bind without touching the premium.

2.4 Summary

The literature shows that cyber losses are heavy-tailed and roughly lognormal, that insurance improves security only if the insurer can observe it, and that real pricing relies on revenue, industry and unverified answers. No published system combines evidencegraded inputs, a published-data model with every factor explained, and post-bind monitoring limited to warranties. That combination is what Chapter 3 builds.

Chapter 3 Methodology and Implementation

This chapter describes how Beretta was designed and built. Section 3.1 gives the block diagram of the whole system. Section 3.2 describes the hardware it runs on. Section 3.3 describes the software of both parts, with the UML diagrams, the pricing model and the main algorithms. Section 3.4 shows the working system. The design descriptions follow the viewpoints of IEEE Std 1016-2009 (context, composition, logical, information, interaction and state dynamics) [37], the requirements were written in the form recommended by ISO/IEC/IEEE 29148:2018 [38], and all diagrams use UML 2.5.1 notation [39].

The work followed an iterative method. Part 1 was built first with an illustrative additive pricing model (V1), the endpoint agent and collector were built next as Part 2, the two were then linked, and finally the pricing engine was replaced with the published-data model V2. V1 is kept, frozen, so that quotes made under it can still be reproduced.

3.1 Block diagram

Fig. 3.1 is the block diagram. The insured company's machines run the read-only agent. The agents report to the collector (Part 2), which checks each report, works out a result per control for the whole company and sends one signed roll-up to the platform (Part 1). In the platform, Engine A records the roll-up as evidence; before bind Engine B uses that evidence to price the policy, and after bind only the warranty state machine reads it. The collector reads the warranty states back for display.

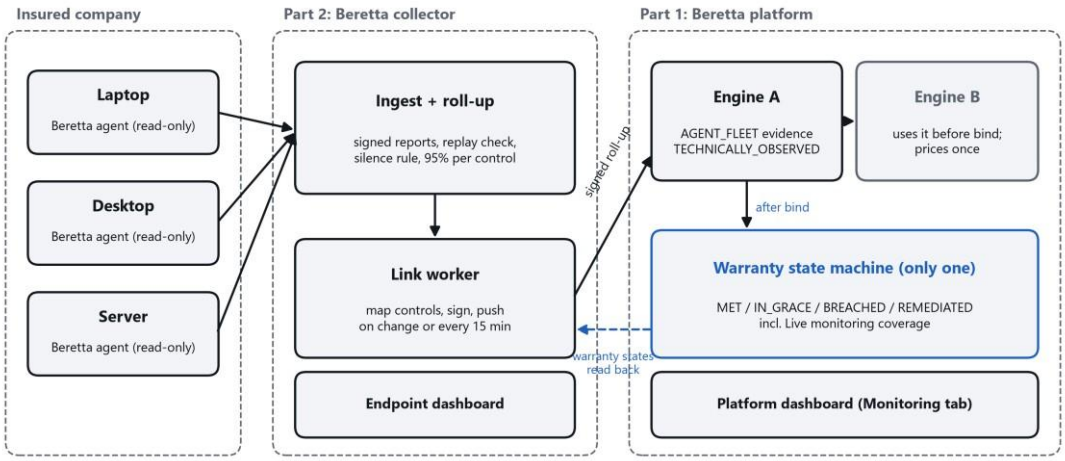


Fig. 3.1 Block diagram of Beretta: insured endpoints, the collector (Part 2) and the platform (Part 1)

Fig. 3.2 shows the same system as UML components with their interfaces, and Fig. 3.3 shows it as a context data-flow diagram: the people and machines outside the system and the data each exchanges with it. Fig. 3.4 breaks the system into its level-1 processes and data stores.

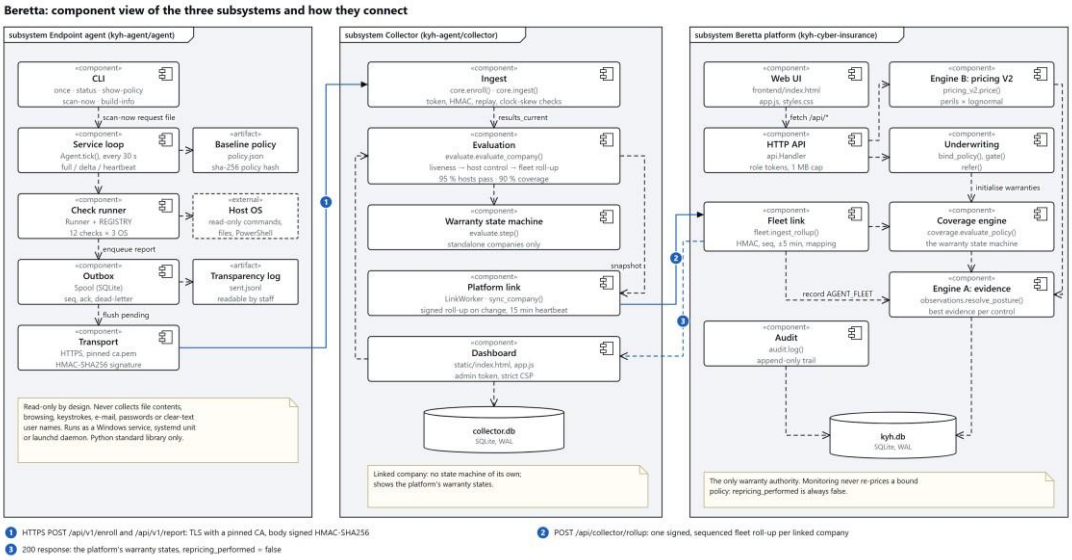


Fig. 3.2 UML component diagram of the whole system

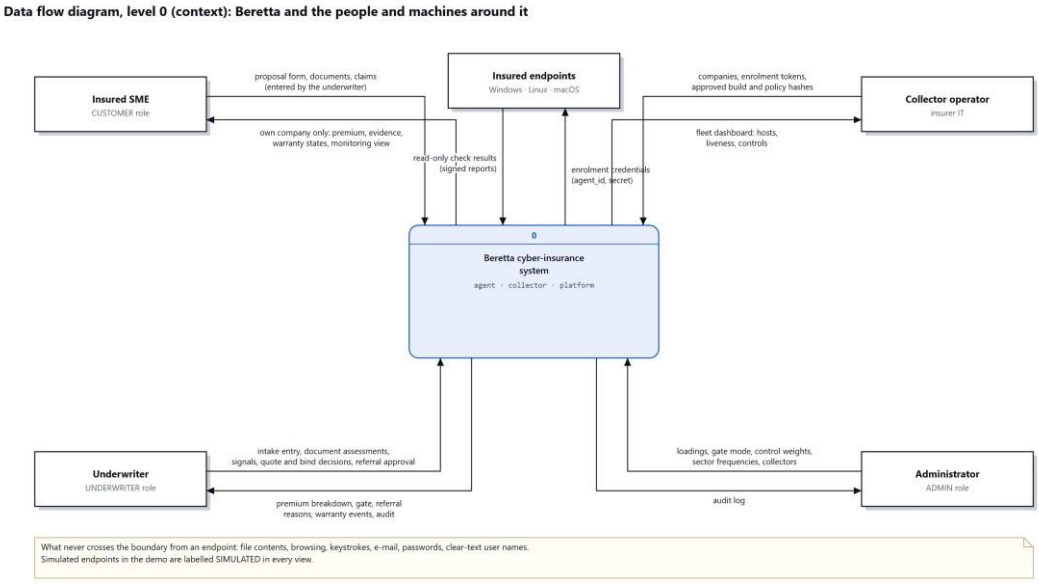


Fig. 3.3 Context data-flow diagram (level 0)

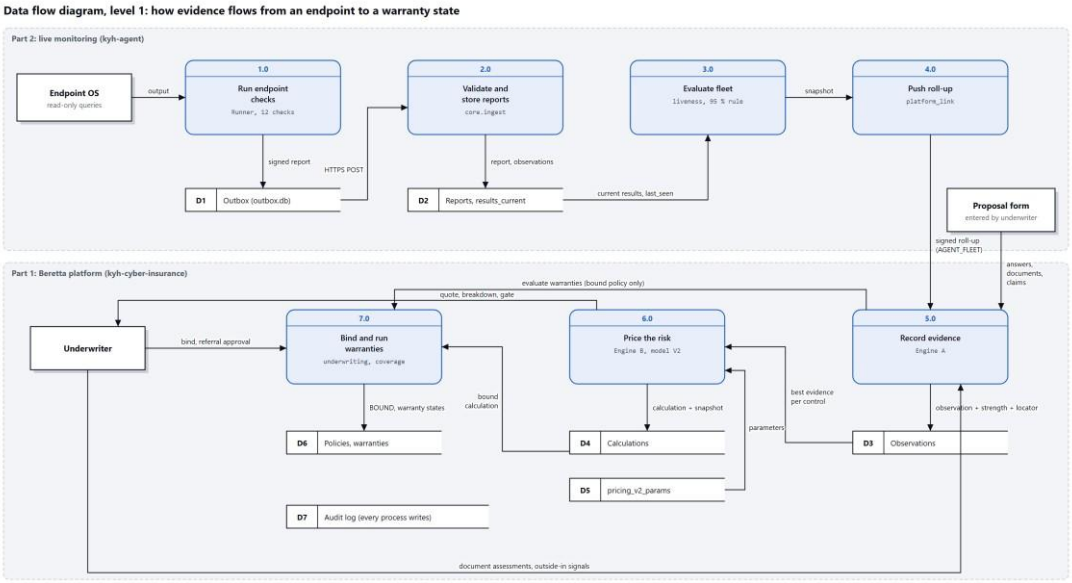


Fig. 3.4 Level-1 data-flow diagram: processes and data stores

3.2 Hardware description

Beretta needs no special hardware. The platform and the collector are ordinary Python programs that run on any computer with Python 3.9 or later, and the agent runs on the insured company's existing laptops, desktops and servers. Table 3.1 lists the machines used to build and demonstrate the system.

Table 3.1 Hardware and software used for development and demonstration

Item	Specification	Role
Development laptop	Intel Core Ultra 9 285H, 16 cores; 32 GB RAM; Windows 11 Home (build 26200)	Runs the platform (port 8000), the collector (port 8443), four simulated endpoints, the tests and the document tools
Windows virtual machine	Oracle VirtualBox 7.2; Windows 11 Enterprise evaluation (26H2, x64); 4 GB RAM, 64 GB disk; virtual TPM 2.0 and Secure Boot	A real endpoint running the agent as a SYSTEM scheduled task, reached over the host-only network 192.168.56.0/24
Ubuntu 24.04 build machine	Linux, Python 3	Real run of the Linux checks during agent development
Python	3.14.5 on Windows (any 3.9+ works)	The only runtime for every part
Web browser	Microsoft Edge 156 (any current browser)	The two dashboards; headless Edge also renders the figures
Node.js	24.16 with the *docx* package	Builds the Word documents only; not part of the system

The deployment is shown in Fig. 3.5. In production the platform would run at the insurer, the collector on a server the insurer controls, and the agents on the insured's machines, with HTTPS between each. The agent reaches only the collector, using a certificate authority file pinned at install. Fig. 3.6 shows the demonstration set-up with the virtual machine.

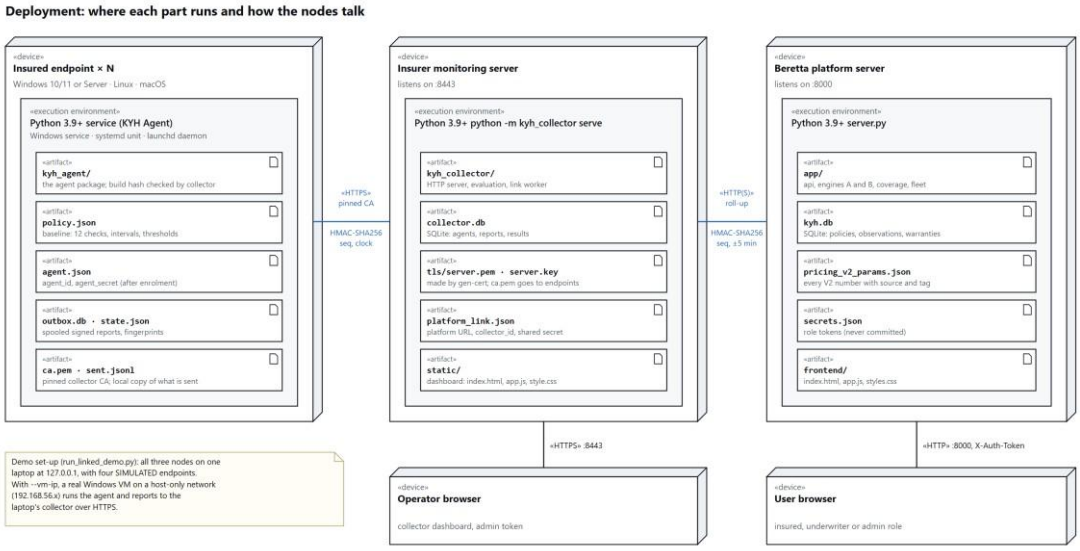


Fig. 3.5 UML deployment diagram

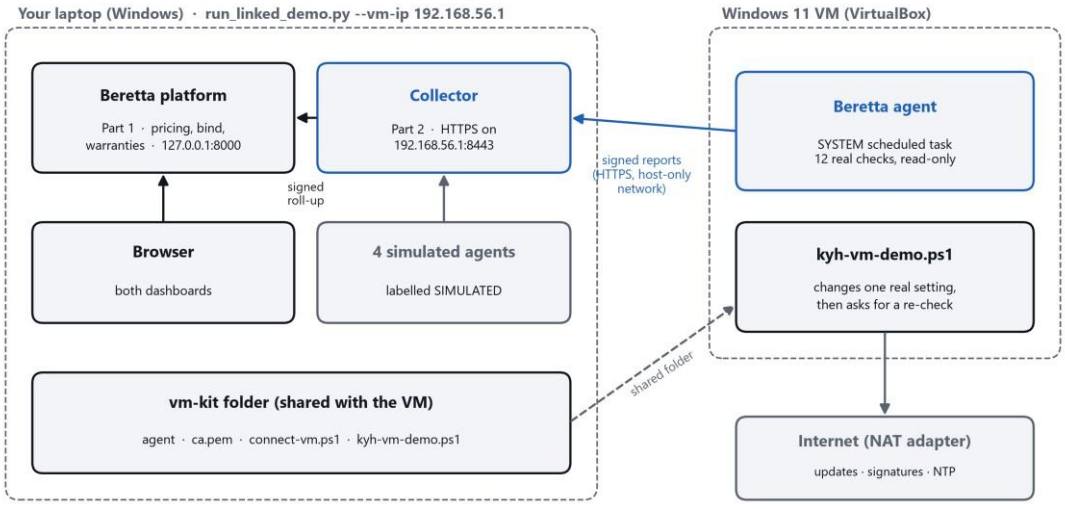


Fig. 3.6 Demonstration set-up: the laptop runs both systems and the simulated endpoints; the Windows VM runs the real agent

The agent's footprint is small. It is a set of Python files and a policy file, it runs each check on its own interval (EDR hourly, patching and backup every four hours, the rest daily), and a report is a few kilobytes of JSON. Reports wait in a local SQLite outbox when the machine is offline.

3.3 Software description, flowcharts and algorithms

3.3.1 Design constraints and technology

Four constraints shaped the software, and each is checked by the tests or the code review rules of the project:

- **Standard library only.** Every part uses only the Python standard library, and the dashboards are plain HTML, CSS and JavaScript with no frameworks or external resources. Anyone can read and run the whole system without installing packages, and there are no third-party dependencies to audit.
- **One place for every number.** Every V2 pricing value lives in `app/pricing_v2_params.json` with its value, unit, source, locator and tag. Code never contains a pricing constant.
- **Monitoring never re-prices.** After bind, monitoring data reaches only the warranty state machine. The pricing engine refuses to price a bound policy, and the platform's answer to every roll-up carries `repricing_performed: false`.
- **Simulated data is always labelled.** Every simulated endpoint result starts with "[SIMULATED]" and every view shows the label.

Table 3.2 Software components and technology

Component	Technology	Main modules
Platform server and API	Python `http.server`, JSON over HTTP [40]	`api.py` (40 routes), `security.py`, `audit.py`
Engine A (evidence)	Python, SQLite	`observations.py`, `catalogue.py`
Engine B (pricing V2)	Python `math`, `statistics.NormalDist`	`pricing_v2.py`, `pricing_v2_params.json`
Binding and warranties	Python, SQLite	`underwriting.py`, `coverage.py`, `fleet.py`
Platform dashboard	HTML, CSS, JavaScript	`frontend/index.html`, `app.js`, `styles.css`
Endpoint agent	Python; PowerShell, `/proc`, `winreg`, OS tools (read-only)	`checks/` (Windows, Linux, macOS), `runner.py`, `security.py`, `spool.py`, `transport.py`
Collector	Python `http.server` over TLS, SQLite	`server.py`, `core.py`, `evaluate.py`, `platform_link.py`
Collector dashboard	HTML, CSS, JavaScript under a strict Content-Security-Policy	`static/index.html`, `app.js`, `style.css`

3.3.2 Structure and data model of the platform

The platform is organised as modules with one job each. Fig. 3.7 shows the classes and modules and how they call each other, and Fig. 3.8 the database. A company has one policy; a policy has its coverages, its calculations (each with the full input snapshot and the model version) and its warranties, and each warranty has a history of state changes. Every observation of a control belongs to a company and records its source, locator, evidence strength and time. Every decision is written to the audit table.

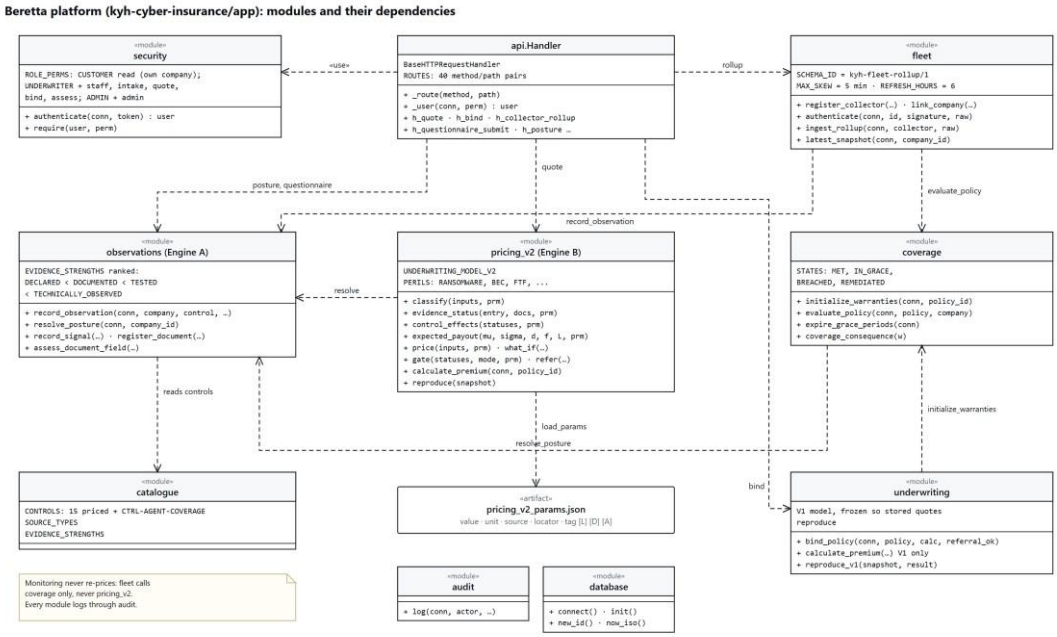


Fig. 3.7 UML class diagram of the platform (Part 1)

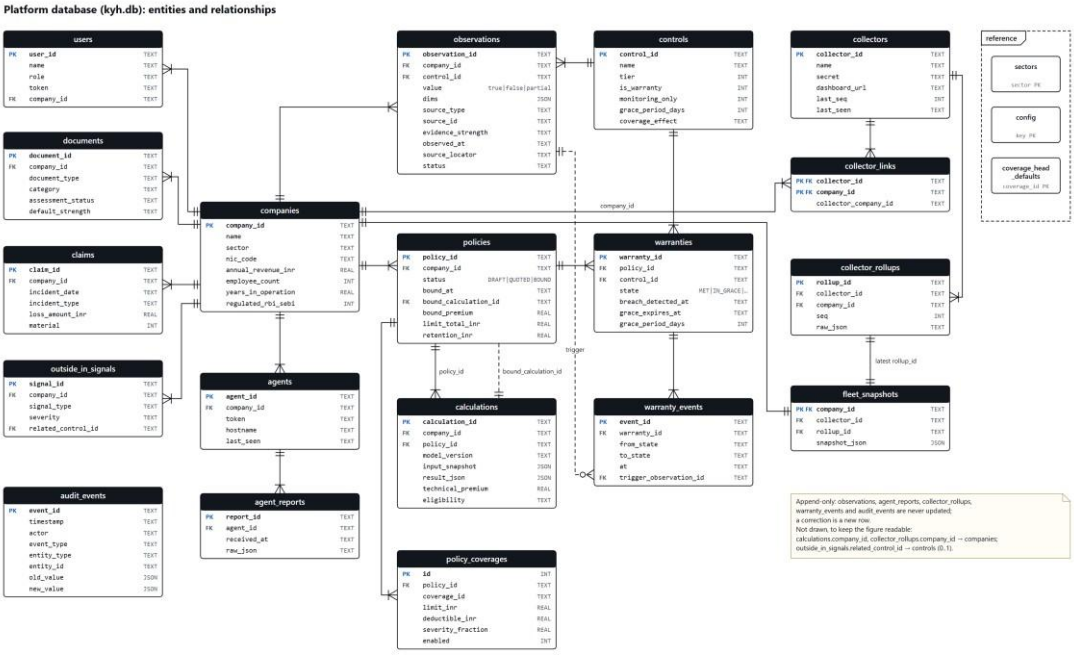


Fig. 3.8 Entity–relationship diagram of the platform database

3.3.3 Engine A: evidence and the evidence ladder

Engine A turns everything known about a company into one resolved value per control. Each input is stored as an **observation** with the control, the value (true, partial or false), the source type, a source locator (for example "Question IAM-04" or "Page 14 §5.2"), the evidence strength and the time. Table 3.3 gives the sources and their default strengths.

Table 3.3 Evidence sources and their strength

Source	Default strength	Example
Questionnaire answer	DECLARED	"Yes, MFA is enforced for e-mail"
Self-assessment or selfauthored policy	DOCUMENTED	CAIQ, SIG Lite, internal IR policy
Independent document	DOCUMENTED	SOC 2 Type II, ISO 27001 statement of applicability, VAPT report, scanner export, IdP MFA export
Test evidence	TESTED	Backup restore test, tabletop exercise, penetration test
Outside-in scan signal	TECHNICALLY_OBSERVED	Exposed RDP; DMARC record present
Endpoint agent / fleet roll-up	TECHNICALLY_OBSERVED	EDR running with current signatures on 4 of 4 hosts

For each control the **strongest, most recent** observation wins. If sources disagree (for example a questionnaire says secure configuration is not in place but the fleet shows every host passing), the stronger evidence wins and the control is flagged so the underwriter can see the conflict. The full source trail of every control is one click away in the interface. Observations are never edited: a correction is a new observation. For the incident-response plan and backups, the questionnaire also asks whether the artefact exists, is current and has been tested, because a backup that exists but has never been restored is not the same as a tested backup.

Model V2 reads the resolved evidence as one of four **statuses** (Table 3.4). This mapping is the link between Engine A and Engine B. Fig. 3.9 shows it as a state machine. **Table 3.4** Evidence status used by the pricing model

Status	When	Pricing effect
VERIFIED	Agent, fleet or outside-in PASS; tested evidence; an independent document	Full effect of the control (m_has)

STATED	Questionnaire "yes"; a self-assessment or selfauthored policy; a partial result such as "backups recent, restore not tested"	60% of the verified credit
NOT_WORKING	Agent, fleet or scan FAIL; questionnaire "no"	Priced as lacking the control (m_lacks)
UNKNOWN	No evidence, or "don't know"	Priced as lacking the control (m_lacks)

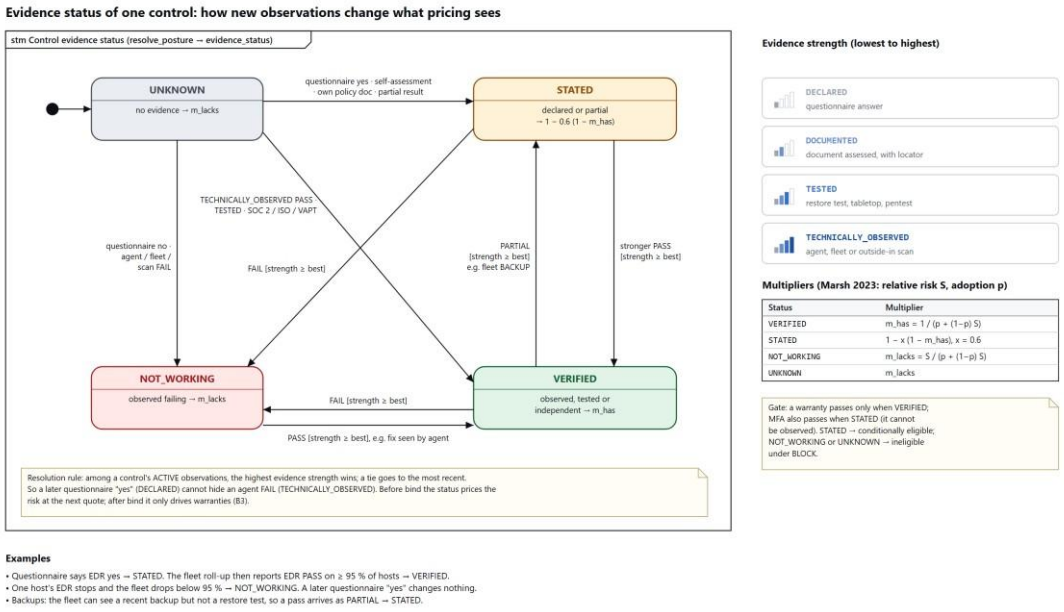


Fig. 3.9 State diagram of a control's evidence status

3.3.4 Engine B: the pricing model V2

Engine B prices a policy once, at inception. Model V2 replaces V1's additive posture score with a frequency–severity model built from published claims data. It splits the risk into five perils, prices how often each happens and how much each costs, applies the policy's retention and overall limit, and divides the expected loss by a target loss ratio. Fig. 3.10 is the flowchart of a quote.

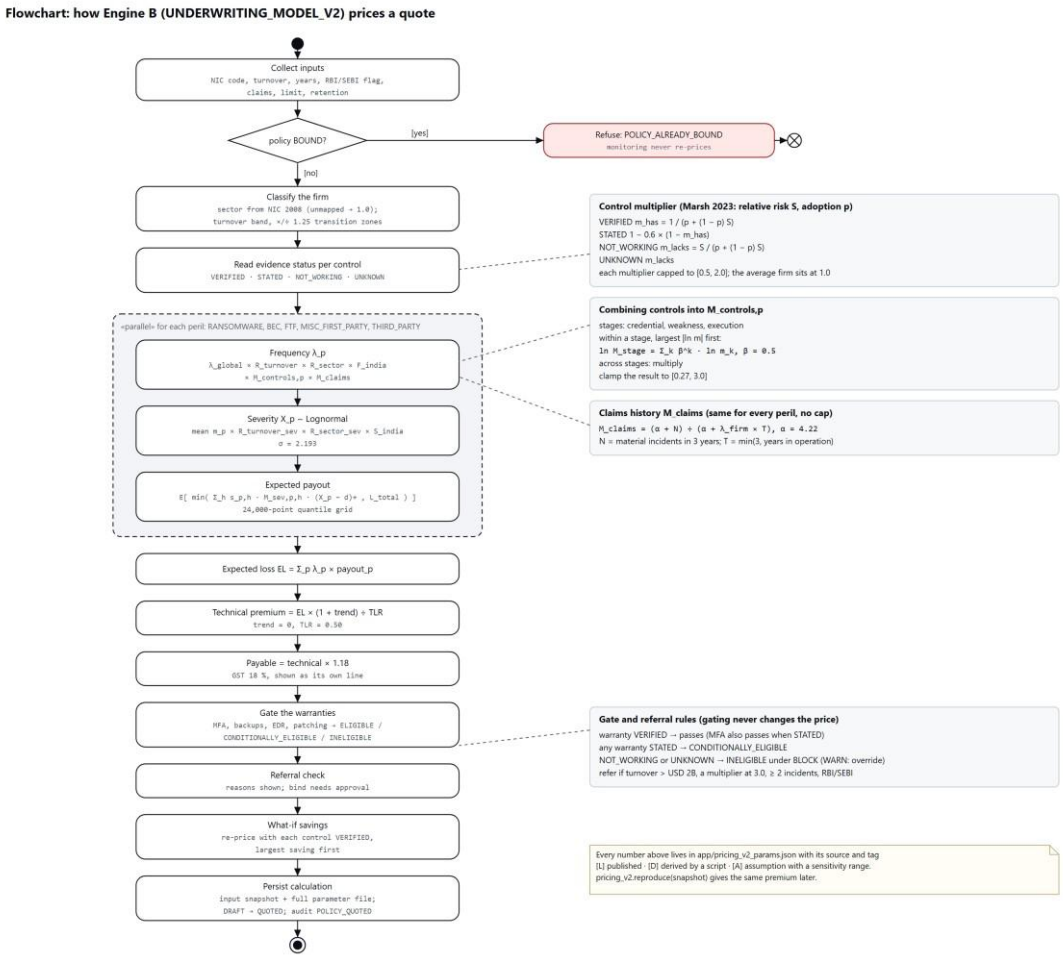


Fig. 3.10 Flowchart of a V2 quote (UML activity diagram)

Inputs. The model rates on the NIC 2008 code (two-digit division) [35], annual turnover in rupees, years in operation, whether the firm is RBI- or SEBI-regulated, the number of material incidents in the last three years, the overall limit and retention, and the evidence status of each priced control. Employees, endpoint counts and record counts are kept for the record but not priced. Source data is in US dollars and is converted at a placeholder rate of ₹88 per dollar. **Frequency.** For each peril p the annual frequency is $\lambda_p = \lambda_{p,global} \times R_turnover,freq \times R_sector,freq \times F_india,p \times M_controls,p \times M_claims$ (3.1)

where $\lambda_{p,global}$ is Coalition's 2025 frequency for the peril [1] (Table 3.5), the two relativities come from the firm's turnover band and sector, F_india is kept as a parameter but set to 1 because no Indian claims data is used yet, and the last two factors are the control and claims multipliers described below.

Table 3.5 Perils, base frequencies and mean losses (Coalition 2025)

Peril	λ global	Mean loss m_p	Source locator
Ransomware	0.31%	$292,000	Figs 6.1, 6.2
Business email compromise (BEC)	0.44%	$35,000	Figs 4.1, 4.2
Funds transfer fraud (FTF)	0.44%	$185,000 (before clawbacks)	Figs 5.1, 5.2
Other first-party	0.17%	$49,000	Figs 7.1, 7.2
Third-party liability	0.02%	$23,000	Figs 8.1, 8.2

The five perils sum to 1.38%; adding the 6.2% of claims Coalition files as "other" gives 1.47%, against its published all-claims frequency of 1.48%. The turnover bands and their values are in Table 3.6. To avoid a cliff at each band boundary b, within a factor of 1.25 either side the value moves log-linearly from the lower band's value to the upper one's:

$$f(R) = f_{lo} \times (f_{hi} / f_{lo}) \wedge [\ \ln(R / (b / 1.25)) / \ln(1.25^2)\ ] \qquad (3.2)$$

Table 3.6 Turnover bands

Band (USD)	Frequency	Average loss	Tag
< 10M	1.07%	$79,000	[A] frequency: Coalition's < $25M rate applied to < $10M; [L] severity
10M – 25M	1.19%	$79,000	[D] 1.07% ÷ 0.90, the 0.90 from IRIS 2025 [18]
25M – 100M	3.99%	$139,000	[L] Coalition 2025 Figs 3.1–3.6
≥ 100M	5.97%	$228,000	[L]

The relativities are the band value divided by Coalition's global frequency (1.48%) and average loss ($115,000). The NIC division maps to a rating sector (for example NIC 62 → Information Technology, 86 → Healthcare, 28 → Industrials) whose frequency and severity relativities come from Coalition's sector figures; three sectors missing there come from IRIS 2025 and NetDiligence 2025 [17]. A missing or unmapped NIC code is priced at the global average.

Severity and payout. Each peril's loss X is lognormal with mean $m_p \times R_{turnover,sev} \times R_{sector,sev}$ and a shared shape σ = 2.193. The shape was fitted to NetDiligence's 2025 SME claims: mean $264,000 and 395 of 8,936 claims above $1M. These two conditions have two solutions; σ = 2.193 also predicts the observed share above $500K (8.26% against 8.24%), while σ = 1.214 predicts 12.86%, so the first is kept (Section

4.2). Each loss is split across coverage heads h in fixed shares s_p,h, and the policy pays the loss above the retention d up to the overall limit L:

$$\text{payout}_p = E[\ \min(\ \Sigma_h\ s_{p,h} \cdot M_sev,p,h \cdot (X_p - d)^+\ ,\ L\ )\ ] \qquad (3.3)$$

There are no per-cover sub-limits: every cover pays its share above the retention and the overall limit is the only cap. The head shares still matter, because the severity controls (tested backups, incident response plan, training, encryption) act on the heads they affect. The expectation is integrated numerically over 24,000 quantiles of the lognormal (20,000 below the 99th percentile and 4,000 above) and checked against the closed-form limited expected value

$$LEV(u) = e^{(\mu+\sigma^2/2)} \cdot \Phi((\ln u - \mu - \sigma^2)/\sigma) + u \cdot (1 - \Phi((\ln u - \mu)/\sigma)) \qquad (3.4)$$

For one head with share 1, the grid matches LEV(d + L) − LEV(d) to about one part in 10^7.

Controls. For a control with relative risk S (claim rate without it divided by the rate with it) and adoption p, the multipliers for having and lacking it are

$$m_has = 1 / (p + (1 - p)\cdot S) \qquad m_lacks = S / (p + (1 - p)\cdot S) \qquad (3.5)$$

so that p·m_has + (1 − p)·m_lacks = 1: a firm with the average mix of controls pays the base rate. Each multiplier is capped to [0.5, 2.0]. S and p for the frequency controls are from Marsh McLennan's 2023 study [24] (Table 3.7); MFA's adoption is an assumption. The severity controls are tested backups (S = 8.0, from Sophos [25]), the incidentresponse plan and training (S = 1.055 each, from IBM 2023 [26]) and encryption (S = 1.101, from IBM 2019 [27]).

Table 3.7 Frequency controls (Marsh McLennan 2023)

Control	S	p	Perils	Attack stage
Secure configuration	5.58	0.96	Ransomware, other firstparty	Weakness
Privileged access management	2.92	0.35	Ransomware, other firstparty	Execution
EDR	2.23	0.82	Ransomware, other firstparty	Execution
Logging & monitoring	2.19	0.88	Ransomware, other firstparty	Execution
Vulnerability management (high	2.19	0.24	Ransomware, other firstparty	Weakness

CVEs ≤ 7 days)				
Critical patching (critical CVEs ≤ 7 days)	1.57	0.53	Ransomware, other firstparty	Weakness
Control	**S**	**p**	**Perils**	**Attack stage**
Security awareness (phishing simulation)	1.76	0.89	Ransomware, BEC, FTF	Credential
Email security (attachment sandboxing)	1.56	0.75	Ransomware, BEC, FTF	Credential
MFA	1.40	0.70 [A]	Ransomware, BEC, FTF	Credential

A VERIFIED control gets m_has, NOT_WORKING and UNKNOWN get m_lacks, and a STATED control gets part of the verified credit:

$$m_stated = 1 - x \cdot (1 - m_has), \quad x = 0.6 \tag{3.6}$$

The credit x is derived, not chosen. Marsh measured S on declared controls. If a share f of declarations are hollow, the true relative risk satisfies $1/S_true = (1/S - f)/(1 - f)$, and a declaration is worth $(1 - m_has(S)) / (1 - m_has(S_true))$ of the true credit. Averaged over the Marsh controls this gives x = 0.81, 0.62 and 0.43 for f = 0.1, 0.2 and 0.3; the model uses f = 0.2 [A].

Controls that defend the same attack stage overlap, so their effects are damped. Within a stage the multipliers are sorted by |ln m|, largest first, and combined as

$$\ln M_stage = \Sigma_k \beta^k \cdot \ln m_k, \quad \beta = 0.5 \tag{3.7}$$

The stage results then multiply, and the combined multiplier for a peril is clamped to [0.27, 3.0]: the floor is Coalition's finding that its policyholders have 73% fewer claims than the industry, the ceiling its finding that end-of-life software is associated with about three times as many claims [1].

Claims history. Material incidents in the last three years (a loss above the retention, a DPDP or CERT-In notification, or a ransom decision) enter through a credibility multiplier applied to every peril:

$$M_claims = (\alpha + N) / (\alpha + \lambda_firm \cdot T), \quad \alpha = 4.22 \tag{3.8}$$

where N is the number of material incidents, T = min(3, years in operation) and λ_firm is the firm's total frequency after classification and controls. This is the Poisson–gamma credibility form [29]. The value α = 4.22 comes from a negative-binomial fit to IRIS 2022 incident counts (P(0) = 87.1%, P(1) = 11.3%, α_total = 1.09), with the

heterogeneity explained by controls and sector removed [19]. At $\lambda T = 0.03$ it gives multipliers of 0.99, 1.23 and 1.46 for zero, one and two incidents.

Premium. The expected loss is $EL = \Sigma_p \lambda_p \times payout_p$, and

$$\text{Technical premium} = EL \times (1 + \text{trend}) / TLR, \quad \text{trend} = 0, \quad TLR = 0.50 \qquad (3.9)$$

The target loss ratio of 0.50 lies between Fitch's nine-year average for US standalone cyber (48%) [41] and NAIC's latest figure (57%) [42]. The breakdown shows it as 50% expected loss, 30% expenses (the IRDAI 2023 expense cap, commission included [34]) and 20% for profit, risk margin, reinsurance and accumulation. GST at 18% is added as its own line; the bound premium is the technical premium.

Gating and referral. MFA, tested backups, EDR and critical patching are warranties. A warranty passes when VERIFIED (MFA also when STATED, because the platform cannot verify MFA and the questionnaire addendum carries a misrepresentation declaration). A warranty that is only STATED makes the quote CONDITIONALLY_ELIGIBLE; one that is NOT_WORKING or UNKNOWN makes it INELIGIBLE under the default gate mode BLOCK. Gating never changes the price. A quote is referred, so binding needs an underwriter's approval, when turnover exceeds USD 2 billion, a peril's control multiplier is at the 3.0 ceiling, the firm has two or more material incidents, or it is RBI- or SEBI-regulated.

Explainability and reproducibility. Every quote stores and returns the classification, the per-peril chain, every control's multiplier with its status and source, the payout per head, the loadings, the gate and referral reasons, the list of assumptions used, and a what-if list that re-prices the quote with each control in turn set to VERIFIED. The snapshot stores the inputs and the whole parameter file, so

`pricing_v2.reproduce(snapshot)` gives the same premium later, whatever has changed since. Python floats are IEEE 754 binary64 [43] and the calculation has no random element, so the same inputs always give the same premium. The algorithm is:

```
Algorithm 3.1  calculate_premium(policy)
  if policy.status == BOUND: raise POLICY_ALREADY_BOUND        # monitoring never
reprices
  prm    <- load pricing_v2_params.json   firm   <- classify(NIC code,
turnover, years, regulated)     # sector, bands, transition zones
  status <- {c: evidence_status(resolve_posture(c)) for each priced control c}   for
each peril p in {RANSOMWARE, BEC, FTF, MISC_FIRST_PARTY, THIRD_PARTY}:       M_ctl[p]
<- clamp( prod over stages ( exp( sum_k beta^k * ln m_k ) ), 0.27, 3.0
)       lam[p]   <- lam_global[p] * R_turn_f * R_sec_f * F_india[p] *
M_ctl[p]   M_claims <- (alpha + N) / (alpha + sum(lam) * min(3, years))
for each peril p:
      lam[p]   <- lam[p] * M_claims
      mu       <- ln(m[p] * R_turn_s * R_sec_s * S_india) - sigma^2 / 2
```

```
      pay[p]   <- E[ min( sum_h s[p,h] * M_sev[p,h] * max(X - d, 0), L ) ],  X ~
LN(mu, sigma)
  EL <- sum_p lam[p] * pay[p];   technical <- EL * (1 + trend) / TLR;   gst <- 0.18 *
technical
  gate <- warranties(status);   referral <- triggers(firm, M_ctl, N, regulated)
what_if <- for each control not VERIFIED: re-price with it VERIFIED, record the
saving   store calculation with input snapshot, parameter file and model version;
audit POLICY_QUOTED
```

3.3.5 Eligibility, binding and the policy life cycle

A policy moves through the states shown in Fig. 3.11: DRAFT, QUOTED after a calculation, and BOUND when an underwriter binds it at a stored calculation. Binding refuses a policy that is already bound or a calculation that does not belong to the policy, refuses an INELIGIBLE quote under gate mode BLOCK, and refuses a referred quote unless the underwriter has approved the referral. At bind the warranties are created, all MET, and evaluated once against the current evidence. Fig. 3.12 shows the sequence of a quote and a bind across the modules.

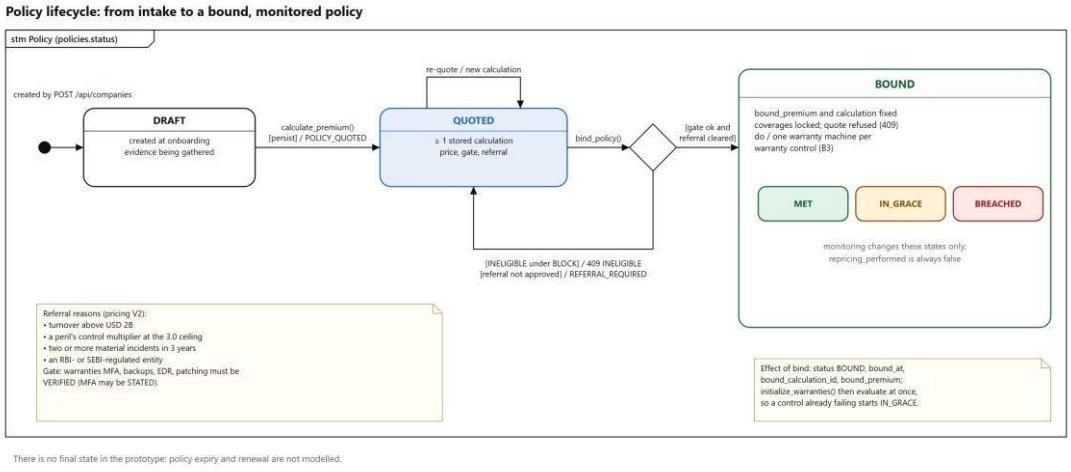


Fig. 3.11 State diagram of a policy

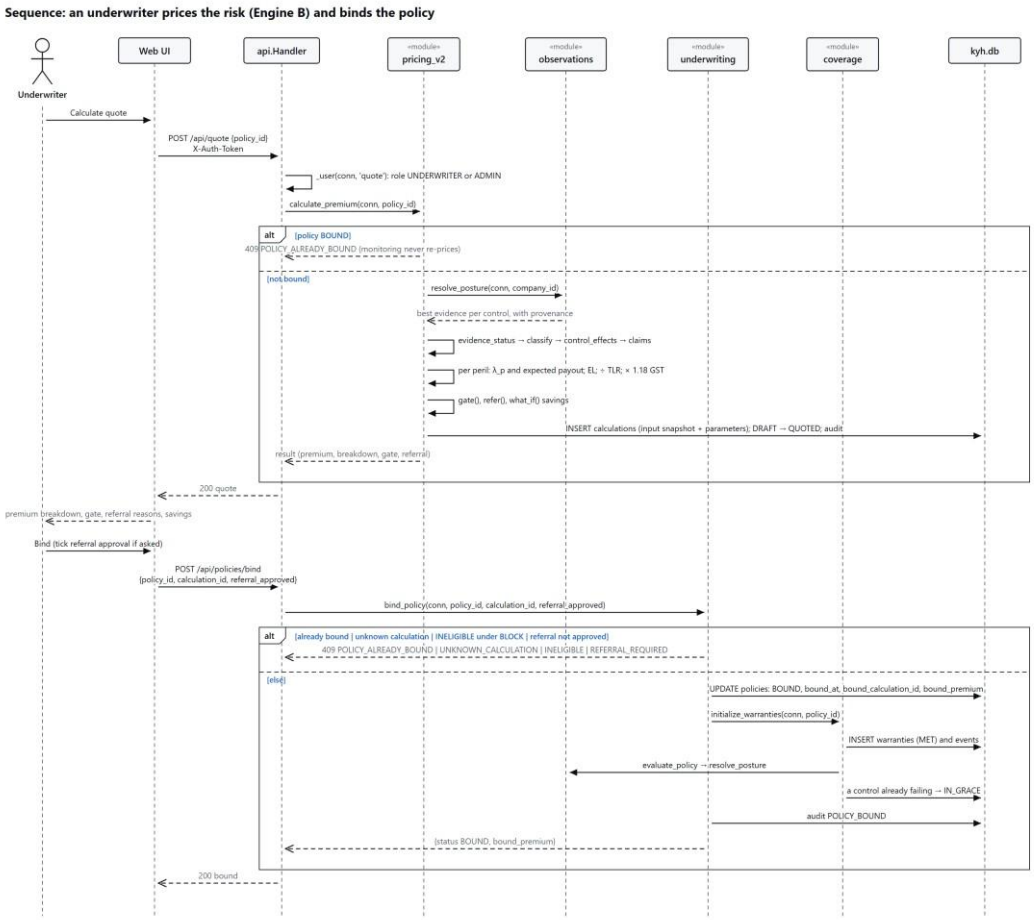


Fig. 3.12 Sequence diagram: quote and bind

3.3.6 The warranty state machine

After bind, each warranty control is in one of four states (Fig. 3.13). A failing result moves a MET warranty to IN_GRACE and starts its grace period (14 days for patching, 7 for EDR, backups and MFA, 14 for monitoring coverage). If the control is still failing when the grace period ends, the warranty is BREACHED, and the configured coverage consequence applies to claims from then on (for example, data-restoration claims subject to 50% co-insurance while the backup warranty is breached; these consequences are illustrative, not policy wording). A passing result moves an IN_GRACE or BREACHED warranty to REMEDIATED and then to MET. Missing evidence is never a breach. Every transition is written to the warranty event log and the audit log. Grace periods are whole days, so the demonstration uses a `fast-forward` command, which is itself audited as DEMO_TIME_TRAVEL.

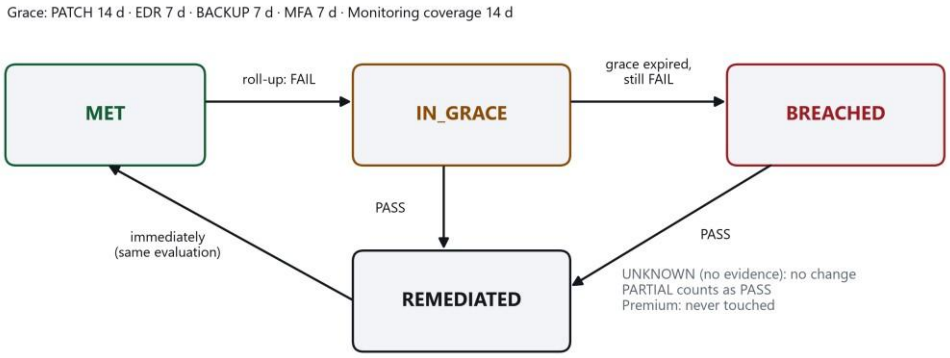
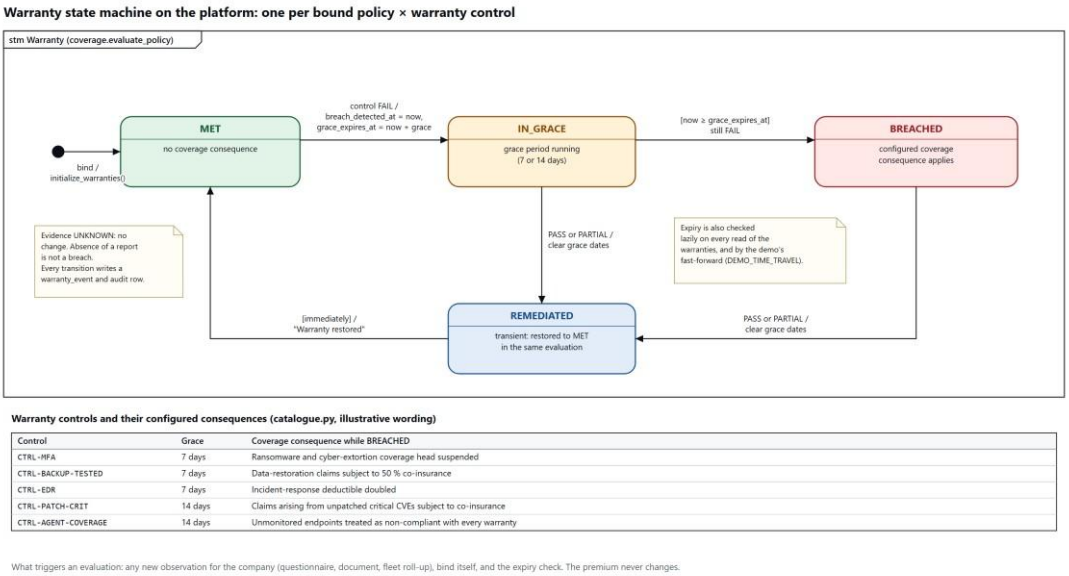


Fig. 3.13 Warranty states on the platform and the grace period of each warranty



Warranty controls and their configured consequences (catalogue.py, illustrative wording)

Control	Grace	Coverage consequence while BREACHED
CTRL-MFA	7 days	Ransomware and cyber-extortion coverage head suspended
CTRL-BACKUP-TESTED	7 days	Data-restoration claims subject to 50 % co-insurance
CTRL-EDR	7 days	Incident-response deductible doubled
CTRL-PATCH-CRIT	14 days	Claims arising from unpatched critical CVEs subject to co-insurance
CTRL-AGENT-COVERAGE	14 days	Unmonitored endpoints treated as non-compliant with every warranty

What triggers an evaluation: any new observation for the company (questionnaire, document, fleet roll-up), bind itself, and the expiry check. The premium never changes.

Fig. 3.14 UML state machine of a platform warranty

3.3.7 Part 2: the endpoint agent

The agent is a small Python program installed on each of the insured's machines. It reads settings and status from the operating system, never changes anything, and sends a signed report. It has no window on the endpoint; it runs as a Windows scheduled task as SYSTEM, a systemd service on Linux or a launchd daemon on macOS. An administrator can run `once --no-send` to see the results locally, and `show-policy` to see exactly what is checked and what is never collected. The agent is built in ten layers (Fig. 3.15); each layer talks only to its neighbours, which is why the checks can be tested on recorded output without a real Windows machine.

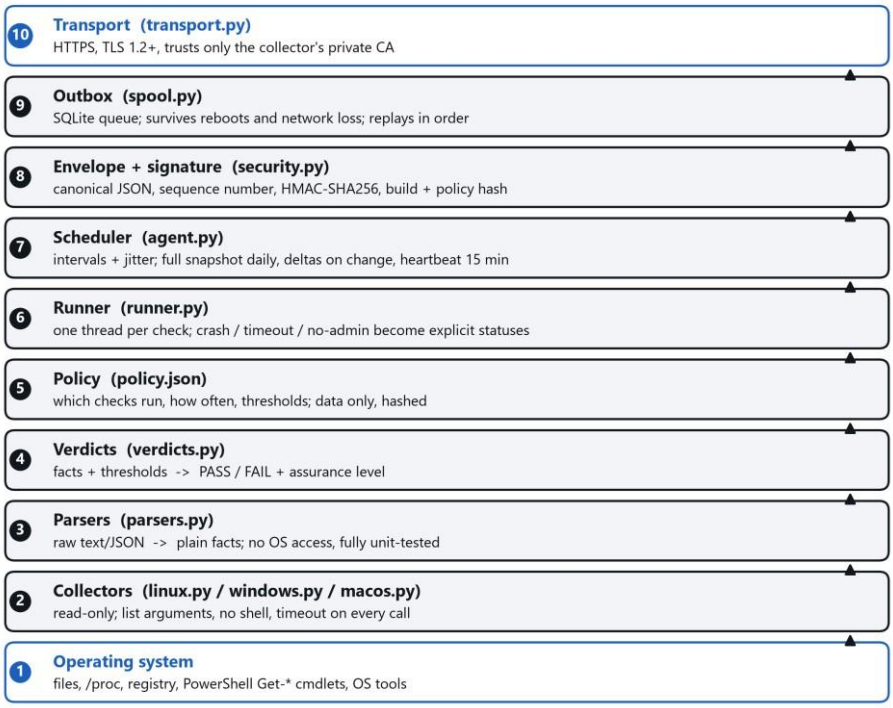


Fig. 3.15 Agent layers, from the operating system (1) to the network (10)

Checks. The policy groups twelve checks into three tiers (Table 3.8). Warranty checks feed the coverage warranties; the two regulatory checks cover the CERT-In obligations on log retention and time synchronisation [4]; the rest are hygiene checks stored as evidence.

Table 3.8 The twelve agent checks

Control (tier)	Windows	Linux	macOS
PATCH (warranty)	Windows Update Agent search, Get-HotFix	apt-get -s upgrade, dpkg.log; dnf updateinfo	softwareupdate -l
EDR (warranty)	Get-Process, Get-MpComputerStatus	/proc process table; mdatp health	ps process table
BACKUP (warranty)	agent marker; Get-WBSummary; backup processes	agent marker; backup processes	marker; tmutil latestbackup
MFA (warranty)	not observable: NOT_SUPPORTED	same	same
DISK_ENCRYPTION	Get-BitLockerVolume; manage-bde	lsblk: dm-crypt under /	fdesetup status
FIREWALL	Get-NetFirewallProfile	ufw, firewalld, nftables, iptables	socketfilterfw
EXPOSED_SERVICES	Get-NetTCPConnection State Listen	/proc/net/tcp, tcp6	netstat -anv

REMOTE_ACCESS	registry: RDP, NLA	sshd -T or sshd_config with Includes	sshd_config
Control (tier)	**Windows**	**Linux**	**macOS**
ADMIN_ACCOUNTS	Administrators group, Guest account	sudo/wheel/admin groups, extra UID 0	dscl admin group
OS_SUPPORT	build number vs lifecycle table	/etc/os-release vs lifecycle table	major version
LOG_RETENTION (regulatory)	oldest Security/System event; forwarder	oldest log or journal entry; forwarder	forwarder process
TIME_SYNC (regulatory)	w32tm /query /status	timedatectl, chronyc, timesyncd	systemsetup

Every OS command goes through one function that takes an argument list (never a shell string), applies a timeout, forces the C locale and hides the console window. Each verdict returns a status (PASS, FAIL, NOT_APPLICABLE, NOT_SUPPORTED or ERROR) and an assurance level (OPERATING, RUNNING, PRESENT or ABSENT), so a Defender installation with nine-day-old signatures is a FAIL with assurance PRESENT, not a PASS. For V2, the patch check passes only if the machine was patched within seven days, matching Marsh's "critical CVEs patched within 7 days". MFA is reported as NOT_SUPPORTED because it is enforced by the identity provider, not the laptop.

Reports and their protection. A report is a JSON envelope with a kind (full snapshot daily, delta on any change, heartbeat every 15 minutes), the agent's identity, a sequence number, the build hash and policy hash, and the results. The agent serialises it with sorted keys and computes HMAC-SHA256 [44], [45] over the exact bytes with its 256bit secret. The sequence number only increases, so an old "all PASS" report cannot be replayed. The build hash is a SHA-256 over the agent's code and policy, so an agent edited to always pass is detected. Reports are written to an outbox before sending and sent over TLS 1.2 or later to a collector whose CA is pinned [46]. Fig. 3.16 shows the classes of the agent and Fig. 3.17 the enrolment and reporting sequence. Fig. 3.18 traces one check from a Windows setting to the platform.

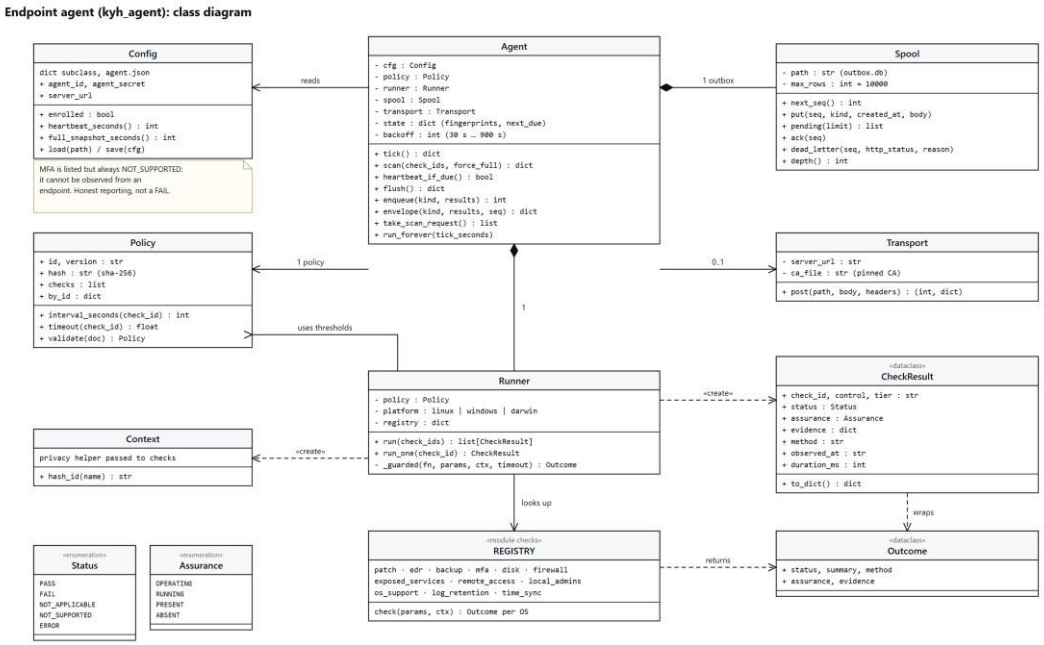


Fig. 3.16 UML class diagram of the agent

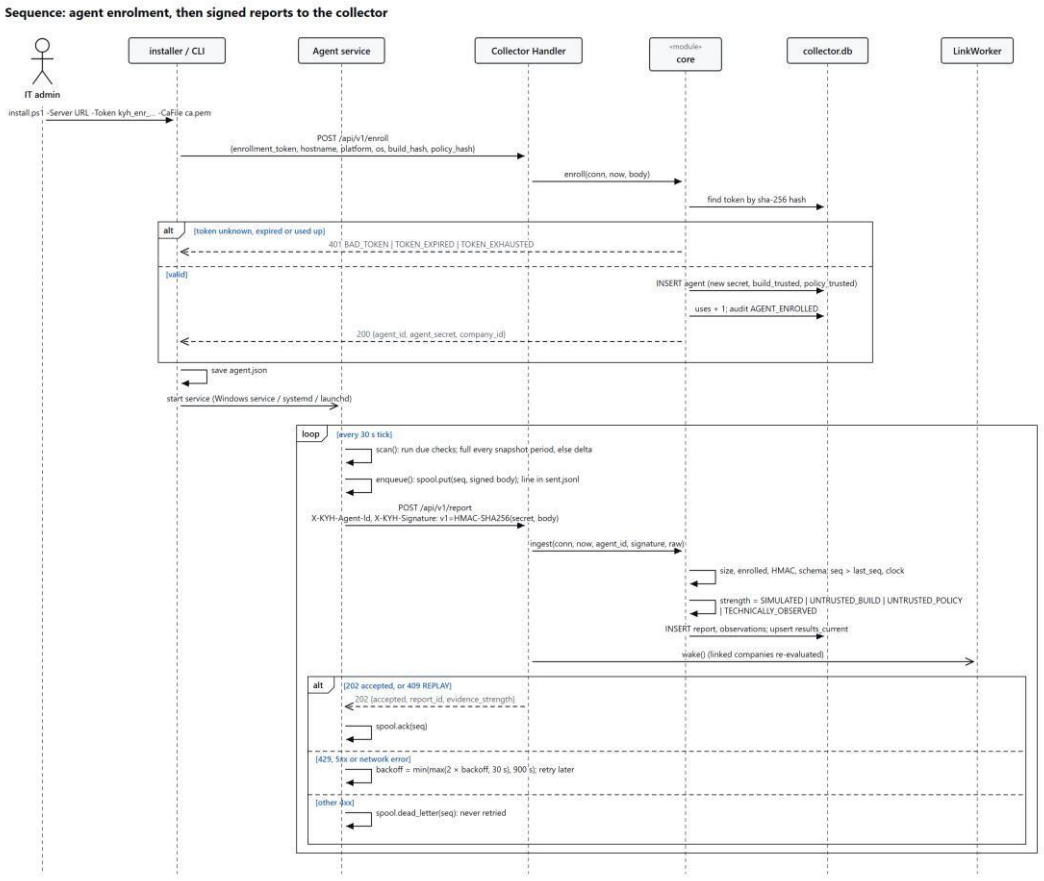


Fig. 3.17 Sequence diagram: agent enrolment and a signed report

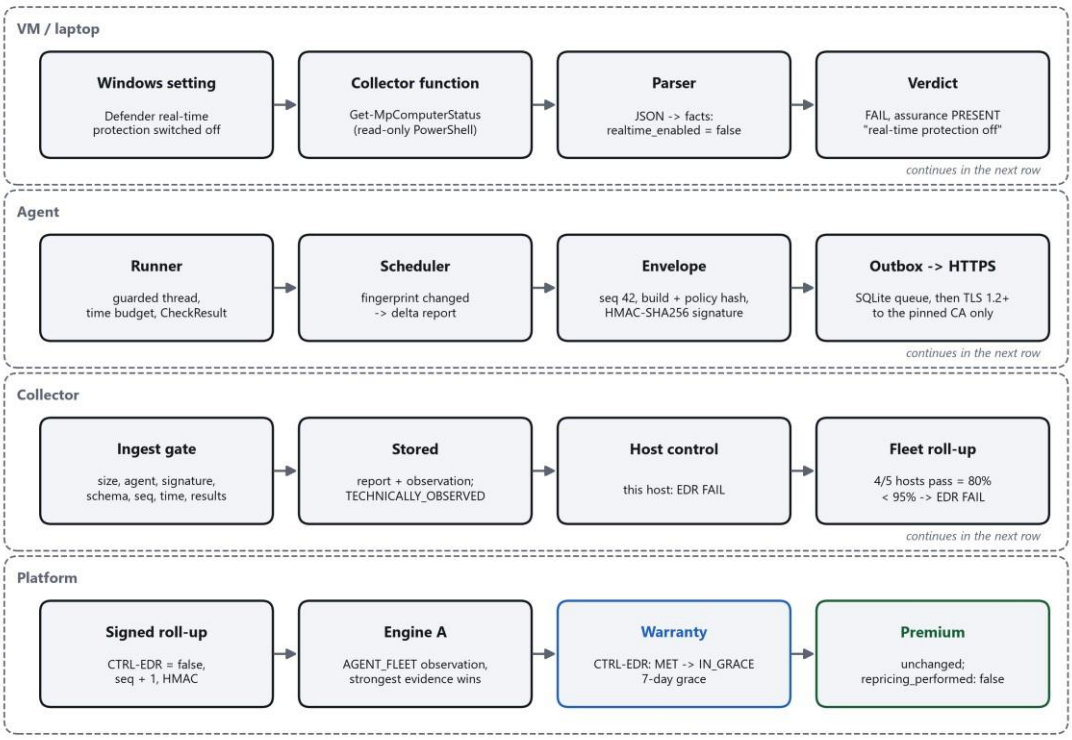


Fig. 3.18 One check from the Windows setting to the platform warranty

3.3.8 Part 2: the collector

The collector is the insurer-side server for the agents. An overview is in Fig. 3.19. It enrols agents with limited-use tokens, checks every report, stores accepted reports unchanged with their SHA-256, and rolls the results up per company.

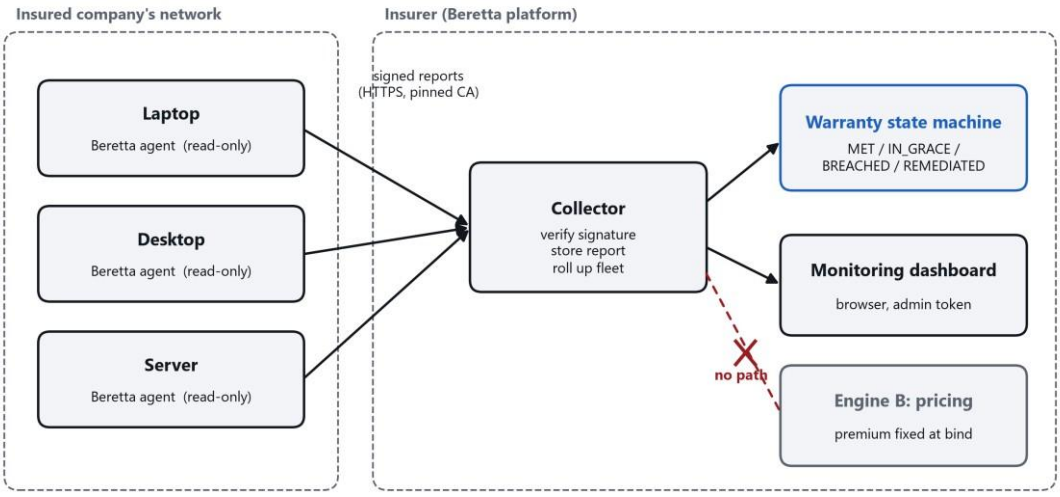


Fig. 3.19 Collector overview: agents, ingestion, roll-up, warranties and the dashboard

Ingestion. A report is checked in a fixed order before anything is stored (Fig. 3.20): size, known agent, signature, schema, sequence, timestamp and result format. Signature failures and replays are written to the audit log as security events. Each stored result is

labelled TECHNICALLY_OBSERVED only if the build hash and policy hash are on the approved list; otherwise it is stored as UNTRUSTED_BUILD or UNTRUSTED_POLICY and ignored. Simulated reports are accepted only for companies created in demo mode.

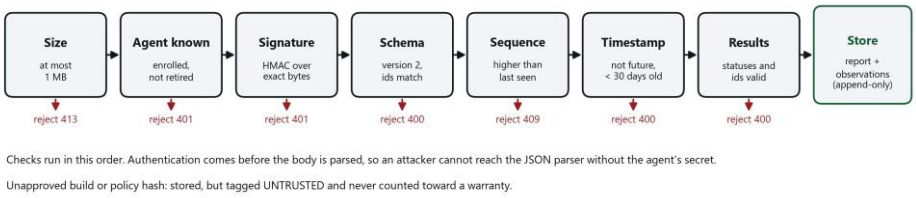


Fig. 3.20 The ingestion gate: each check must pass before the next runs

Roll-up and the silence rule. Each host is ONLINE, UNVERIFIED after six hours without a report, or SILENT after 72 hours (Fig. 3.21). A silent host counts as a FAIL for every control it used to report, which defeats the simplest way to hide a failing machine: uninstalling the agent. A host fails a control if any of its checks for that control fails. For the company, only hosts that could observe the control count, and the control passes if at least 95% of them pass (Fig. 3.22). If no host can observe a control, as for MFA, its status is NOT_OBSERVABLE and nothing is sent for it.



Fig. 3.21 The silence rule

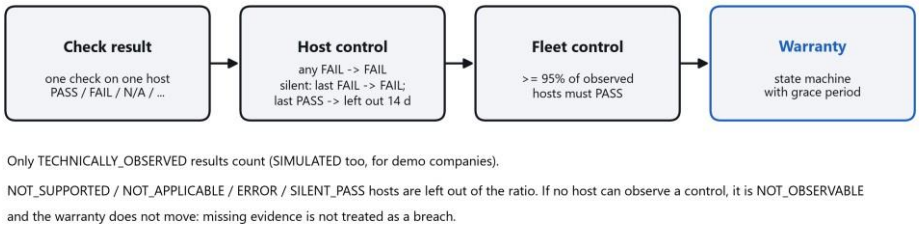


Fig. 3.22 Roll-up from a check result to a company control

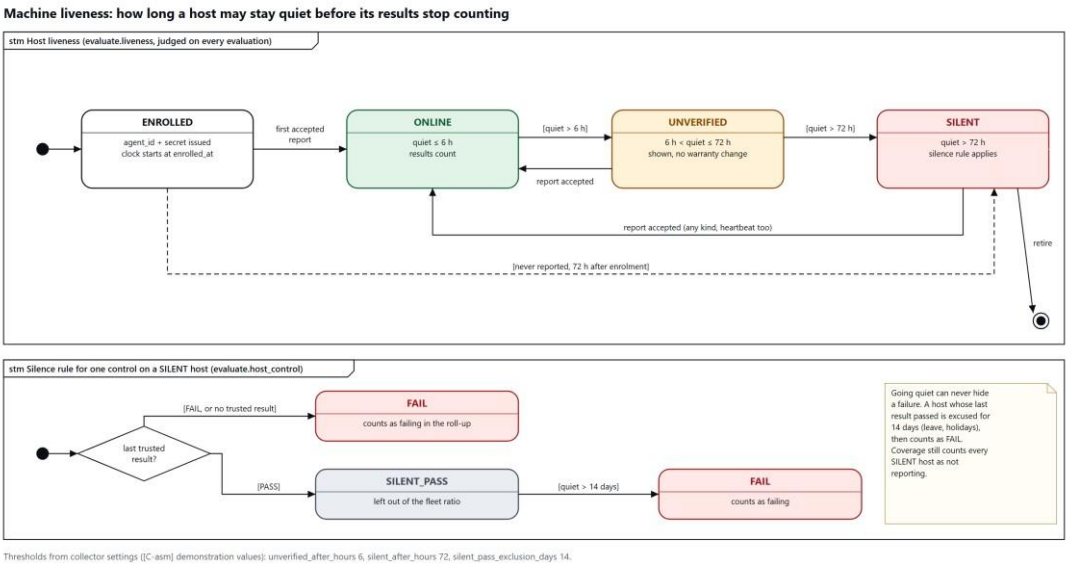


Fig. 3.23 UML state machine of host liveness

A company could enrol only its best machines, so the collector adds a monitoringcoverage warranty: at least 90% of the endpoints declared on the proposal form must be reporting, with 14 days after bind to roll the agent out. For companies that are not linked to the platform, the collector runs its own warranty state machine (Fig. 3.24, Fig. 3.25), in which a fix must hold for 24 hours before REMEDIATED returns to MET. For linked companies it runs none and shows the platform's states. The collector's classes and database are in Fig. 3.26 and Fig. 3.27.

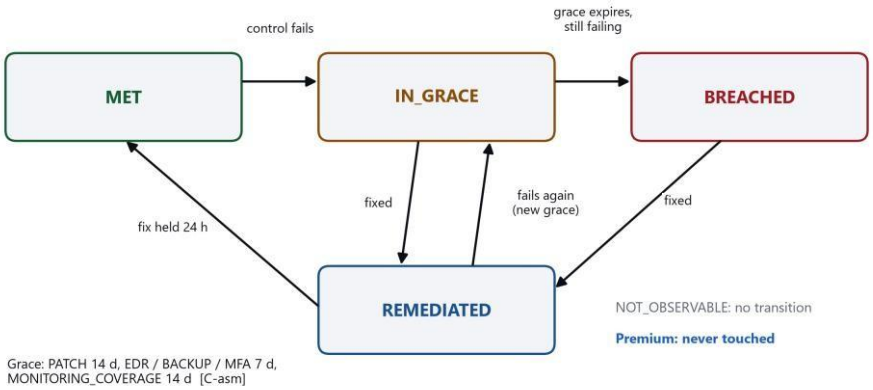


Fig. 3.24 Collector warranty states (stand-alone mode)

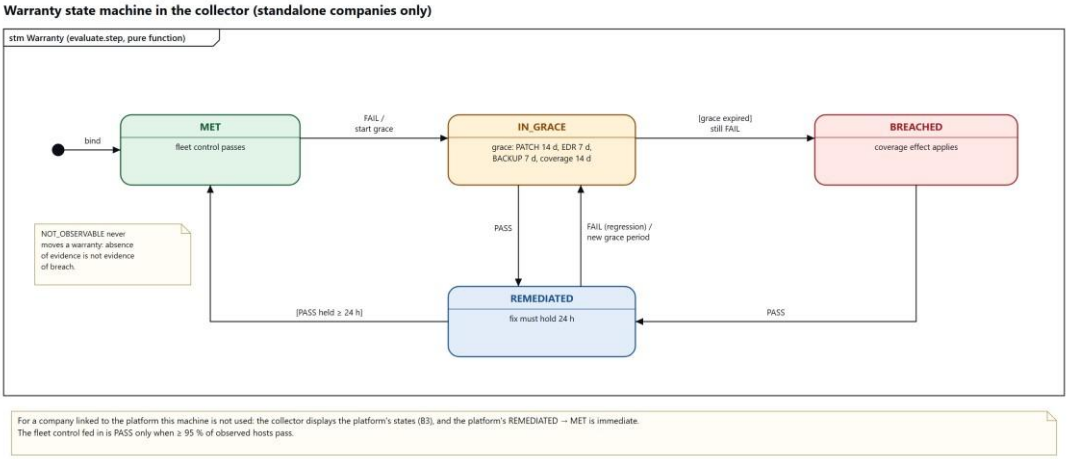


Fig. 3.25 UML state machine of a collector warranty (stand-alone mode)

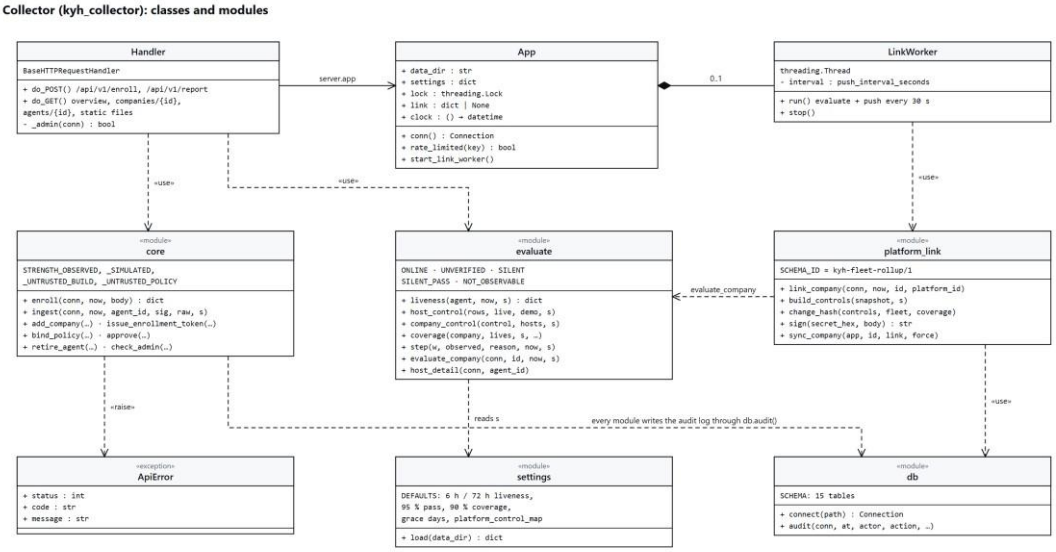


Fig. 3.26 UML class diagram of the collector

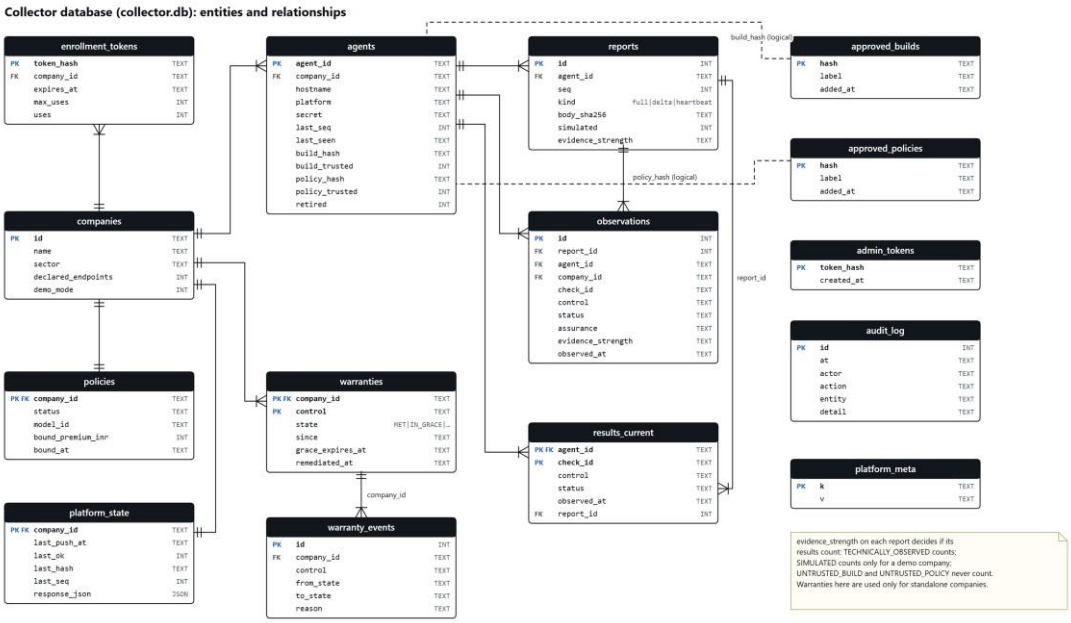


Fig. 3.27 Entity–relationship diagram of the collector database

3.3.9 Linking Part 2 to Part 1

The link gives each part one job. The collector owns everything about endpoints; the platform owns everything about the policy and runs the only warranty state machine for linked companies. The collector maps its twelve controls to the platform's controls (Table 3.9) and sends one signed roll-up per company when a result changes and as a heartbeat every 15 minutes.

Table 3.9 Mapping of collector controls to platform controls

Collector control	Platform control	Rule
PATCH	CTRL-PATCH-CRIT (warranty)	
EDR	CTRL-EDR (warranty)	
BACKUP	CTRL-BACKUP-TESTED (warranty)	A pass is sent as *partial*: restore testing is not observable, so it reads as STATED
MFA	CTRL-MFA (warranty)	Never sent: not observable from endpoints
DISK_ENCRYPTION	CTRL-ENCRYPTION	
FIREWALL, REMOTE_ACCESS, EXPOSED_SERVICES, OS_SUPPORT	CTRL-SECCONFIG	Fails if any source fails
ADMIN_ACCOUNTS	CTRL-PAM	Changed for V2, which prices PAM separately
LOG_RETENTION, TIME_SYNC	CTRL-LOGGING	The two CERT-In checks

MONITORING_COVERAGE	CTRL-AGENT-COVERAGE (monitoring only)	Never priced, never gates a quote

The platform checks each roll-up before storing it: the collector must be registered, the HMAC-SHA256 over the body must match its secret, the sequence number must be higher than the last one accepted, the timestamp must be within five minutes, and the company must be linked to that collector. Each result becomes an AGENT_FLEET observation at TECHNICALLY_OBSERVED strength. Before bind it feeds the next quote like any other evidence; after bind it reaches only the warranty state machine. Fig. 3.28 shows the two jobs, and Fig. 3.29 and Fig. 3.30 the sequence of one roll-up.

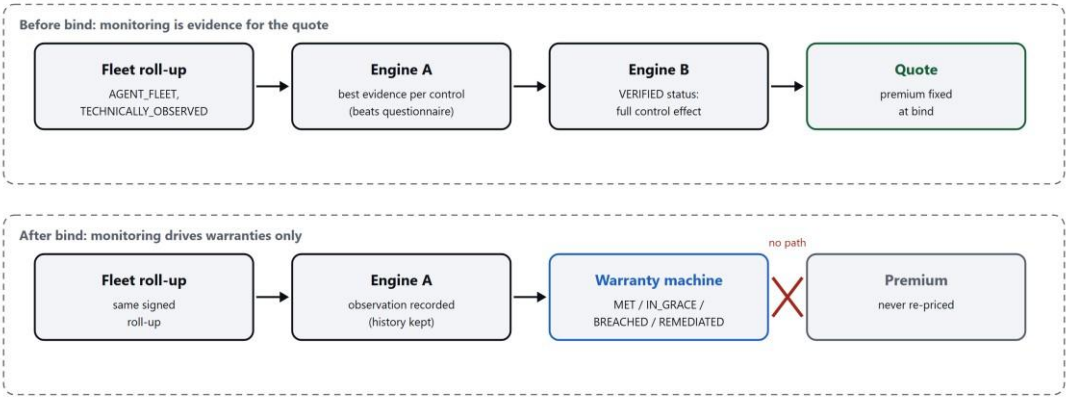


Fig. 3.28 What monitoring does before and after bind

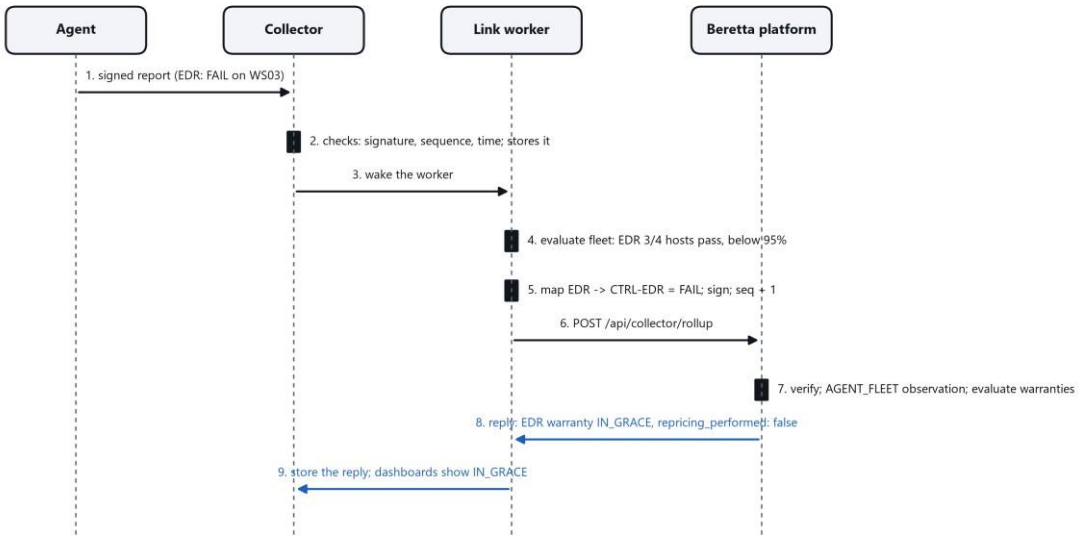


Fig. 3.29 What happens when EDR stops on one of four machines after bind

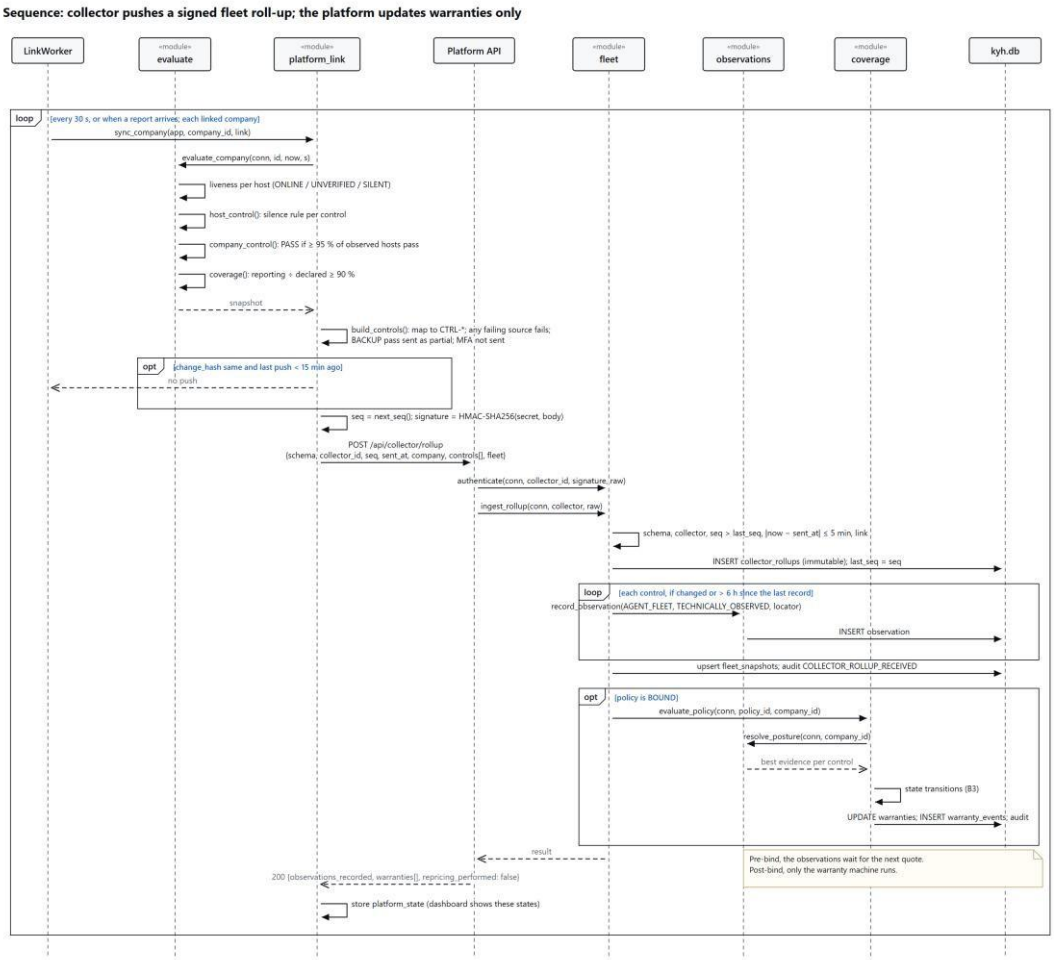


Fig. 3.30 UML sequence diagram of the Part 2 → Part 1 roll-up

3.3.10 Roles and access control

The platform has three roles (Table 3.10). Role separation is enforced on the server: every route checks the caller's permission, and for the insured customer every read is limited to its own company, so asking for another company's records returns "not found". The interface follows the same rules, but hiding a button is never the only protection.

Table 3.10 Roles and permissions

Role	Permissions	Can do
Insured customer	read (own company only)	See its profile, classification, evidence per control, documents and claims on record, premium, eligibility and what would lower the premium, policy and warranty states, and the Monitoring tab. Cannot edit anything, see other companies, the portfolio, the audit log or the model configuration.
Underwriter	read, staff, intake, quote, bind, assess	Enter intake, record claims, documents and signals, assess documents, price, approve referrals and bind, review warranties and the audit log

Role	Permissions	Can do
Administrator	all of the above + admin	Edit model configuration and gate mode, register collectors and link companies

Fig. 3.31 and Fig. 3.32 are the use-case diagrams for the insured and the insurer.

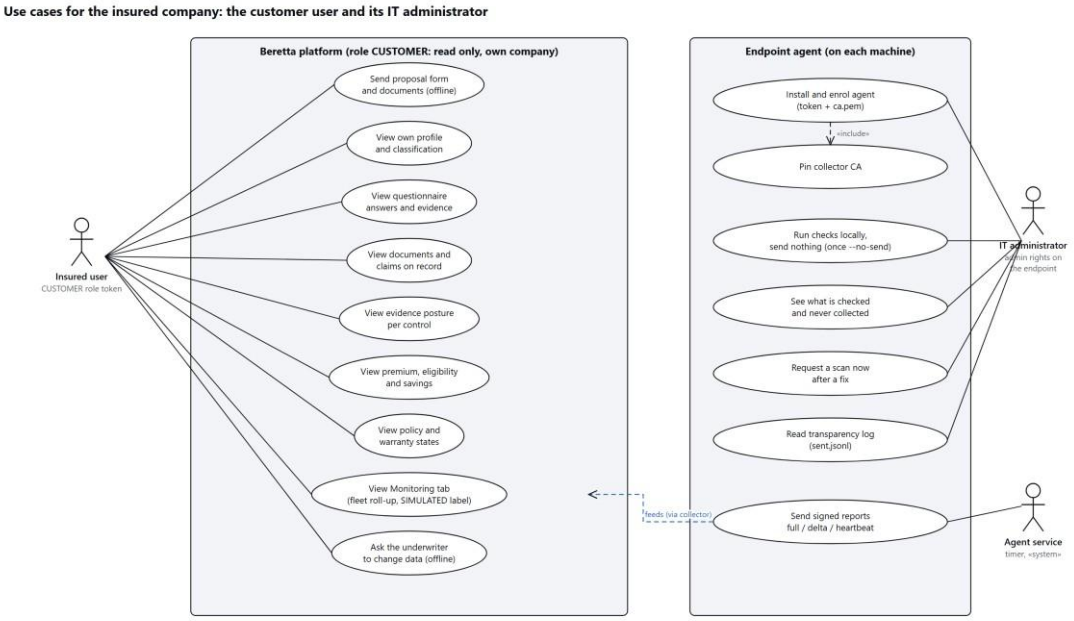


Fig. 3.31 Use-case diagram for the insured company

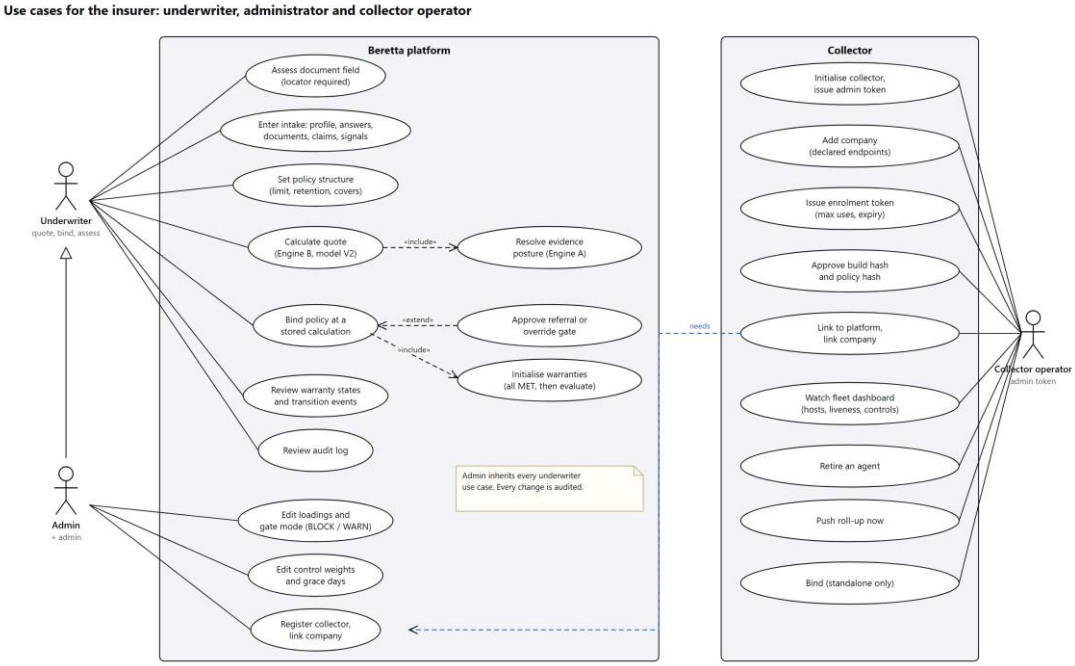


Fig. 3.32 Use-case diagram for the insurer

3.3.11 Security and privacy design

Table 3.11 lists the main threats to the system and how each is handled. **Table 3.11** Threats and countermeasures

Threat	Countermeasure
Forged or altered agent report	HMAC-SHA256 over the exact bytes with a per-agent 256-bit secret; rejected and audited on mismatch
Replay of an old passing report	Monotonic sequence number per agent, kept across restarts; timestamp window
Modified agent that always passes	Build hash of code and policy; results from an unapproved build are not trusted
Uninstalling the agent from a failing machine	Silence rule: a host silent for 72 hours fails every control it used to report; monitoring-coverage warranty
Fake collector or TLS interception	Agent trusts only the collector's pinned CA; TLS 1.2 or later
Forged roll-up to the platform	HMAC-SHA256 with the collector's secret, sequence check, fiveminute window, company must be linked
Pushing code to endpoints through the policy	The policy file can only choose from checks compiled into the agent and set thresholds
Command injection in checks	Commands passed as argument lists, never shell strings; read-only commands only
Customer editing its own data or seeing others'	Server-side permissions; customer reads scoped to its company; all writes need staff permissions
SQL injection, oversized requests, abuse	Parameterised SQL throughout; 1 MB body cap; per-token rate limit; input validation with structured errors
Script injection in the collector dashboard	Strict Content-Security-Policy: no inline scripts or styles, no external resources

For privacy, the agent never collects file contents, browsing history, keystrokes, e-mail, passwords or clear-text user names, and it writes a local log of every report it sends so the insured's staff can see exactly what left the machine. Uploaded documents are stored as metadata with assessed fields and locators; their contents are never opened or run.

3.3.12 User interface

Both dashboards share one visual style: ink on a blue-grey ground, white cards, colour only on states and on the main figure of a screen, and the evidence ladder (four ascending bars) on every evidence chip to show its strength. Labels are in plain language and say what a state means and what happens next. Fig. 3.33 to Fig. 3.40 show the platform as the underwriter sees it, and Fig. 3.41 and Fig. 3.42 as the insured customer sees it.

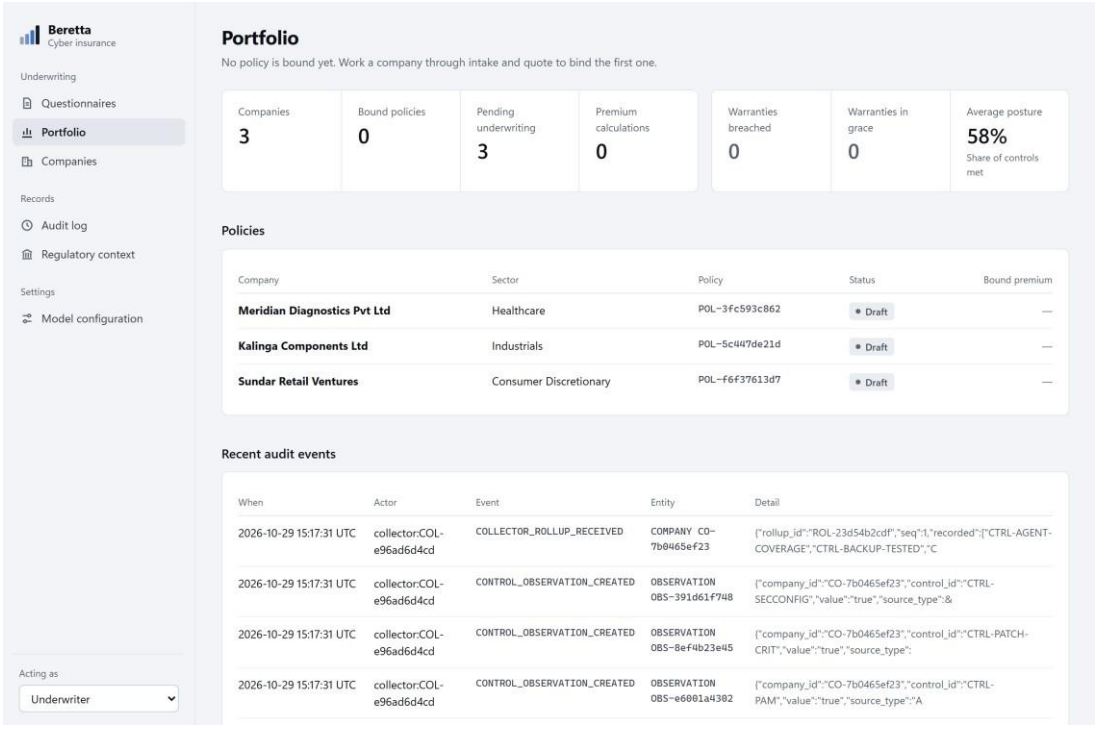


Fig. 3.33 Platform: portfolio dashboard (underwriter)

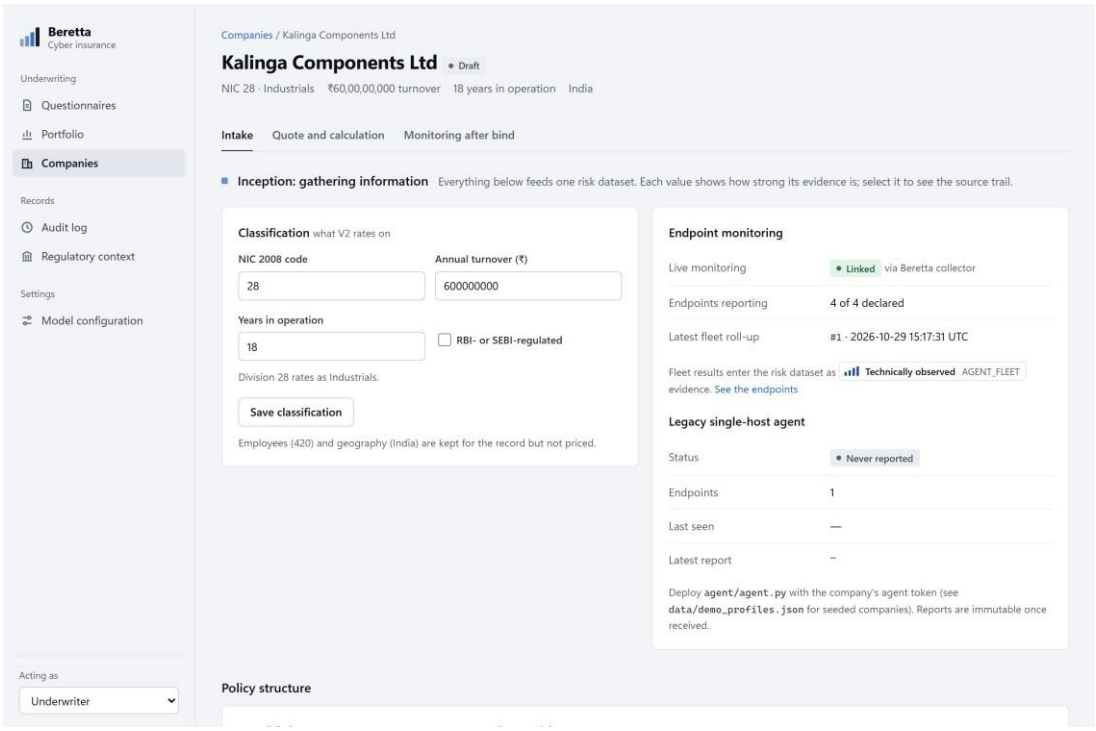


Fig. 3.34 Platform: company intake, with classification and endpoint monitoring

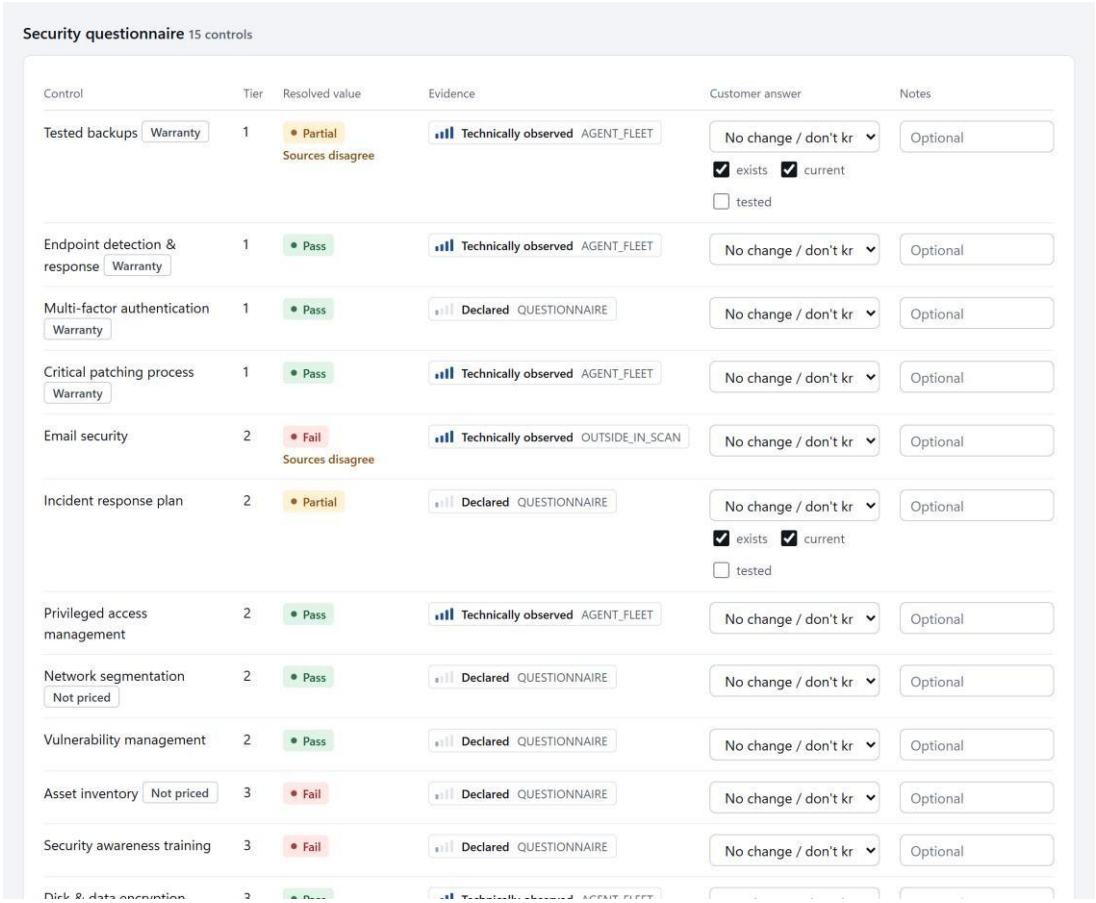


Fig. 3.35 Platform: security questionnaire with resolved value and evidence per control

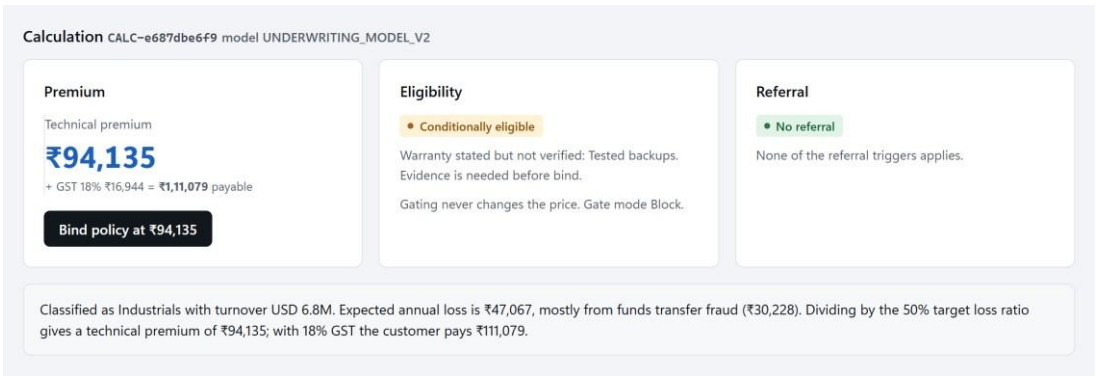
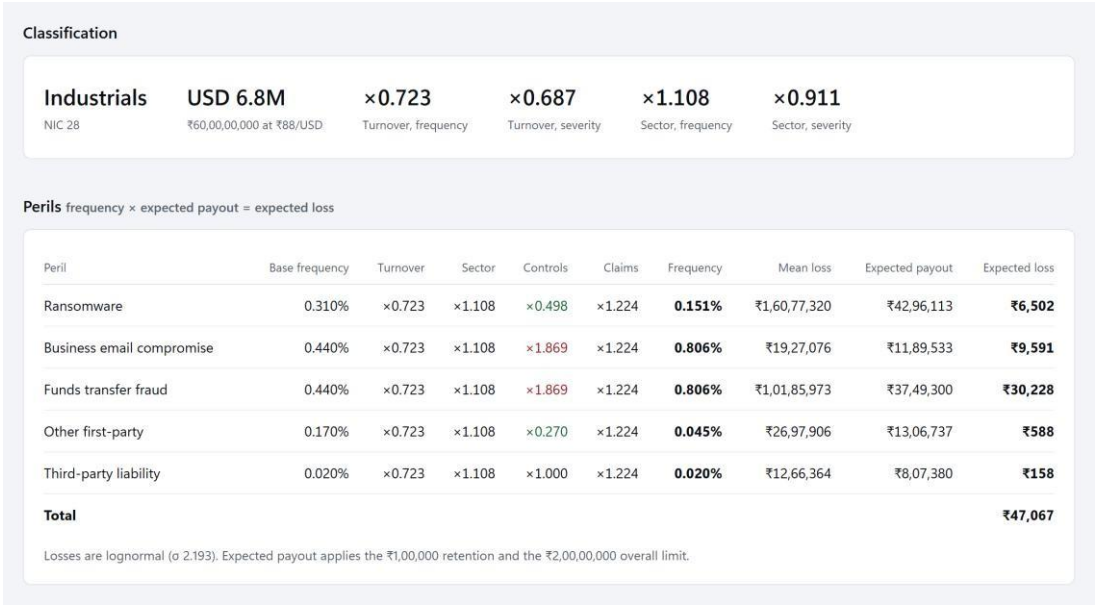


Fig. 3.36 Platform: quote summary with premium, eligibility and referral



Classification

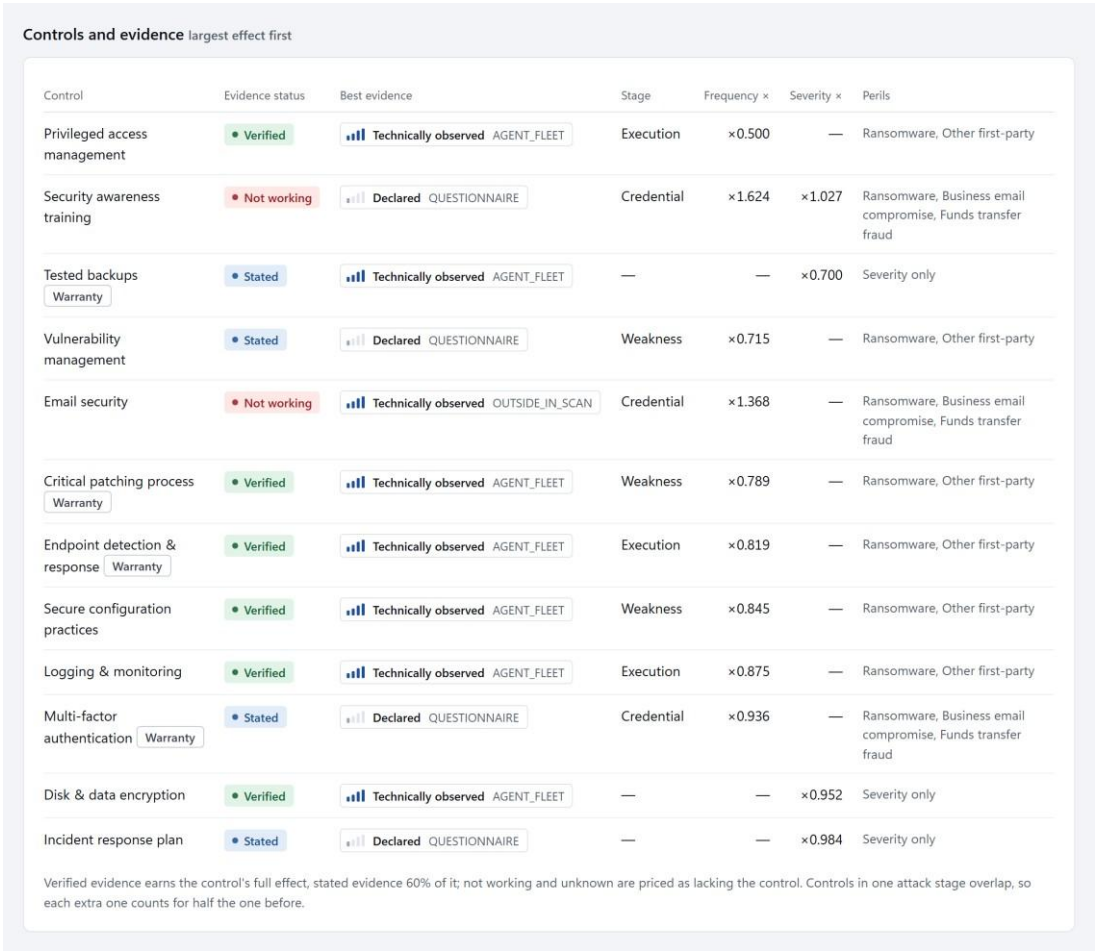
Industrials	USD 6.8M	×0.723	×0.687	×1.108	×0.911
NIC 28	₹60,00,00,000 at ₹88/USD	Turnover, frequency	Turnover, severity	Sector, frequency	Sector, severity

Perils frequency × expected payout = expected loss

Peril	Base frequency	Turnover	Sector	Controls	Claims	Frequency	Mean loss	Expected payout	Expected loss
Ransomware	0.310%	×0.723	×1.108	×0.498	×1.224	**0.151%**	₹1,60,77,320	₹42,96,113	**₹6,502**
Business email compromise	0.440%	×0.723	×1.108	×1.869	×1.224	**0.806%**	₹19,27,076	₹11,89,533	**₹9,591**
Funds transfer fraud	0.440%	×0.723	×1.108	×1.869	×1.224	**0.806%**	₹1,01,85,973	₹37,49,300	**₹30,228**
Other first-party	0.170%	×0.723	×1.108	×0.270	×1.224	**0.045%**	₹26,97,906	₹13,06,737	**₹588**
Third-party liability	0.020%	×0.723	×1.108	×1.000	×1.224	**0.020%**	₹12,66,364	₹8,07,380	**₹158**
Total									**₹47,067**

Losses are lognormal (σ 2.193). Expected payout applies the ₹1,00,000 retention and the ₹2,00,00,000 overall limit.

Fig. 3.37 Platform: classification and the per-peril frequency and payout



Controls and evidence largest effect first

Control	Evidence status	Best evidence	Stage	Frequency ×	Severity ×	Perils
Privileged access management	Verified	Technically observed AGENT_FLEET	Execution	×0.500	—	Ransomware, Other first-party
Security awareness training	Not working	Declared QUESTIONNAIRE	Credential	×1.624	×1.027	Ransomware, Business email compromise, Funds transfer fraud
Tested backups Warranty	Stated	Technically observed AGENT_FLEET	—	—	×0.700	Severity only
Vulnerability management	Stated	Declared QUESTIONNAIRE	Weakness	×0.715	—	Ransomware, Other first-party
Email security	Not working	Technically observed OUTSIDE_IN_SCAN	Credential	×1.368	—	Ransomware, Business email compromise, Funds transfer fraud
Critical patching process Warranty	Verified	Technically observed AGENT_FLEET	Weakness	×0.789	—	Ransomware, Other first-party
Endpoint detection & response Warranty	Verified	Technically observed AGENT_FLEET	Execution	×0.819	—	Ransomware, Other first-party
Secure configuration practices	Verified	Technically observed AGENT_FLEET	Weakness	×0.845	—	Ransomware, Other first-party
Logging & monitoring	Verified	Technically observed AGENT_FLEET	Execution	×0.875	—	Ransomware, Other first-party
Multi-factor authentication Warranty	Stated	Declared QUESTIONNAIRE	Credential	×0.936	—	Ransomware, Business email compromise, Funds transfer fraud
Disk & data encryption	Verified	Technically observed AGENT_FLEET	—	—	×0.952	Severity only
Incident response plan	Stated	Declared QUESTIONNAIRE	—	—	×0.984	Severity only

Verified evidence earns the control's full effect, stated evidence 60% of it; not working and unknown are priced as lacking the control. Controls in one attack stage overlap, so each extra one counts for half the one before.

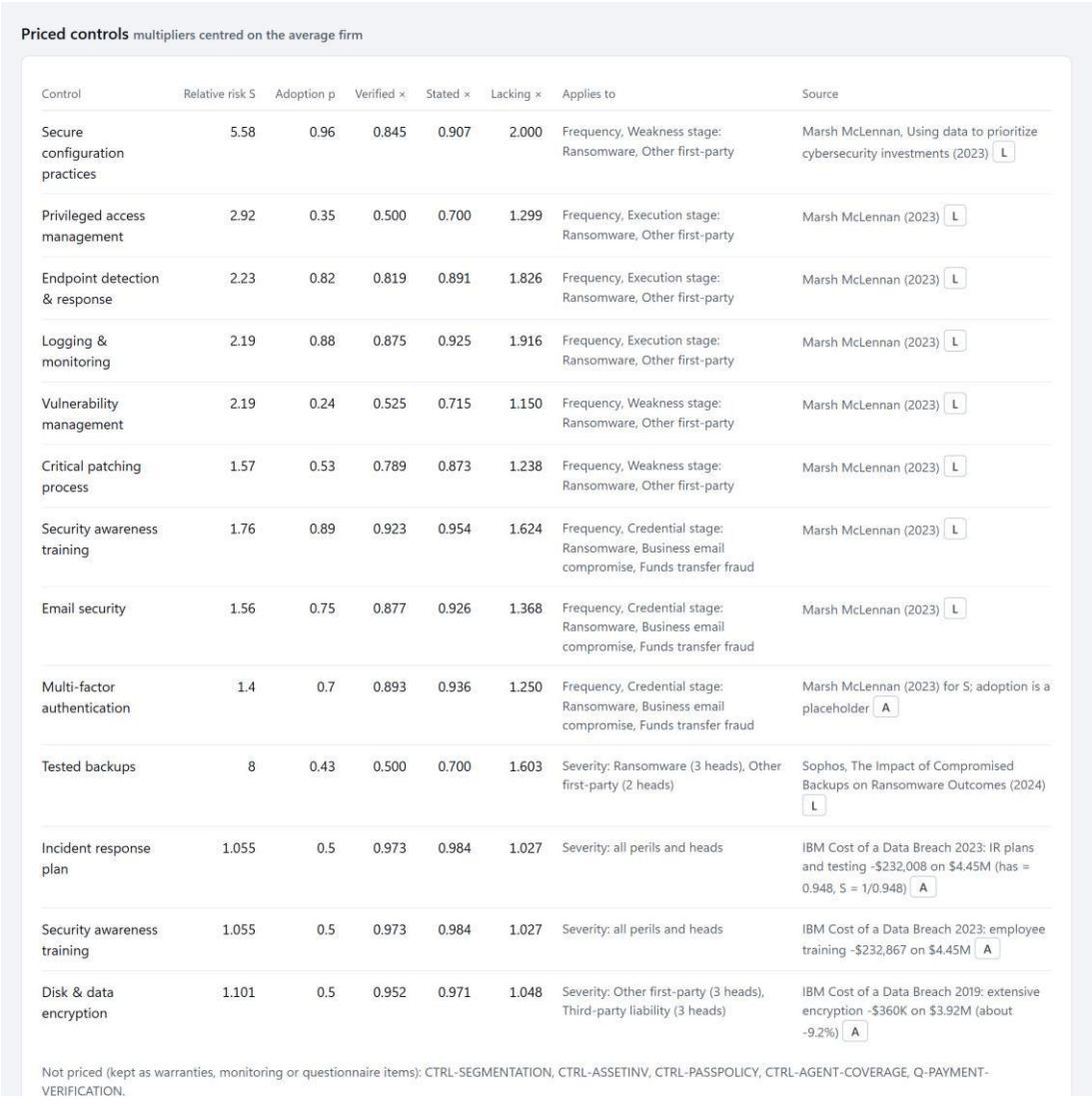
Fig. 3.38 Platform: controls and evidence, largest effect first



From expected loss to premium

Expected loss	₹47,067 50%
Expenses, commission included (IRDAI cap)	₹28,240 30%
Profit, risk margin, reinsurance and accumulation	₹18,827 20%
Technical premium = expected loss ÷ 0.5	**₹94,135**
GST 18%	₹16,944
Payable	**₹1,11,079**

Fig. 3.39 Platform: from expected loss to premium



Priced controls multipliers centred on the average firm

Control	Relative risk S	Adoption p	Verified ×	Stated ×	Lacking ×	Applies to	Source
Secure configuration practices	5.58	0.96	0.845	0.907	2.000	Frequency, Weakness stage: Ransomware, Other first-party	Marsh McLennan, Using data to prioritize cybersecurity investments (2023) L
Privileged access management	2.92	0.35	0.500	0.700	1.299	Frequency, Execution stage: Ransomware, Other first-party	Marsh McLennan (2023) L
Endpoint detection & response	2.23	0.82	0.819	0.891	1.826	Frequency, Execution stage: Ransomware, Other first-party	Marsh McLennan (2023) L
Logging & monitoring	2.19	0.88	0.875	0.925	1.916	Frequency, Execution stage: Ransomware, Other first-party	Marsh McLennan (2023) L
Vulnerability management	2.19	0.24	0.525	0.715	1.150	Frequency, Weakness stage: Ransomware, Other first-party	Marsh McLennan (2023) L
Critical patching process	1.57	0.53	0.789	0.873	1.238	Frequency, Weakness stage: Ransomware, Other first-party	Marsh McLennan (2023) L
Security awareness training	1.76	0.89	0.923	0.954	1.624	Frequency, Credential stage: Ransomware, Business email compromise, Funds transfer fraud	Marsh McLennan (2023) L
Email security	1.56	0.75	0.877	0.926	1.368	Frequency, Credential stage: Ransomware, Business email compromise, Funds transfer fraud	Marsh McLennan (2023) L
Multi-factor authentication	1.4	0.7	0.893	0.936	1.250	Frequency, Credential stage: Ransomware, Business email compromise, Funds transfer fraud	Marsh McLennan (2023) for S; adoption is a placeholder A
Tested backups	8	0.43	0.500	0.700	1.603	Severity: Ransomware (3 heads), Other first-party (2 heads)	Sophos, The Impact of Compromised Backups on Ransomware Outcomes (2024) L
Incident response plan	1.055	0.5	0.973	0.984	1.027	Severity: all perils and heads	IBM Cost of a Data Breach 2023: IR plans and testing -$232,008 on $4.45M (has = 0.948, S = 1/0.948) A
Security awareness training	1.055	0.5	0.973	0.984	1.027	Severity: all perils and heads	IBM Cost of a Data Breach 2023: employee training -$232,867 on $4.45M A
Disk & data encryption	1.101	0.5	0.952	0.971	1.048	Severity: Other first-party (3 heads), Third-party liability (3 heads)	IBM Cost of a Data Breach 2019: extensive encryption -$360K on $3.92M (about -9.2%) A

Not priced (kept as warranties, monitoring or questionnaire items): CTRL-SEGMENTATION, CTRL-ASSETINV, CTRL-PASSPOLICY, CTRL-AGENT-COVERAGE, Q-PAYMENT-VERIFICATION.

Fig. 3.40 Platform: model configuration (administrator), priced controls with their sources

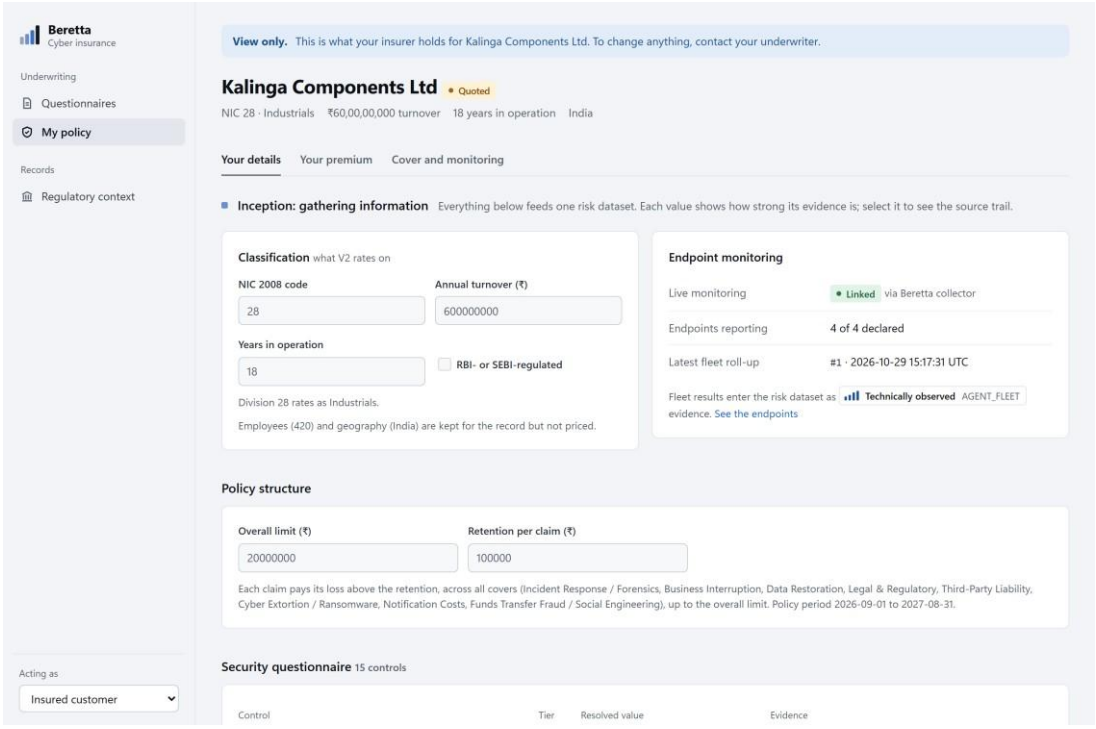


Fig. 3.41 Platform as the insured customer: read-only view of its own company

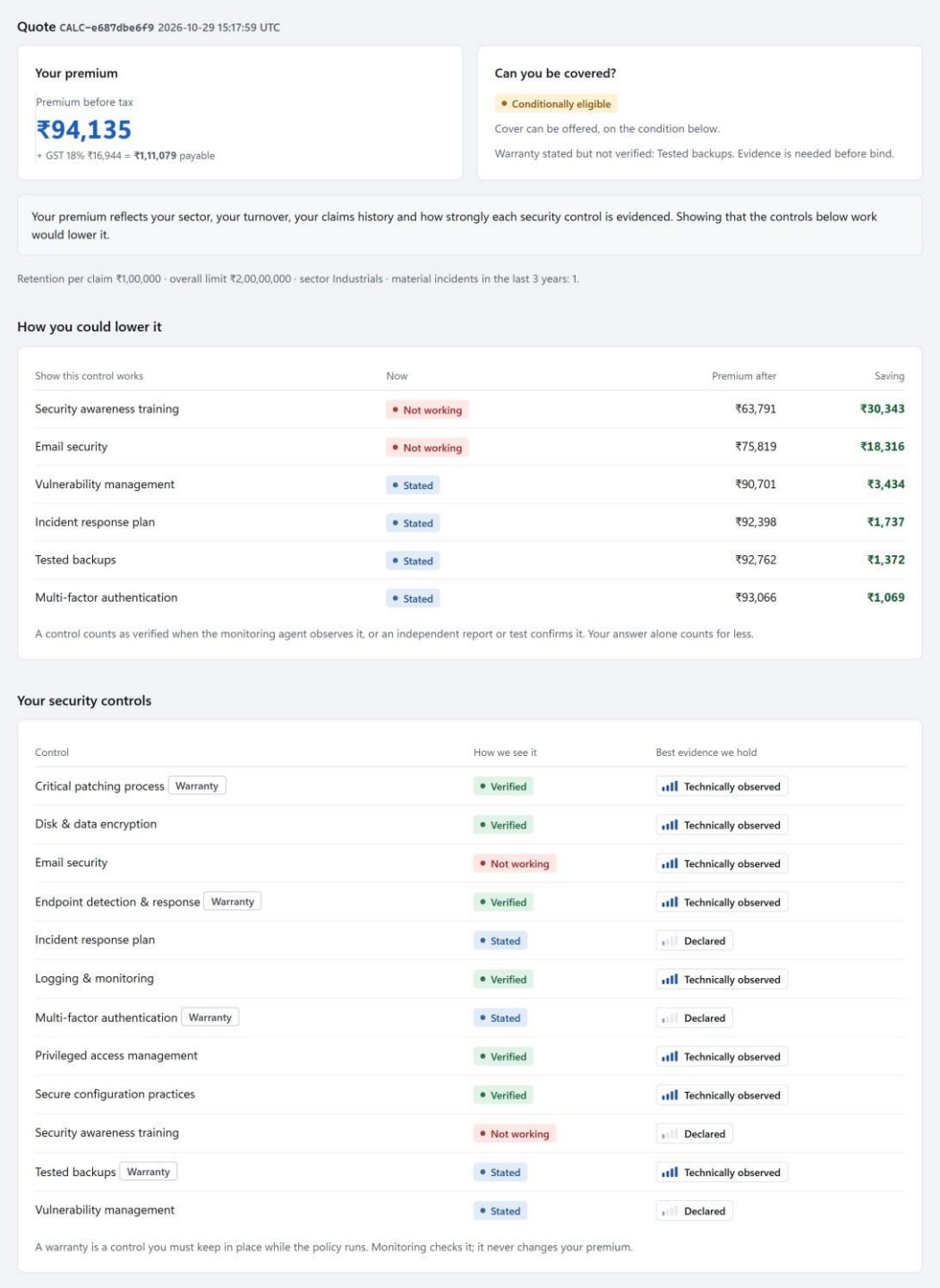


Fig. 3.42 Platform as the insured customer: premium, eligibility, savings and evidence per control

3.4 Implementation

The system was implemented and run as described above. Table 3.12 gives the size of each part. The tests are described in Chapter 4.

Table 3.12 Size of the implementation (lines, including comments)

Part	Folder	Lines

Platform application	kyh-cyber-insurance/app	2,908
Platform dashboard	kyh-cyber-insurance/frontend	1,654
Model derivation and example scripts	kyh-cyber-insurance/scripts	468
Platform tests	kyh-cyber-insurance/tests	1,053
Endpoint agent	kyh-agent/agent	2,642
Collector and its dashboard	kyh-agent/collector	1,904
Agent and collector tests	kyh-agent/tests	1,716
Integration test	integration	178
UML figure generator	docs/diagrams	2,212

Installers are provided for Windows (PowerShell, scheduled task as SYSTEM), Linux (systemd) and macOS (launchd). Fig. 3.43 shows a production deployment, and Fig. 3.44 the steps of the demonstration with the Windows virtual machine. The collector's own dashboard is shown in Fig. 3.45 to Fig. 3.47.

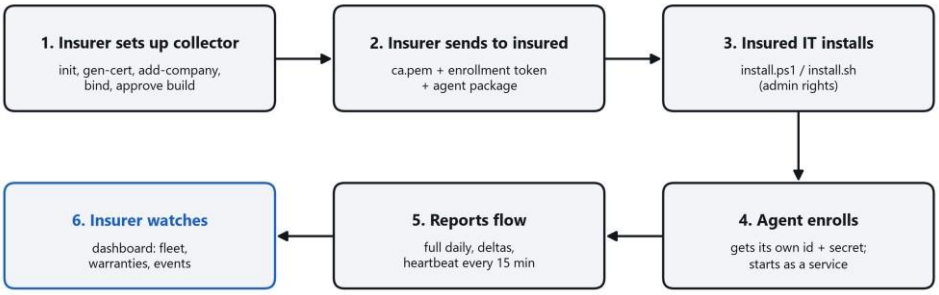


Fig. 3.43 Deployment of the agent and collector

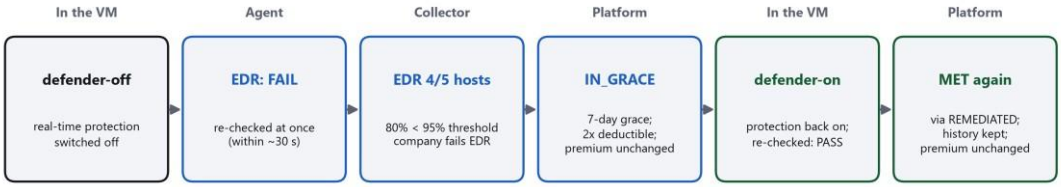


Fig. 3.44 Demonstration flow with the Windows virtual machine

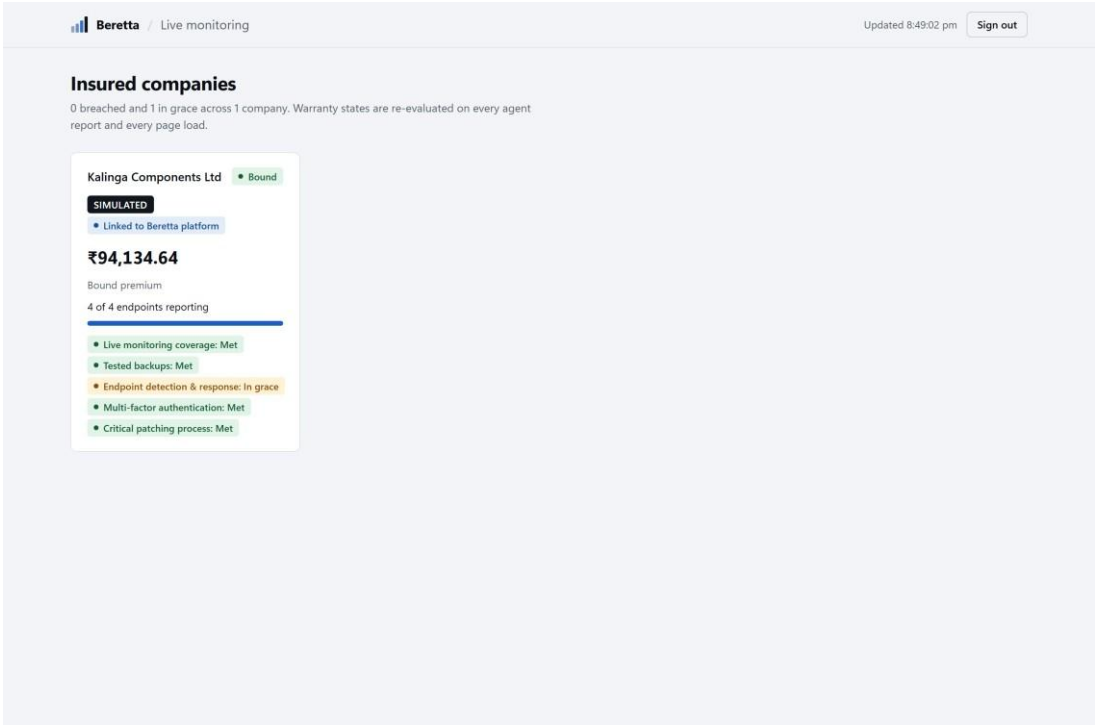


Fig. 3.45 Collector: overview of companies and hosts

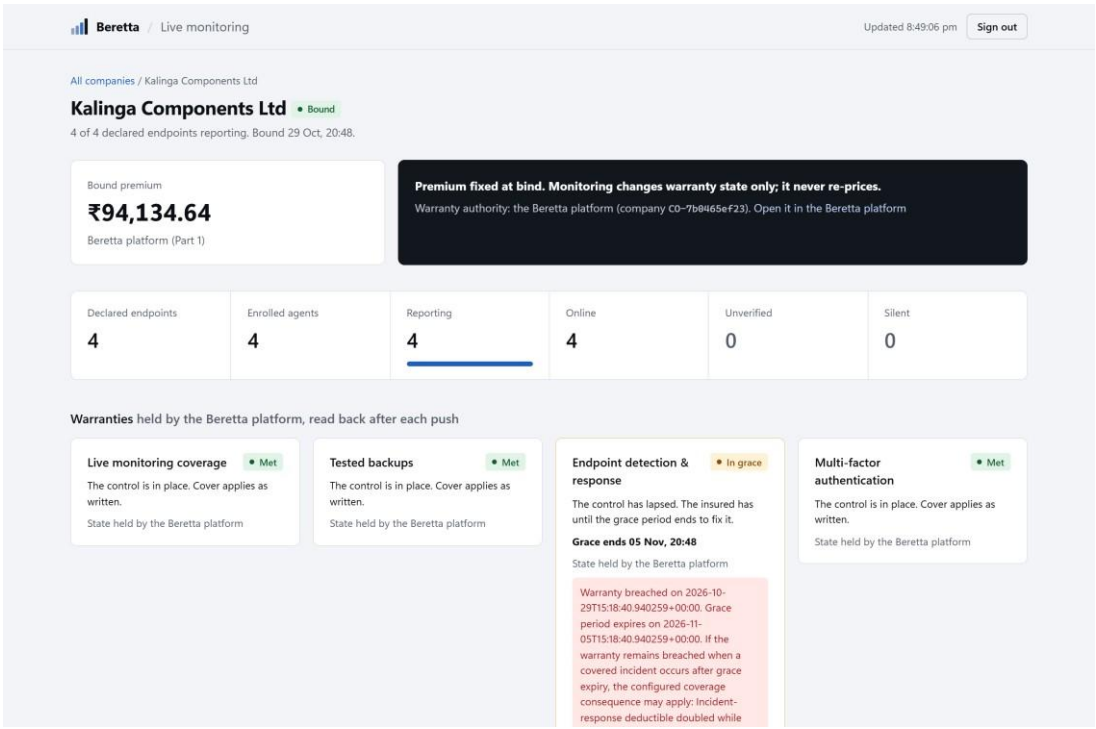


Fig. 3.46 Collector: one company, with the platform's warranty states and the link panel

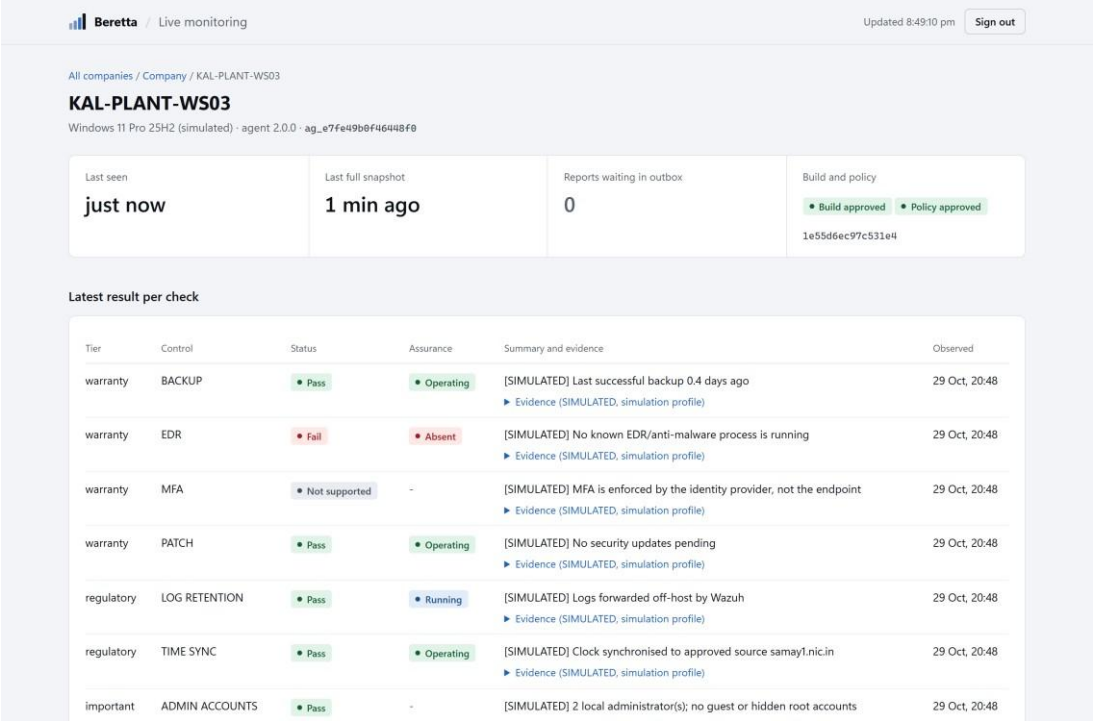


Fig. 3.47 Collector: one host, with the evidence behind each check

3.5 Summary

Beretta is built from two standard-library Python systems. The platform grades evidence, prices with a published-data frequency–severity model whose every factor is shown, binds, and runs the only warranty state machine; the agent and collector supply signed, read-only endpoint evidence that drives warranties after bind and never the premium. The next chapter tests whether it works and whether the model matches its sources.

Chapter 4 Results and Analysis

This chapter evaluates the system. Section 4.1 gives the test results and how they trace to the requirements. Section 4.2 checks the pricing model against its sources. Section 4.3 works through a full quote, and Section 4.4 measures how much the premium depends on each assumption. Section 4.5 reports the linked demonstration, Section 4.6 the role-separation checks, and Section 4.7 compares V2 with V1 and with current practice. Verification and validation followed IEEE Std 1012-2016 [47]: every requirement has at least one automated test, and the tests were run after every change. Test design followed the concepts of ISO/IEC/IEEE 29119-1:2022 [48]: unit tests for

each module, property tests for the model (for example, that upgrading a control never raises the premium), and end-to-end tests that run the real servers.

4.1 Test results

All three test suites pass (Table 4.1, Fig. 4.1). Part 2's suite starts real HTTPS servers with a private certificate authority; on Windows it needs `openssl` on the path, otherwise six of its tests are skipped rather than failed.

Table 4.1 Automated test suites

Suite	Command	Tests	Result
Part 1: platform and pricing	python -m unittest discover -s kyh-cyberinsurance/tests -t kyh-cyber-insurance	65	OK
Part 2: agent and collector	python kyh-agent/run_tests.py	158	OK
Integration: both systems	python integration/test_linked_system.py	1	OK
Total		**224**	**All pass**

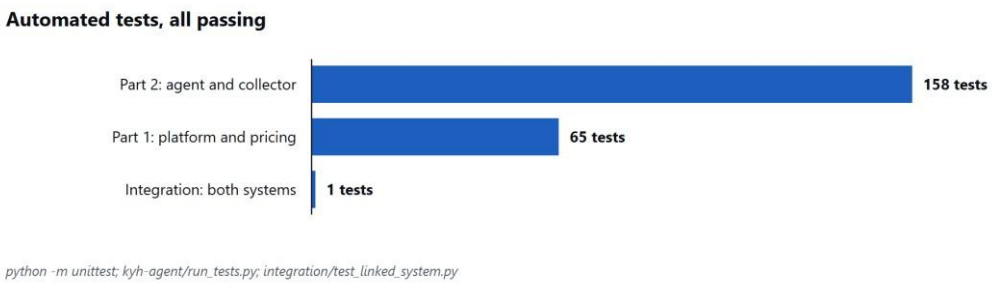


Fig. 4.1 Automated tests per suite

Table 4.2 What the tests cover

File	Tests	Covers
test_pricing_v2.py	28	Every control centred; upgrading a control never raises the premium; NOT_WORKING priced like UNKNOWN; turnover, retention and limit monotonic; continuity at band edges; payout matches LEV; aggregate limit caps the payout; no sub-limits; peril frequencies; α values and use; gating and referral; snapshot reproduces; savings real and sorted; every parameter tagged and sourced
test_platform.py	21	Evidence strength scales credit; documents verify, self-assessments state; best evidence wins; existence, currency and testing; outside-in signals as evidence; quote, bind and snapshot; V1 quotes still reproduce; **no re-pricing after bind**; full warranty state machine; grace days; audit events; HTTP end-to-end; roles; customer read-only and scoped; malformed input
test_platform_fleet_link.py	16	Bad signature, unknown collector, replay, stale time, unlinked company, invalid values; fleet evidence recorded at TECHNICALLY_OBSERVED; repeats skipped; fleet evidence feeds the quote; backup pass priced as STATED; monitoring-only control never priced; post-bind failure moves the warranty, never the premium

kyh-agent: test_parsers.py	36	Parsing of recorded Windows, Linux and macOS output, including PowerShell single-object JSON, BitLocker codes, sshd Include files and IPv6 in /proc/net/tcp6
kyh-agent: test_verdicts.py	30	Every decision rule: patch age, EDR health and signature age, backup age, exposed services, time sources and the rest
kyh-agent: test_agent_core.py	37	Policy validation and hashing, runner failure handling, scheduler deltas and heartbeats, signing, outbox ordering and limits, transport answers
kyh-agent: test_collector.py	38	Enrolment, the ingestion gate, replay and signature rejection, build and policy approval, liveness and the silence rule, the 95% roll-up, the warranty state machine, the API
kyh-agent: test_platform_link.py	10	Mapping and grouping, backup sent as partial, MFA never sent, signed and numbered roll-ups sent only on change, heartbeat, retry, silent host reaches the platform as a failure, no state machine for linked companies
kyh-agent: test_end_to_end.py	7	Real HTTPS collector and agents: enrol, report, tamper, replay, offline outbox
integration/test_linked_system.py	1	Both servers: four agents report, the quote uses the fleet, bind, EDR stops on one machine, the platform warranty goes IN_GRACE and the collector shows it, the fix returns it to MET, the premium never changes

The requirements were traced to tests as recommended by ISO/IEC/IEEE 29148 [38] (Table 4.3).

Table 4.3 Requirement traceability

Requirement	**Verified by**
R1. Every input carries a source, locator and evidence strength; the strongest evidence wins	test_best_evidence_wins_resolution, test_evidence_strength_scales_credit
R2. Every pricing parameter has a value, unit, source, locator and tag; every assumption a sensitivity range	test_every_parameter_has_a_tag_and_source_and_every_assumption_a_sensitivity
R3. The average firm pays the base rate for every control	test_every_control_is_centred
R4. Better evidence never raises the premium	test_upgrading_any_control_never_raises_the_premium
R5. The payout integration is accurate	test_matches_lev, test_aggregate_limit_caps_the_payout
Requirement	**Verified by**
R6. The same inputs always give the same premium, now and later	test_deterministic, test_v2_snapshot_reproduces, test_v1_quotes_still_reproduce
R7. A bound policy is never re-priced	test_MANDATORY_post_bind_no_reprice, test_post_bind_failure_moves_warranty_not_premium, integration test
R8. Agent reports cannot be forged or replayed	test_end_to_end (tamper, replay), collector ingestion tests
R9. Roll-ups to the platform cannot be forged, replayed or sent for unlinked companies	test_bad_signature, test_replay_rejected, test_company_must_be_linked_to_this_collector
R10. The insured customer is read-only and sees only its own company	test_customer_is_read_only_and_sees_only_their_company

4.2 Checks of the pricing model against its sources

Severity shape. The lognormal shape was fitted to two facts from NetDiligence 2025: the SME mean claim ($264K) and the share of claims above $1M (395 of 8,936, or 4.42%) [17]. Two values of σ satisfy both. The third published point, the share above $500K (8.24%), separates them (Fig. 4.2): σ = 2.193 predicts 8.26% and σ = 1.214 predicts 12.86%. The model uses 2.193. IRIS 2025's median and 95th percentile for firms under $100M give σ ≈ 1.97, which is used as the low value in the sensitivity analysis.



Fig. 4.2 Check on the lognormal shape against the share of SME claims above $500K

Frequency. The five peril frequencies add to 1.38%; with Coalition's 6.2% "other" claims the total is 1.47%, against the published 1.48%. The 10–25M turnover band (1.19%) is reproduced by `derive_sigma.py` from 1.07% ÷ 0.90.

Payout integration. For a single head with share 1, the 24,000-point quantile grid matches the closed-form LEV(d + L) − LEV(d) of Equation 3.4 to about one part in 10^7; the test requires agreement within 0.1%.

Centring and monotonicity. For every priced control, p·m_has + (1 − p)·m_lacks = 1 within rounding, so the average firm pays the base rate. Upgrading any control from UNKNOWN to STATED to VERIFIED never raises the premium, a higher retention never raises it, and a higher limit or turnover never lowers it. Fig. 4.3 shows each control's multiplier by evidence status. The spread between verified and failing is widest for secure configuration, EDR and logging, which have high adoption: lacking a control that 96% of firms have is a strong signal. Privileged access management and vulnerability management have low adoption, so verifying them gives the largest reduction.

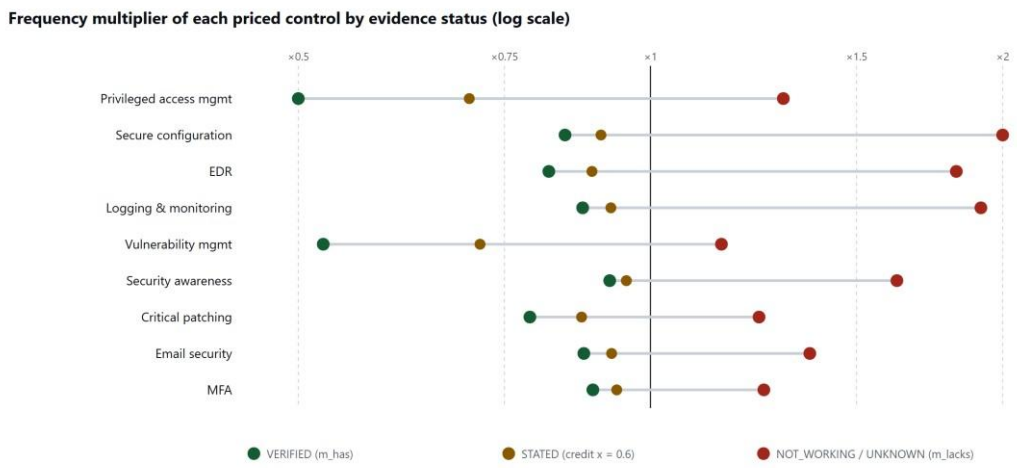


Fig. 4.3 Frequency multiplier of each priced control by evidence status

Claims credibility. Fig. 4.4 shows the claims multiplier for zero to four incidents at λT = 0.03 for the base α and the ends of its sensitivity range. One material incident raises the premium by about 23%, two by about 46%. A larger α gives the firm's own history less weight.

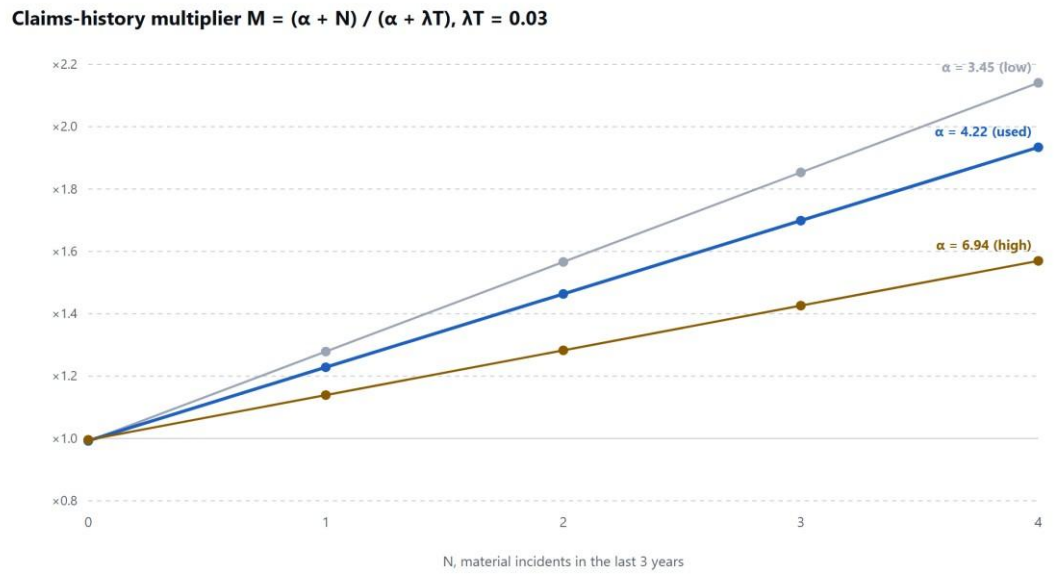


Fig. 4.4 Claims-history multiplier against the number of material incidents

Derived values. `derive_credit_x.py` gives x = 0.62 at f = 0.2 (the file uses 0.6). `derive_alpha.py` reproduces α ≈ 4.5 by the specification's method; the specification's value of 4.22 is used. The script also shows that measuring heterogeneity on the allperil frequency instead of ransomware would give α ≈ 1.6 (Section 5.2).

4.3 Worked example

`scripts/worked_example_v2.py` prices an IT services firm (Table 4.4). Its evidence is mixed on purpose: four controls verified, EDR failing, PAM unknown and the rest only stated.

Table 4.4 Worked example: inputs

Input	Value
Sector	NIC 62 → Information Technology
Turnover	₹150 crore (USD 17.05M at ₹88/USD), 10–25M band
Years in operation; material incidents	10; 1
Policy	Overall limit ₹5 crore; retention ₹5 lakh per claim
VERIFIED	Critical patching, secure configuration, logging, encryption
NOT_WORKING / UNKNOWN	EDR / privileged access management
STATED	MFA, email security, awareness, vulnerability management, tested backups, IR plan

Table 4.5 Worked example: results

Step	Result
Classification	Turnover ×0.804 (frequency), ×0.687 (severity); sector ×0.919, ×1.141
Controls, ransomware	Weakness ×0.609, execution ×2.013 (EDR failing, PAM unknown), credential ×0.885 → ×1.085
Claims history	(4.22 + 1) / (4.22 + 0.00993 × 3) = ×1.228
Expected loss	₹49,947 (ransomware ₹20,823; FTF ₹20,287; BEC ₹5,379; other firstparty ₹3,277; third-party ₹181)
Premium	Technical ₹99,895; GST ₹17,981; payable ₹1,17,876
Gate	INELIGIBLE: EDR not working (gate mode BLOCK)
Largest savings	Verify PAM ₹32,421; fix and verify EDR ₹20,877; verify vulnerability management ₹12,739

Ransomware and funds transfer fraud carry 82% of the expected loss (Fig. 4.5). Funds transfer fraud is large because its mean loss ($185K) is high and its frequency (0.44%) is the joint highest. The premium is built from the expected loss as in Fig. 4.6, and Fig. 4.7 lists what verifying each control would save. The firm cannot be bound until EDR is fixed, because EDR is a warranty and it is failing; the premium is still computed so the firm can see the price it would pay.

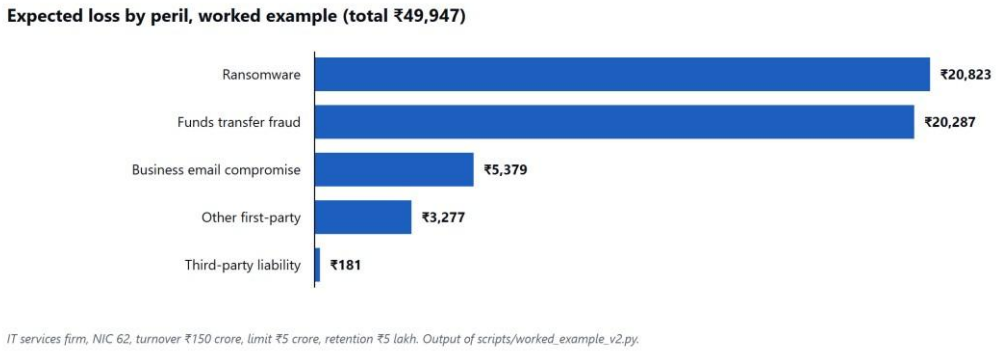


Fig. 4.5 Expected loss by peril, worked example

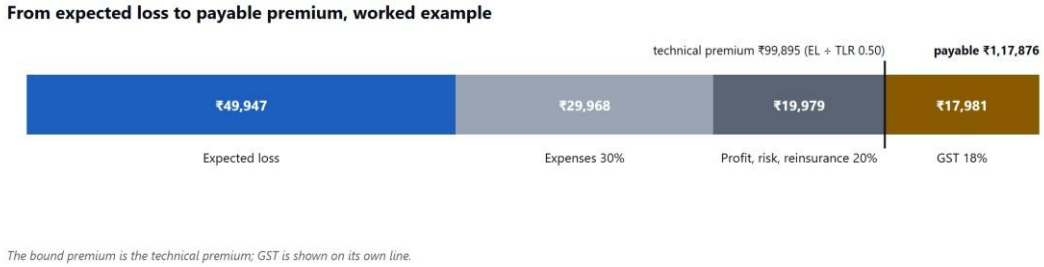


Fig. 4.6 From expected loss to payable premium, worked example

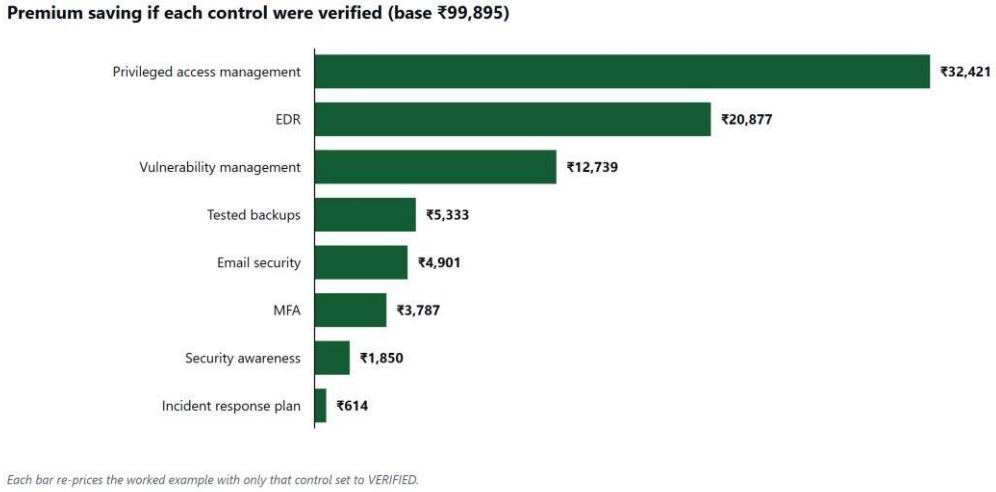


Fig. 4.7 Premium saving if each control were verified, worked example

A full quote including the eight what-if re-pricings took 0.18 s on the development laptop, so the what-if list can be computed on every quote.

4.4 Sensitivity analysis

Every assumption [A] in the parameter file has a low and a high value. `scripts/sensitivity_v2.py` moves each in turn for the worked-example firm (Table 4.6, Fig. 4.8).

Table 4.6 Sensitivity of the worked-example premium (base ₹99,895)

Assumption	Low	Change	High	Change
σ (severity shape)	1.97	+16.3%	2.19	+0.2%
β (within-stage damping)	0	+8.2%	1	−15.5%
x (credit for STATED)	0.4	+13.2%	0.8	−12.6%
TLR	0.48	+4.2%	0.57	−12.3%
Trend	−10%	−10.0%	+10%	+10.0%
α (claims credibility)	3.45	+4.1%	6.94	−7.3%
Turnover zone factor	1.10	0.0%	1.50	+4.5%
USD/INR	83	−4.1%	92	+3.2%
MFA adoption p	0.5	−3.1%	0.9	+1.4%
Control caps	[0.5, 2.0]	base	[0.4, 2.5]	−1.5%
Head-share placeholders	×0.75	+0.1%	×1.25	−0.1%

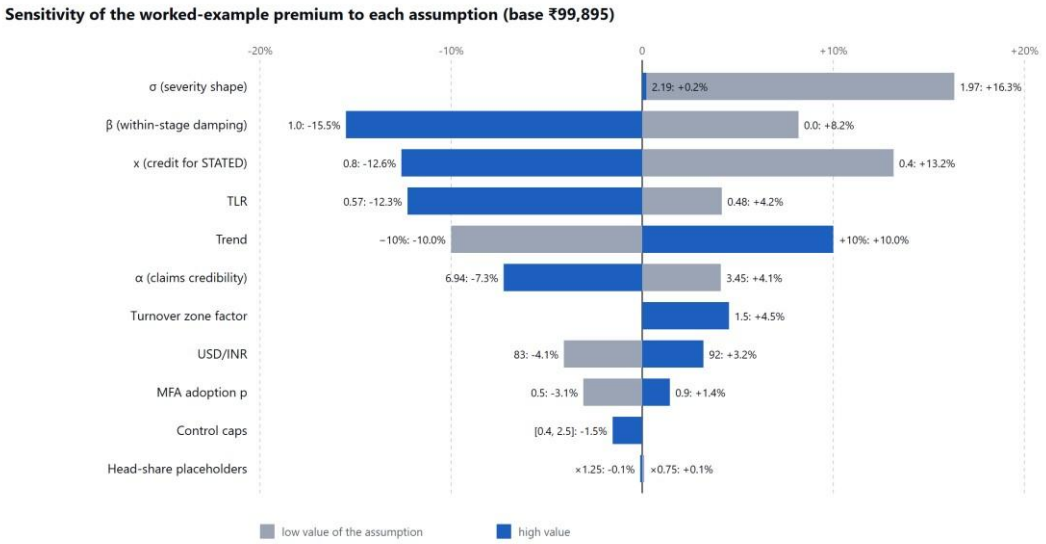


Fig. 4.8 Sensitivity of the worked-example premium to each assumption

Four observations follow.

- No single assumption moves the premium by more than about 16%, so the model's output is driven mainly by its published inputs.
- A lower σ raises the premium. With the mean held fixed, a thinner tail puts less of the loss above the ₹5 crore limit and more in the band the policy pays.
- The credit for a stated control (x) and the damping within a stage (β) matter most among the controls assumptions. Both are reasons to collect verified evidence: the more controls are verified, the less these assumptions matter.

- Without sub-limits, the head-share placeholders barely matter (±0.1%); they only decide which covers the severity controls act on.

4.5 Linked demonstration

`run_linked_demo.py` starts the platform, the collector and four simulated endpoints for Kalinga Components (NIC 28, Industrials, ₹60 crore turnover, 18 years, one material incident, limit ₹2 crore, retention ₹1 lakh). Every simulated result is labelled SIMULATED. The demonstration was run end to end for this report, and Table 4.7 lists what happened at each step.

Table 4.7 Demonstration run

Step	Action	Observed result
1	Agents report; collector sends rollup #1	Platform records AGENT_FLEET evidence; 4 of 4 endpoints reporting; secure configuration, PAM, EDR, patching, logging and encryption VERIFIED
2	Underwriter calculates the premium	₹94,135 technical, ₹1,11,079 payable; CONDITIONALLY_ELIGIBLE (tested backups only stated, because restore testing cannot be observed)
3	Underwriter binds	Policy BOUND at ₹94,135; warranties created, all MET
4	`break KAL-PLANT-WS03 edr_down`	3 of 4 hosts pass EDR (75% < 95%); CTRL-EDR warranty MET → IN_GRACE within seconds; premium ₹94,135
5	`fast-forward` (audited DEMO_TIME_TRAVEL)	EDR warranty IN_GRACE → BREACHED; coverage consequence shown; premium ₹94,135
6	`fix KAL-PLANT-WS03`	EDR warranty BREACHED → REMEDIATED → MET; premium ₹94,135
7	Re-price the bound policy (as in the integration and platform tests)	Refused: 409 POLICY_ALREADY_BOUND

The premium did not change at any step after bind, and every roll-up answer from the platform carried `repricing_performed: false`. Fig. 4.9 to Fig. 4.13 are screens from this run, and Fig. 4.14 is the timing diagram of a warranty moving through its states.

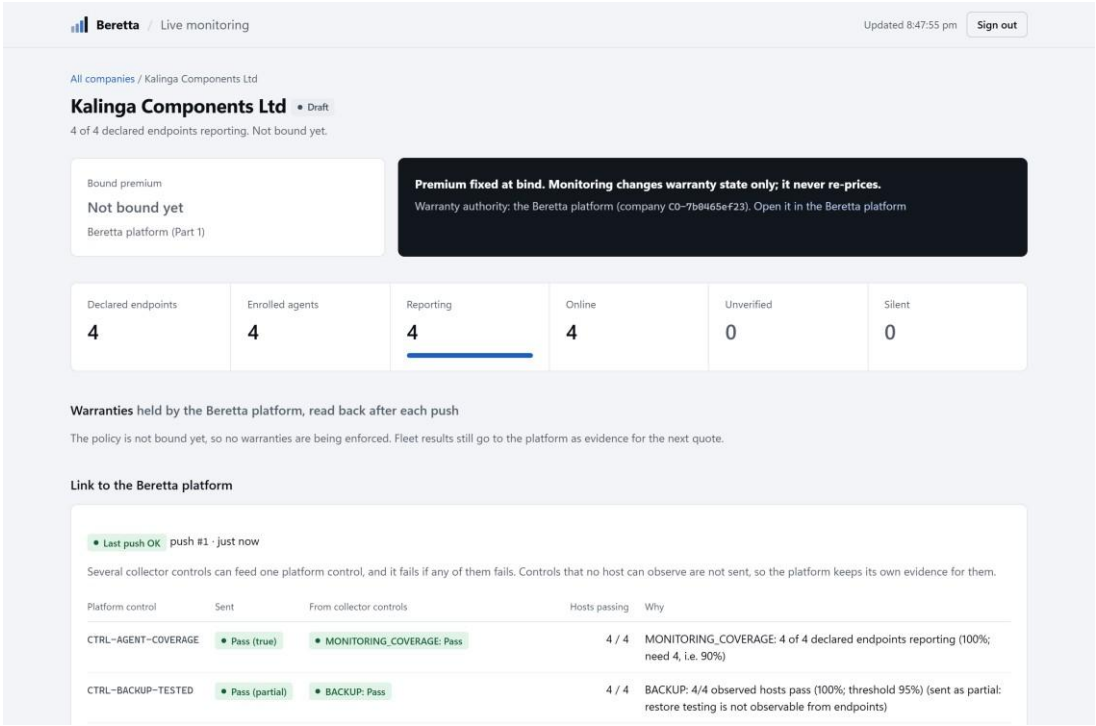


Fig. 4.9 Collector before bind: four simulated endpoints reporting for Kalinga Components

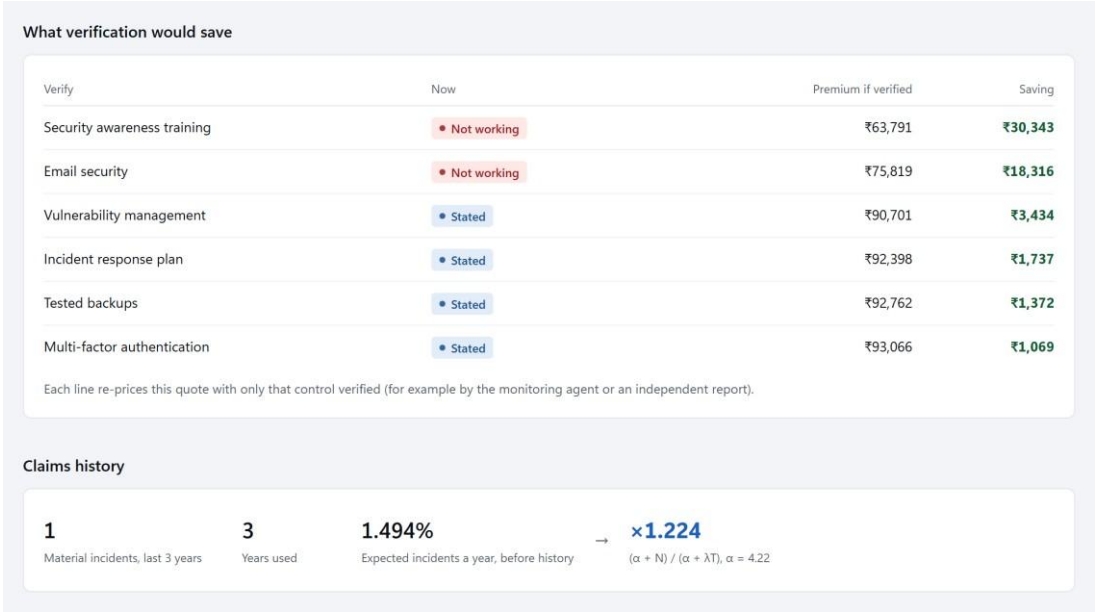


Fig. 4.10 Quote for Kalinga Components: what verification would save, and claims history

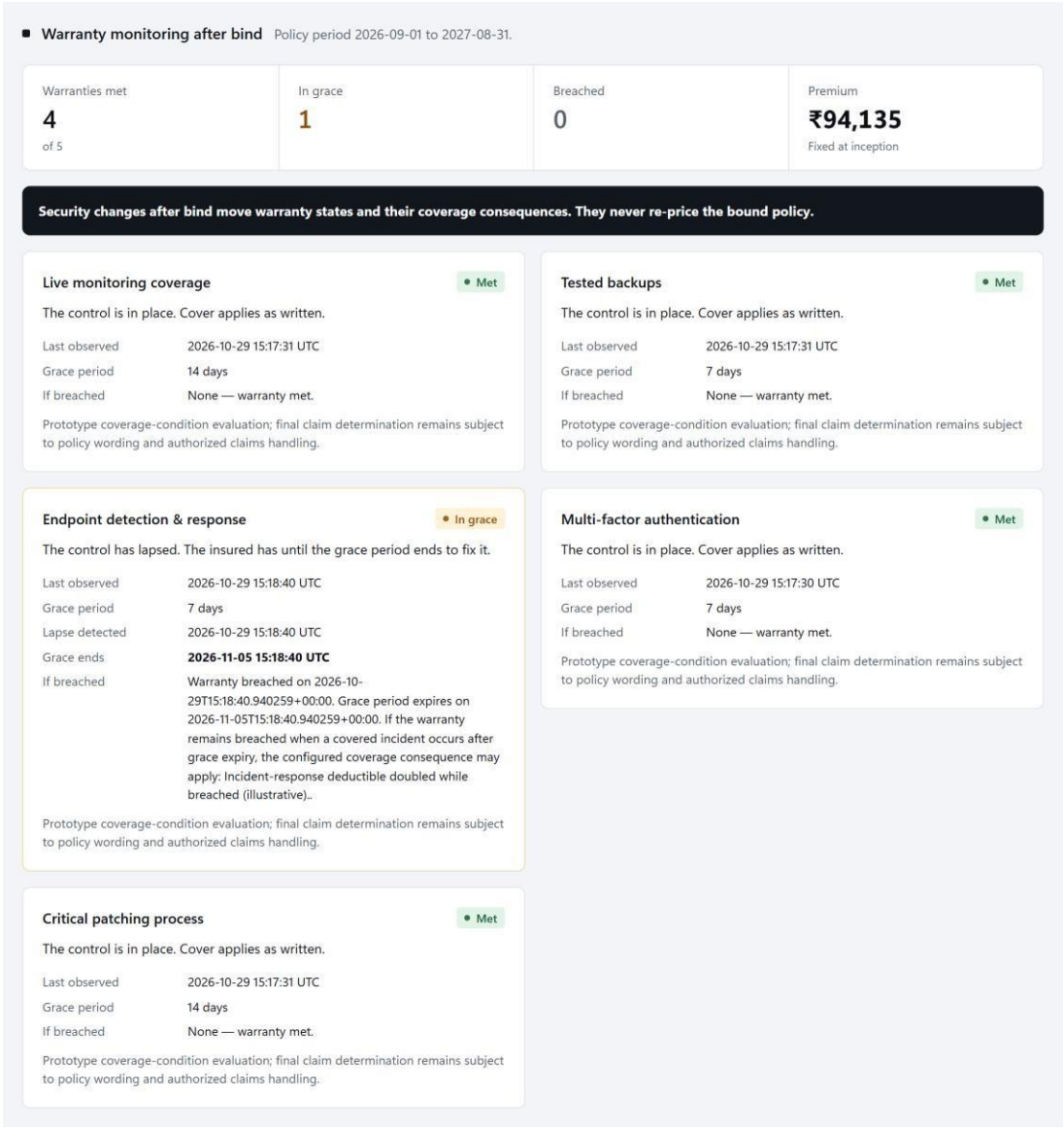


Fig. 4.11 Platform Monitoring tab after EDR stopped on one machine: only the EDR warranty moved

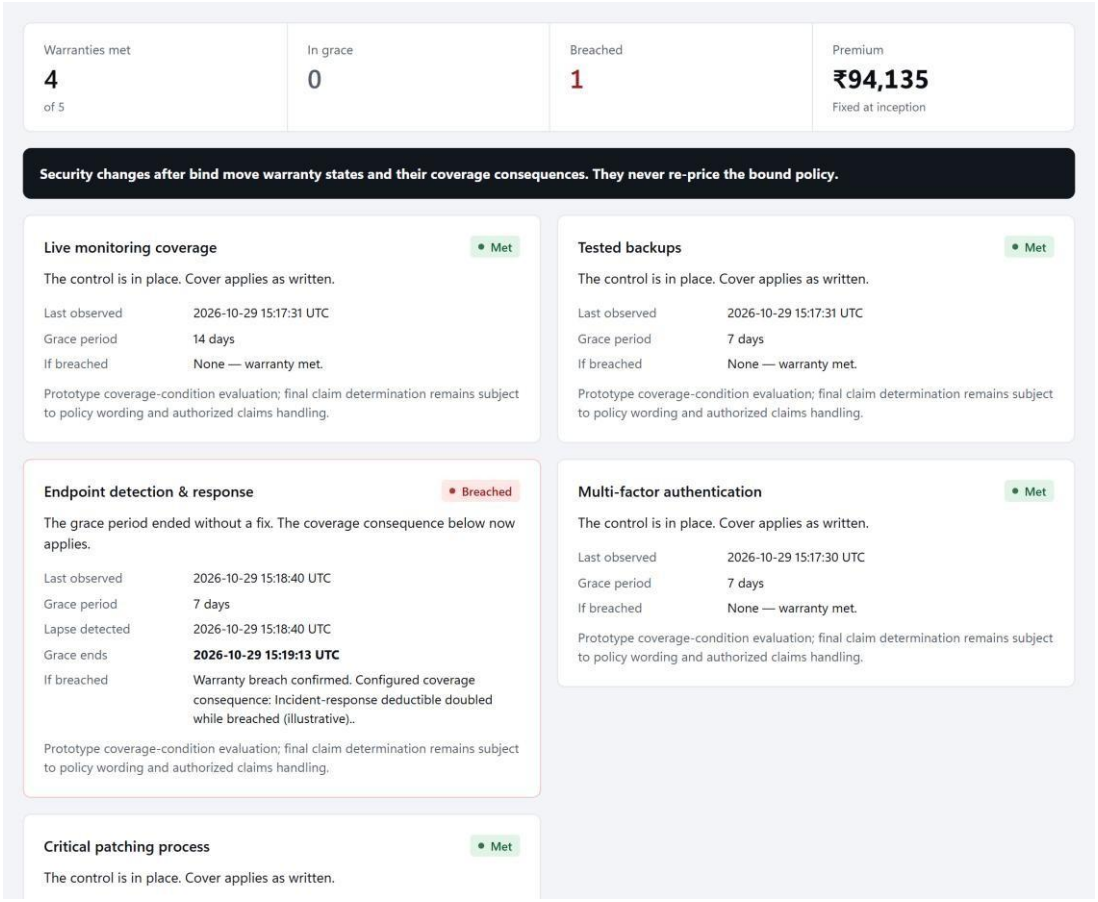


Fig. 4.12 After the demonstration's fast-forward: the EDR warranty is breached; the premium is unchanged

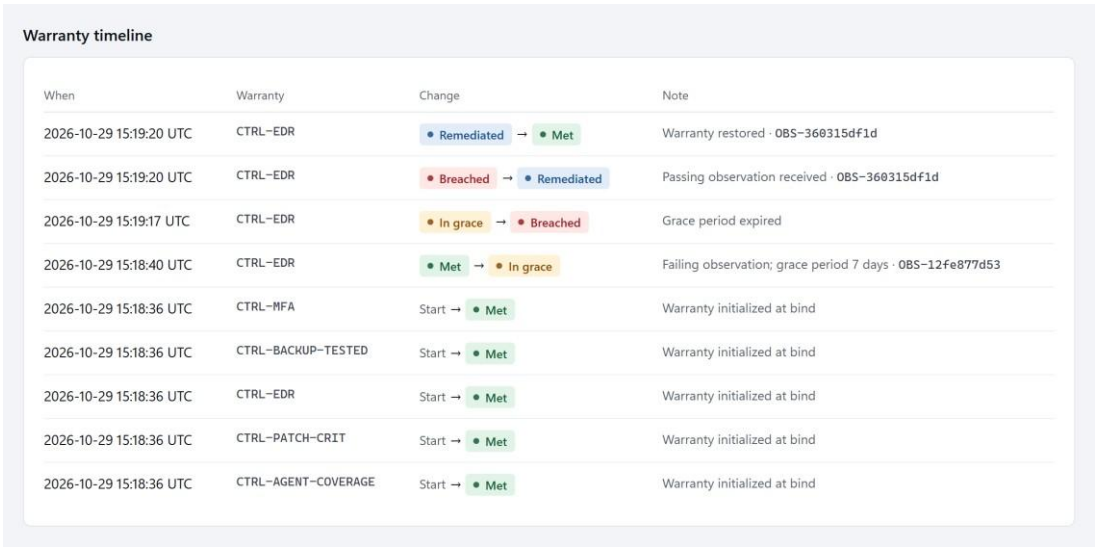


Fig. 4.13 Warranty timeline: MET, IN_GRACE, BREACHED, REMEDIATED, MET

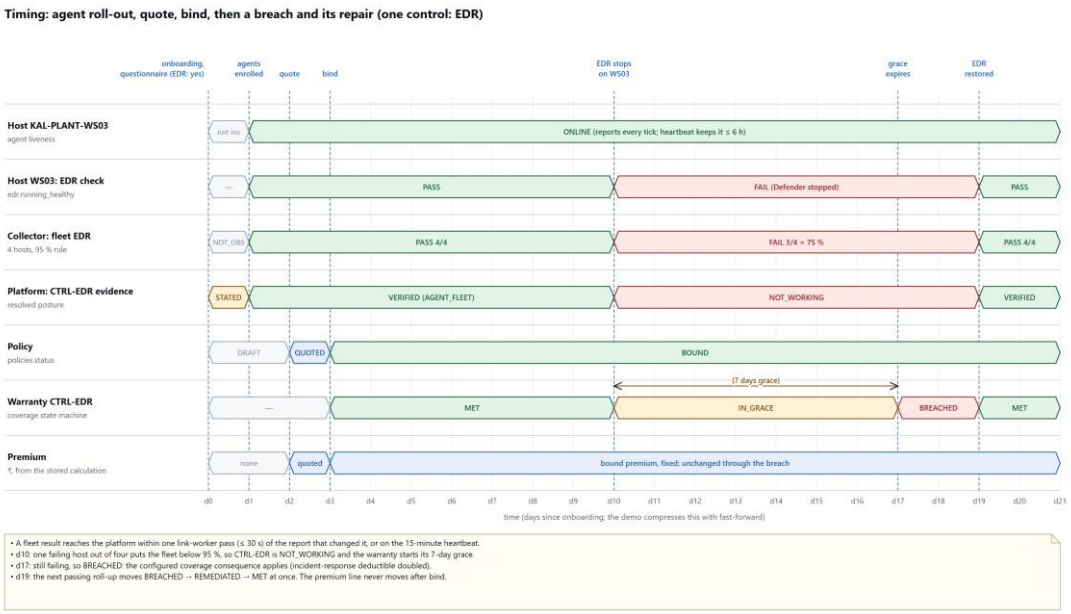


Fig. 4.14 UML timing diagram of the EDR warranty during the demonstration

The fleet section of the Monitoring tab (Fig. 4.15) shows each host's liveness and results, what the fleet sent for each control, which collector controls built it, and why.

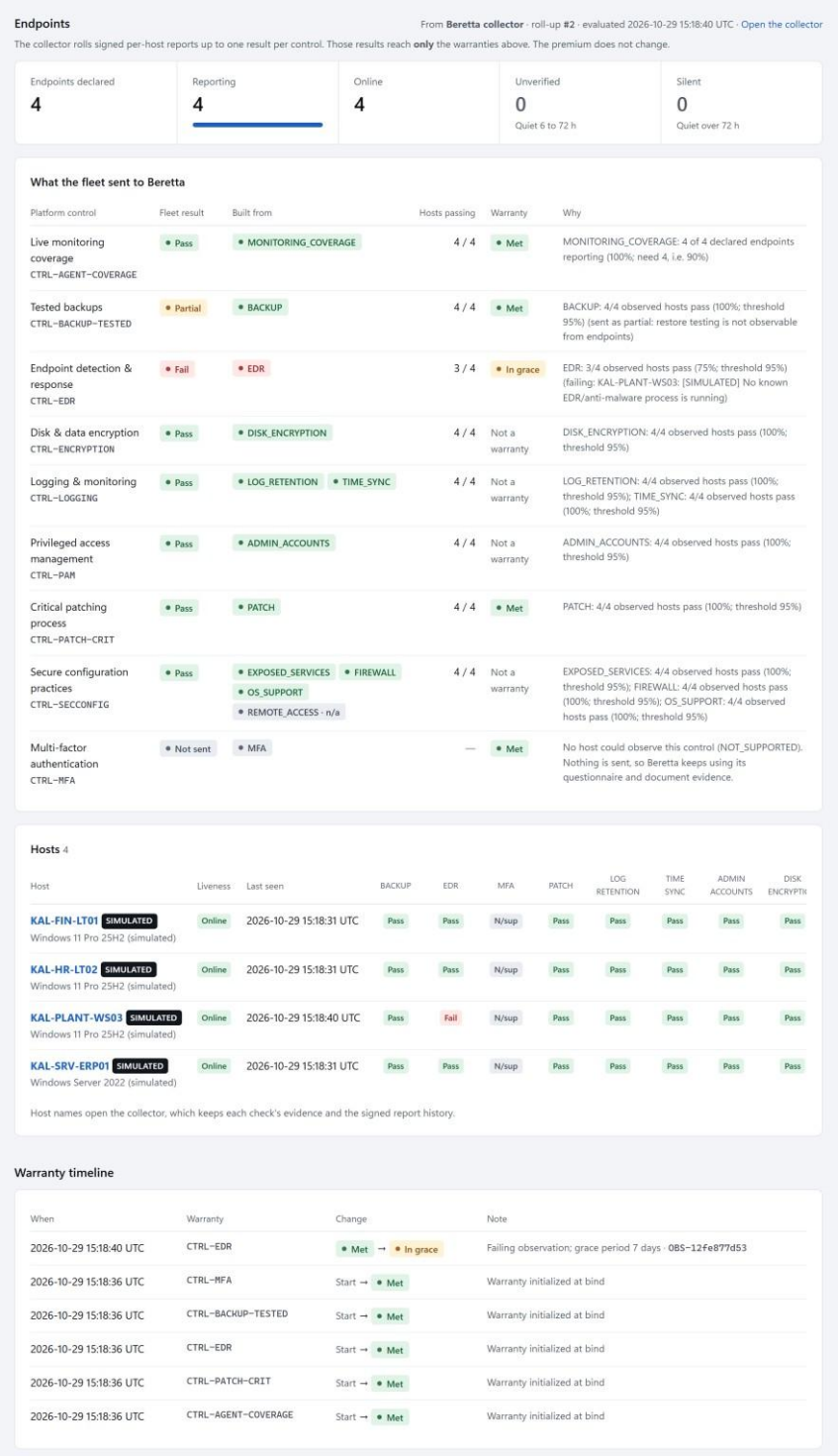


Fig. 4.15 Monitoring tab: endpoints, what the fleet sent, and why

4.6 Role separation

The insured-customer role was checked against a running server with the customer's token (Table 4.8). Every write was refused, every read of another company returned "not found", and the customer's quote contained only the price, the eligibility, the evidence per control and the savings. The underwriter's working (perils, loadings,

referral details and input snapshot) is not sent to the customer at all. **Table 4.8** Customer-role checks

Request as the insured customer	Result
GET /api/me	200: read-only, own company
GET /api/companies	Only its own company
GET another company, or its policy	404 NOT_FOUND
GET dashboard, audit log, model configuration	403
POST claims, questionnaires, documents, outside-in signals	403
PUT company profile or policy structure; POST quote or bind	403
GET its own quote	200, with premium, GST, eligibility, statuses, evidence, savings and policy structure only

4.7 Comparison with the earlier model and with practice

Table 4.9 compares V2 with V1. V1 remains in the code, frozen, and its stored quotes still reproduce exactly.

Table 4.9 Model V1 and model V2

	V1	V2
Structure	One frequency × Σ(severity fraction × limit)	Five perils, each frequency × lognormal payout
Base frequency	Seven illustrative sectors (8–20%)	Coalition 2025 sector and turnover data (about 1–6%)
Controls	Additive weights, clamped to [0.6, 1.6]	Multiplicative, centred, damped within a stage, bounded [0.27, 3.0]
Evidence	Credit scaled 0.6 / 0.8 / 0.9 / 1.0 by strength	VERIFIED / STATED (x = 0.6) / NOT_WORKING / UNKNOWN
Severity	Fraction of each head's limit	Lognormal (σ 2.193) through the retention and the overall limit
Claims	+10% per incident, cap 1.5	Credibility (α + N) / (α + λT)
Outside-in	Separate exposure modifier	Evidence for the control it concerns
Loading	× 1.40 additive	÷ TLR 0.50 with a 50/30/20 split; GST 18% on top
Gating	WARN by default	BLOCK by default; MFA may be stated; referral needs approval
Sources	Illustrative	Every value tagged [L], [D] or [A] with its source

Against the practice described by Romanosky et al. [3], where the price is a revenuebased rate with a few modifiers, V2 prices each peril from published claims data, distinguishes verified from stated controls, and shows the applicant the effect of each one. Against the proposal forms studied by Woods et al. [2], Beretta asks about fifteen controls, records how each answer was verified, and checks the warranty controls after bind.

4.8 Summary

All 224 tests pass. The model reproduces its published inputs, its payout integration matches the closed form, and no assumption moves the worked-example premium by more than about 16%. In the linked demonstration, an EDR failure on one machine moved one warranty through all its states while the bound premium stayed fixed, and the customer role was confirmed read-only.

Chapter 5 Advantages, Limitations and Applications

This chapter sets out what the system does well, where it falls short, and where it could be used.

5.1 Advantages

- **Every number is traceable.** Each pricing value has a source, a locator and a tag in one file; the quote lists the assumptions it used, and a test fails if any parameter lacks a source.
- **The price is explained to the insured.** The customer sees the premium, what is needed before cover, the evidence held for each control and how much verifying each one would save.
- **Evidence strength matters.** A declared control earns 60% of a verified control's credit, and a failing one is priced as absent, so there is a reason to provide evidence and no reward for an unchecked "yes".
- **Monitoring is fair to the insured.** The premium is fixed at bind. Monitoring can only move warranty states, with a grace period before any breach, and every state change is logged.
- **Reports cannot easily be faked.** Signed reports, sequence numbers, build hashes and the silence rule cover forgery, replay, modified agents and uninstalled agents.
- **Privacy by design.** The agent is read-only and never collects file contents, browsing, keystrokes, e-mail, passwords or clear-text user names.
- **Reproducible.** The same inputs always give the same premium, and stored snapshots reproduce old quotes under both V1 and V2.
- **No dependencies.** The standard-library-only design makes the system easy to inspect, run and audit.

5.2 Limitations

- **US data without an India adjustment.** The frequencies and severities come from US-weighted claims studies. The India factors are set to 1 and the USD/INR rate is a placeholder.

- **Shared effects.** Turnover and sector relativities are assumed equal across perils, and one σ is used for all perils.
- **Mixed sources.** Base frequencies come from Coalition's book and control adoption from Marsh's clients; if Marsh's clients are better controlled, the average firm may be under-priced.
- **Placeholders.** The coverage-head shares, the adoption of the IR plan, awareness and encryption, and MFA adoption are assumptions.
- **No sub-limits.** Each cover can pay up to the overall limit. Real policy wordings usually sub-limit some covers, such as funds transfer fraud, which would lower the premium.
- Measuring heterogeneity on ransomware frequency gives 4.22; measuring it on the all-peril frequency gives about 1.6. A real book's experience should settle it.
- **Accumulation** (one event hitting many insureds) is not modelled separately; it sits inside the 20% margin.
- **Windows checks not yet run on real hardware.** They are tested on recorded output; the first run on the Windows virtual machine is their real test.
- **Prototype security.** Static role tokens; a demonstration endpoint (`/api/demo/tokens`) that gives the dashboard's role switcher every role's token and must be removed before any real use, since it would let a customer act as an underwriter; shared HMAC secrets rather than public-key signatures; and a legacy single-host agent endpoint that accepts unsigned reports and must be disabled.
- **MFA cannot be observed** from endpoints; it needs an identity-provider export or connector.
- **Grace periods are whole days**, so the demonstration uses an audited fastforward.

5.3 Applications

- **Insurers and managing general agents in India** can use the design as a reference for evidence-graded SME cyber underwriting with post-bind warranty monitoring.
- **Brokers** can show a client the premium effect of each control before approaching insurers.
- **SMEs** can use the insured view to plan security spending against premium savings.
- **Regulators and auditors** can inspect a complete, reproducible pricing method with every assumption labelled.
- **Teaching and research** in actuarial science, security economics and secure systems design, where every part can be read and changed.
- **Compliance monitoring.** The agent's CERT-In checks (log retention and time synchronisation) are useful on their own for Indian organisations.

5.4 Summary

Beretta's main strengths are traceability, explanation and a clean separation between monitoring and price. Its main limitation is that its loss data is from the US and several shares and adoptions are still assumptions, each labelled and tested for sensitivity.

Chapter 6 Conclusion and Future Scope

6.1 Conclusion

This project built Beretta, an explainable cyber-insurance underwriting platform for Indian SMEs linked to a read-only endpoint-monitoring system, and showed that the whole chain from evidence to pricing, binding and post-bind warranty monitoring can be built openly, with every number traceable.

Engine A records every piece of evidence with its source and strength, and resolves each control to its strongest evidence. Engine B prices five perils from published claims data with a lognormal severity fitted to SME claims, multiplicative control effects centred on the average firm, a 60% credit for stated controls derived from an assumed hollow-declaration rate, and a credibility factor for claims history. The model reproduces its

published inputs, its numerical payout matches the closed form, and no single assumption moves the worked-example premium by more than about 16%. Every quote shows the applicant why the premium is what it is and how much verifying each control would save.

The agent and collector supply signed, tamper-evident endpoint evidence from Windows, Linux and macOS. Before bind this evidence lowers the price of a well-run firm; after bind it moves warranty states only. In the demonstration an EDR failure on one of four machines moved the EDR warranty through IN_GRACE, BREACHED and back to MET while the bound premium stayed at ₹94,135. The system passes 224 automated tests, including one that runs both systems end to end.

The project's conclusion is that the gap identified in Chapter 2 can be closed: cyber insurance can be priced from published data with the strength of the evidence taken into account, explained to the insured, and monitored after bind without re-pricing the policy.

6.2 Future scope

- **Indian claims data.** Calibrate the India factors F_india and S_india, the head shares and the adoptions from Indian claims, and replace the placeholder exchange rate with the RBI reference rate.
- **Identity-provider connectors** for Microsoft Entra ID and Google Workspace, so MFA can be verified rather than stated.
- **Sub-limits and policy wording.** Add per-cover sub-limits from a real wording, which would make the coverage consequences of a breached warranty enforceable terms rather than illustrations.
- **Accumulation modelling** for events that hit many insureds at once, such as a widely used software vulnerability.
- **Public-key signatures** (for example Ed25519) between agent, collector and platform, so no receiver holds the sender's secret, and retirement of the legacy single-host endpoint.
- **Real deployment.** Run the agent on a pilot fleet, starting with the Windows virtual machine, and measure false positives and the agent's load.

- **Experience checking.** Compare the model's expected losses with an insurer's actual claims to test σ, α and the control effects, in particular the choice between $\alpha = 4.22$ and $\alpha \approx 1.6$.
- **Document extraction.** Read assessed fields from SOC 2, ISO 27001 and VAPT reports automatically, keeping the locator for each value.

References

[1] Coalition, Inc., *2025 Cyber Claims Report*. San Francisco, CA, USA: Coalition, 2025.

[2] D. Woods, I. Agrafiotis, J. R. C. Nurse, and S. Creese, "Mapping the coverage of security controls in cyber insurance proposal forms," *Journal of Internet Services and Applications*, vol. 8, no. 1, art. 8, 2017.

[3] S. Romanosky, L. Ablon, A. Kuehn, and T. Jones, "Content analysis of cyber insurance policies: How do carriers price cyber risk?" *Journal of Cybersecurity*, vol. 5, no. 1, art. tyz002, 2019.

[4] Indian Computer Emergency Response Team (CERT-In), *Directions under sub-section (6) of section 70B of the Information Technology Act, 2000 relating to information security practices, procedure, prevention, response and reporting of cyber incidents for Safe & Trusted Internet*, No. 20(3)/2022-CERT-In, New Delhi, India, Apr. 28, 2022.

[5] Government of India, *The Digital Personal Data Protection Act, 2023* (No. 22 of 2023), *The Gazette of India*, Aug. 11, 2023.

[6] L. A. Gordon and M. P. Loeb, "The economics of information security investment," *ACM Transactions on Information and System Security*, vol. 5, no. 4, pp. 438–457, Nov. 2002.

[7] R. Anderson and T. Moore, "The economics of information security," *Science*, vol. 314, no. 5799, pp. 610–613, Oct. 2006.

[8] R. Böhme and G. Schwartz, "Modeling cyber-insurance: Towards a unifying framework," in *Proc. 9th Workshop on the Economics of Information Security (WEIS)*, Cambridge, MA, USA, Jun. 2010.

[9] N. Shetty, G. Schwartz, M. Felegyhazi, and J. Walrand, "Competitive cyber-insurance and Internet security," in *Economics of Information Security and Privacy*, T. Moore, D. Pym, and C. Ioannidis, Eds. Boston, MA, USA: Springer, 2010, pp. 229–247.

[10] R. Pal, L. Golubchik, K. Psounis, and P. Hui, "Will cyber-insurance improve network security? A market analysis," in *Proc. IEEE INFOCOM*, Toronto, ON, Canada, 2014, pp. 235–243.

[11] T. Maillart and D. Sornette, "Heavy-tailed distribution of cyber-risks," *The European Physical Journal B*, vol. 75, no. 3, pp. 357–364, 2010.

[12] B. Edwards, S. Hofmeyr, and S. Forrest, "Hype and heavy tails: A closer look at data breaches," *Journal of Cybersecurity*, vol. 2, no. 1, pp. 3–14, 2016.

[13] S. Wheatley, T. Maillart, and D. Sornette, "The extreme risk of personal data breaches and the erosion of privacy," *The European Physical Journal B*, vol. 89, no. 1, art. 7, 2016.

[14] C. Biener, M. Eling, and J. H. Wirfs, "Insurability of cyber risk: An empirical analysis," *The Geneva Papers on Risk and Insurance Issues and Practice*, vol. 40, no. 1, pp. 131–158, 2015.

[15] M. Eling and J. Wirfs, "What are the actual costs of cyber risk events?" *European Journal of Operational Research*, vol. 272, no. 3, pp. 1109–1119, 2019.

[16] M. Eling and W. Schnell, "What do we know about cyber risk and cyber risk insurance?" *The Journal of Risk Finance*, vol. 17, no. 5, pp. 474–491, 2016.

[17] NetDiligence, *Cyber Claims Study 2025*. Gladwyne, PA, USA: NetDiligence, 2025.

[18] Cyentia Institute, *Information Risk Insights Study (IRIS) 2025*. Cyentia Institute, 2025.

[19] Cyentia Institute, *Information Risk Insights Study (IRIS) 2022*. Cyentia Institute, 2022.

[20] D. Woods and A. Simpson, "Policy measures and cyber insurance: A framework," *Journal of Cyber Policy*, vol. 2, no. 2, pp. 209–226, 2017.

[21] S. A. Talesh, "Data breach, privacy, and cyber insurance: How insurance companies act as 'compliance managers' for businesses," *Law & Social Inquiry*, vol. 43, no. 2, pp. 417–440, 2018.

[22] D. W. Woods, T. Moore, and A. C. Simpson, "The county fair cyber loss distribution: Drawing inferences from insurance prices," *Digital Threats: Research and Practice*, vol. 2, no. 2, art. 10, 2021.

[23] A. Marotta, F. Martinelli, S. Nanni, A. Orlando, and A. Yautsiukhin, "Cyber-insurance survey," *Computer Science Review*, vol. 24, pp. 35–61, May 2017.

[24] Marsh McLennan, *Using Data to Prioritize Cybersecurity Investments*. New York, NY, USA: Marsh McLennan, 2023.

[25] Sophos, *The Impact of Compromised Backups on Ransomware Outcomes*. Abingdon, U.K.: Sophos, 2024.

[26] IBM Security and Ponemon Institute, *Cost of a Data Breach Report 2023*. Armonk, NY, USA: IBM, 2023.

[27] IBM Security and Ponemon Institute, *Cost of a Data Breach Report 2019*. Armonk, NY, USA: IBM, 2019.

[28] L. Meyer, S. Romero, G. Bertoli, T. Burt, A. Weinert, and J. L. Ferres, "How effective is multifactor authentication at deterring cyberattacks?" arXiv:2305.00945, 2023.

[29] H. Bühlmann, "Experience rating and credibility," *ASTIN Bulletin*, vol. 4, no. 3, pp. 199–207, 1967.

[30] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," arXiv:1702.08608, 2017.

[31] National Institute of Standards and Technology, *The NIST Cybersecurity Framework (CSF) 2.0*, NIST CSWP 29, Gaithersburg, MD, USA, Feb. 2024.

[32] Center for Internet Security, *CIS Critical Security Controls, Version 8*. East Greenbush, NY, USA: CIS, 2021.

[33] *Information security, cybersecurity and privacy protection Information security management systems Requirements*, ISO/IEC 27001:2022, International Organization for Standardization, Geneva, Switzerland, 2022.

[34] Insurance Regulatory and Development Authority of India, *IRDAI (Expenses of Management of Insurers transacting General or Health Insurance Business) Regulations, 2023*, Hyderabad, India, 2023.

[35] Central Statistics Office, Ministry of Statistics and Programme Implementation, *National Industrial Classification (All Economic Activities) 2008*. New Delhi, India: Government of India, 2008.

[36] S. Dambra, L. Bilge, and D. Balzarotti, "SoK: Cyber insurance – technical challenges and a system security roadmap," in *Proc. IEEE Symposium on Security and Privacy (SP)*, San Francisco, CA, USA, 2020, pp. 1367–1383.

[37] *IEEE Standard for Information Technology Systems Design Software Design Descriptions*, IEEE Std 1016-2009, 2009.

[38] *Systems and software engineering Life cycle processes Requirements engineering*, ISO/IEC/IEEE 29148:2018, 2018.

[39] Object Management Group, *Unified Modeling Language (UML), Version 2.5.1*, OMG Document formal/17-12-05, Dec. 2017.

[40] T. Bray, Ed., "The JavaScript Object Notation (JSON) data interchange format," IETF RFC 8259, Dec. 2017.

[41] Fitch Ratings, US standalone cyber insurance loss ratios, as reported in *Insurance Journal*, May 2024.

[42] National Association of Insurance Commissioners, *Report on the Cybersecurity Insurance Market*. Kansas City, MO, USA: NAIC, 2026.

[43] *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019, 2019.

[44] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," IETF RFC 2104, Feb. 1997.

[45] National Institute of Standards and Technology, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, Gaithersburg, MD, USA, Aug. 2015.

[46] E. Rescorla, "The Transport Layer Security (TLS) protocol version 1.3," IETF RFC 8446, Aug. 2018.

[47] *IEEE Standard for System, Software, and Hardware Verification and Validation*, IEEE Std 1012-2016, 2016.

[48] *Software and systems engineering Software testing Part 1: General concepts*, ISO/IEC/IEEE 29119-1:2022, 2022.

Appendix A: Soft Code Flowcharts

This appendix gives the flowcharts (UML activity diagrams) of the main procedures that are not already shown in Chapter 3. The flowchart of a V2 quote is Fig. 3.10.

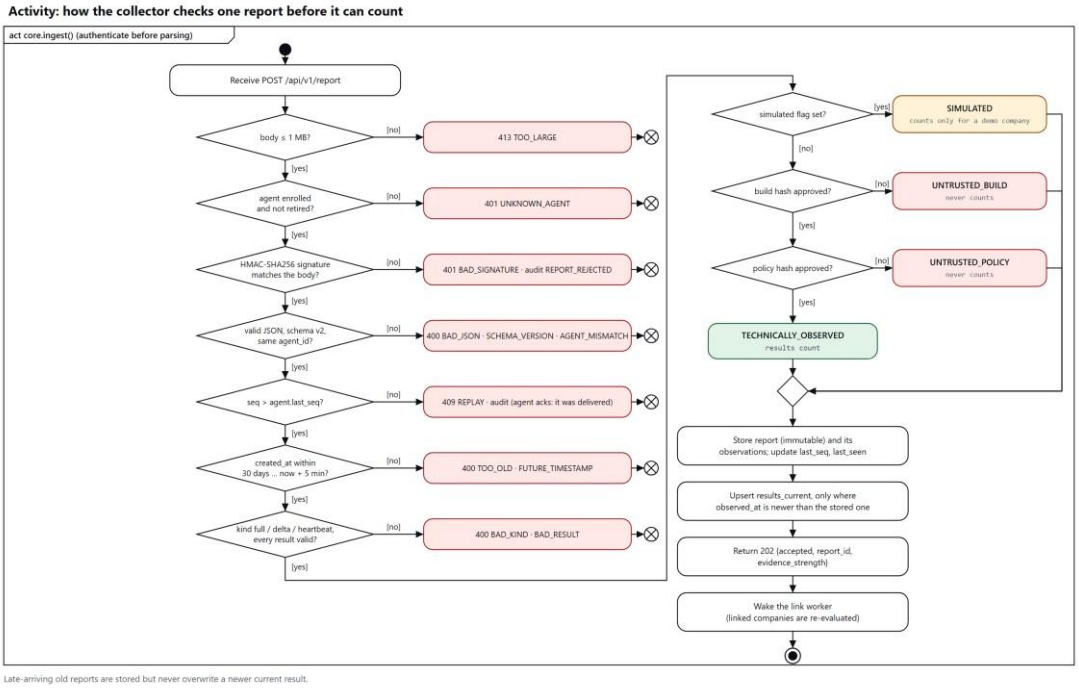


Fig. A.1 Flowchart: the collector's ingestion of an agent report

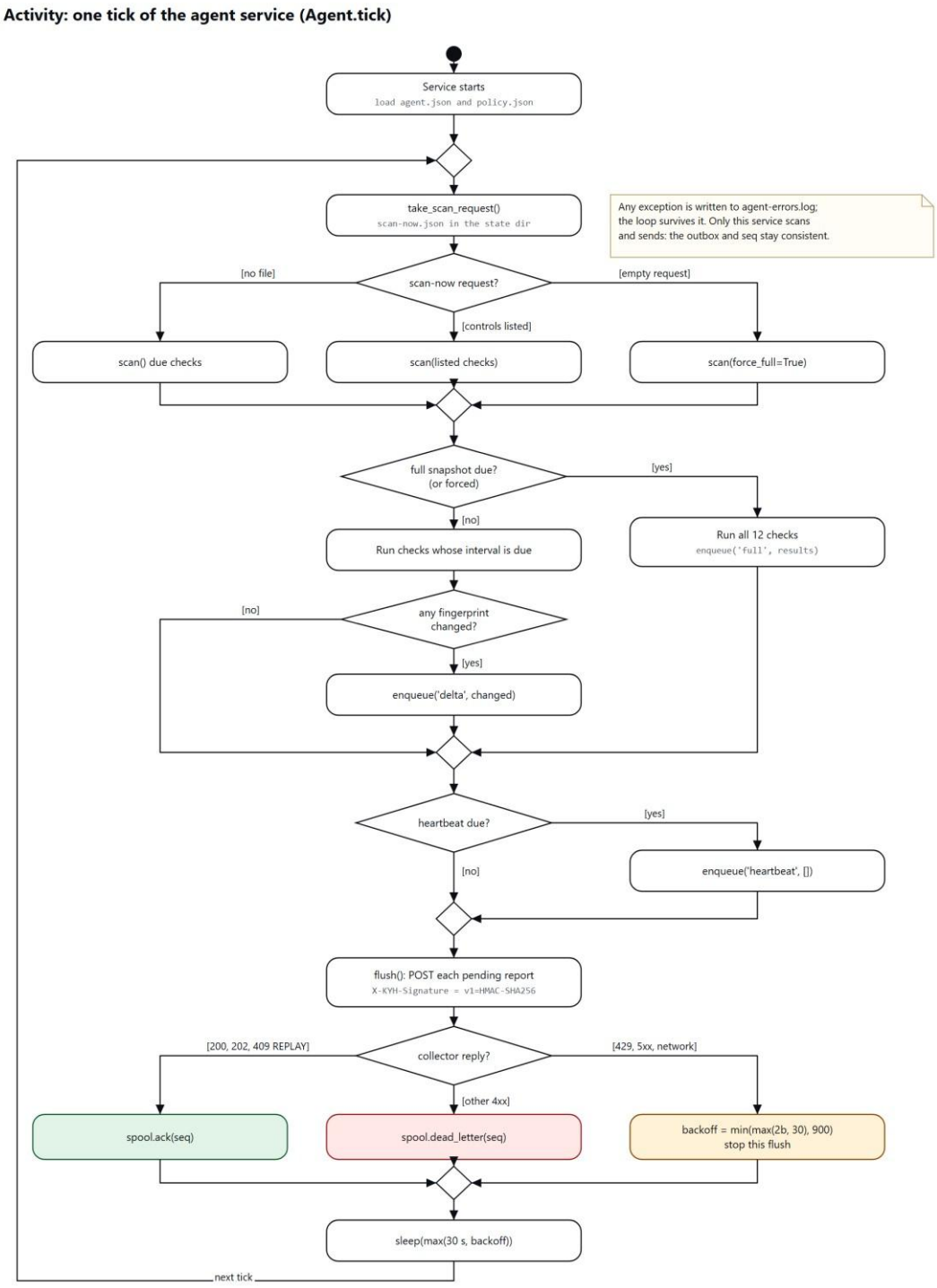


Fig. A.2 Flowchart: the agent's scheduling and reporting loop

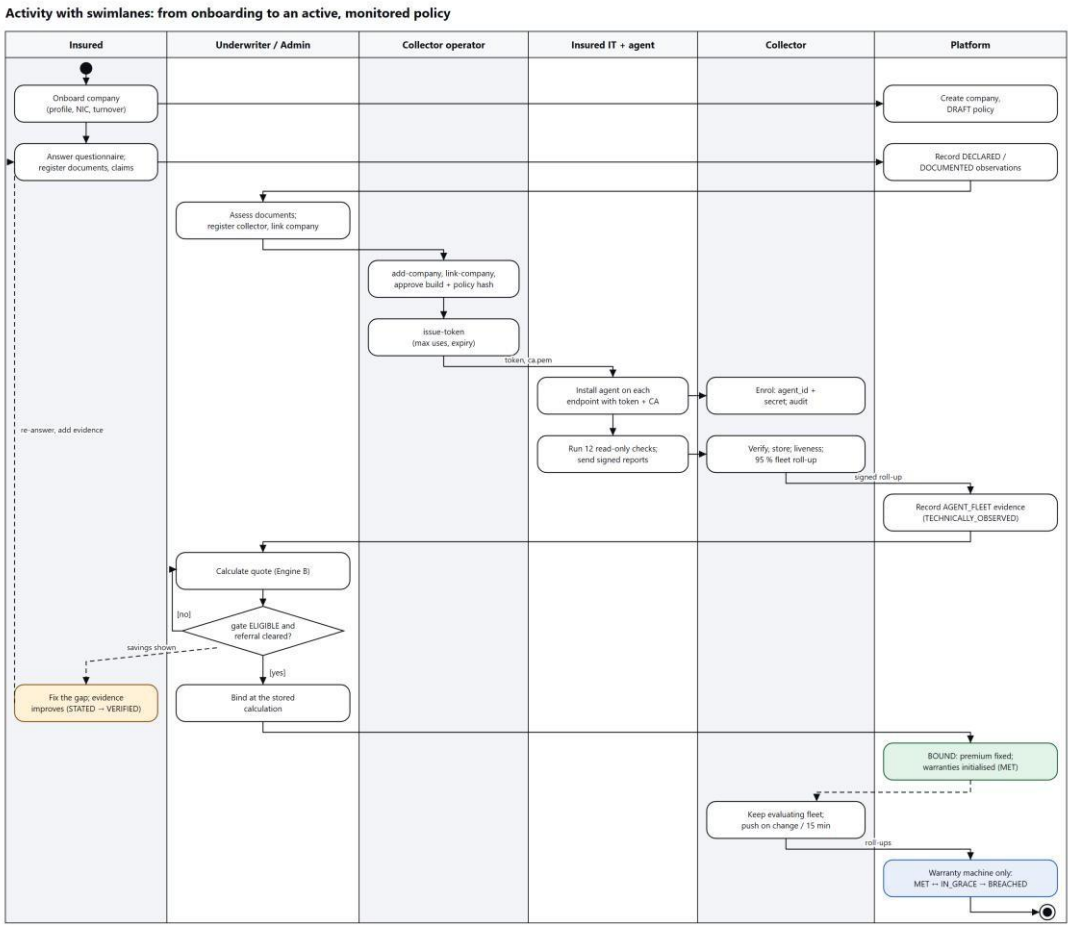


Fig. A.3 Flowchart: end to end, from intake to a breached warranty

The warranty evaluation that runs on every roll-up is:

```
Algorithm A.1  evaluate_policy(policy, now)
if policy.status != BOUND: return   for
each warranty w of the policy:
      e <- resolve_posture(company, w.control)         # strongest, most recent evidence
      if e is None: continue                             # missing evidence is never a
breach
      ok <- e.value in {true, partial}
      if w.state == MET and not ok:           move w to IN_GRACE, grace_until <- now +
grace_days(w)
      elif w.state == IN_GRACE and ok:       move w to REMEDIATED, then MET       elif
w.state == IN_GRACE and now > grace_until:  move w to BREACHED (consequence applies)
      elif w.state == BREACHED and ok:       move w to REMEDIATED, then MET
write every move to warranty_events and audit_events   return states,
repricing_performed = false
```

The collector's fleet roll-up for one control is:

```
Algorithm A.2  fleet_status(company, control, now)
observed <- []   for each host h of the company:
      if now - h.last_seen > 72 h and h ever reported control: observed.append(FAIL)
# silence rule
```

```
      elif h has a result r for control and r.status in {PASS, FAIL}:
observed.append(r.status)
  if observed is empty: return NOT_OBSERVABLE           # nothing is sent to the
platform
  share <- count(PASS in observed) / len(observed)
return PASS if share >= 0.95 else FAIL
```

Appendix B: Data Sheets

The full parameter file is `kyh-cyber-insurance/app/pricing_v2_params.json`; every entry has a value, unit, source, locator and tag. The tables below are extracts. Tags: [L] taken from a published source, [D] derived by a script, [A] an assumption with a sensitivity range.

Table B.1 Global values

Parameter	Value	Tag	Source
Global claim frequency	1.48% per policyyear	L	Coalition 2025, 2024 global frequency
Global average loss	$115,000	L	Coalition 2025, 2024 global average
Other claims share	6.2%	L	Coalition 2025 (reconciliation only)
Severity shape σ	2.193	D	NetDiligence 2025 SMEs; scripts/derive_sigma.py
Credit for STATED x	0.6	D	scripts/derive_credit_x.py, f = 0.2 [A]
Within-stage damping β	0.5	A	Judgement; tested 0 and 1
Per-control cap	[0.5, 2.0]	A	Judgement; tested [0.4, 2.5]
Combined bounds	[0.27, 3.0]	L	Coalition 2025
Claims credibility α	4.22	D	IRIS 2022; scripts/derive_alpha.py
Target loss ratio	0.50	L	Fitch 2024 (48%); NAIC 2026 (57%)
Expense share	30%	L	IRDAI 2023 expenses regulations
Trend	0	A	Tested ±10%
GST	18%	L	GST on general insurance premium
USD/INR	88.0	A	Placeholder; tested 83 and 92
India factors F_india, S_india	1.0	A	Switched off: no India claims data yet

Table B.2 Sector frequencies and average losses (Coalition 2025 unless noted)

Sector	Frequency	Frequency relativity	Average loss (USD)	Severity relativity
Consumer Discretionary	1.33%	0.899	118,493	1.030
Consumer Staples	2.60%	1.757	123,718	1.076
Energy	1.35%	0.912	108,000	0.939
Financial Services	1.47%	0.993	95,066	0.827
Healthcare	1.38%	0.932	144,662	1.258
Industrials	1.64%	1.108	104,741	0.911
Information Technology	1.36%	0.919	131,183	1.141

Materials	2.20%	1.486	99,030	0.861
Nonprofit	1.34%	0.905	73,498	0.639
Real Estate	1.02%	0.689	179,237	1.559
Education (IRIS / NetDiligence)	—	1.60	—	0.65
Sector	**Frequency**	**Frequency relativity**	**Average loss (USD)**	**Severity relativity**
Professional Services (IRIS / NetDiligence)	—	1.50	—	1.03
Public Sector (IRIS / NetDiligence)	—	1.34	—	0.68

Table B.3 Loss split by coverage head (shares of each peril's loss)

Peril	IR	BI	DR	Legal	TPL	Extortion	Notify	FTF
Ransomware	0.20	0.35	0.06	0.08	0.02	0.25	0.04	0
BEC	0.40	0	0	0.25	0.05	0	0.30	0
FTF	0.05	0	0	0.05	0	0	0	0.90
Other first-party	0.25	0.40	0.25	0.10	0	0	0	0
Third-party	0	0	0	0.30	0.70	0	0	0

The ransomware IR, BI and DR shares are derived from Coalition 2025; the other shares are placeholders [A].

Table B.4 Agent policy thresholds (policy kyh-endpoint-baseline 2.1.0)

Setting	Value	Provenance
Patch age	7 days	Matches Marsh 2023 "critical CVEs patched within 7 days"
Package index age	14 days	Assumption
EDR signature age	3 days	Assumption
Backup age	7 days	Assumption
Local administrators	At most 3	Assumption
Log retention	180 days	CERT-In Directions, 28 April 2022
Approved time sources	samay1.nic.in, samay2.nic.in, time.nplindia.org	CERT-In Directions
Liveness	UNVERIFIED after 6 h, SILENT after 72 h	Assumption
Fleet pass threshold	95% of observing hosts	Assumption
Monitoring coverage	90% of declared endpoints, 14 days after bind	Assumption

Table B.5 Platform API routes

Method and path	Permission
GET /api/meta, /api/regulatory, /api/forms	none (public reference data)
GET /api/me	any signed-in role
GET /api/dashboard, /api/audit	staff
GET /api/companies (customer: own company only)	read
POST /api/companies; PUT /api/companies/{id}	intake
GET /api/companies/{id}, /fleet; /api/posture/{id}; /api/questionnaires/{id}; /api/documents/{id}; /api/claims/{id}; /api/outside-in/{id}; /api/agent/status/{id}	read (customer: own company only)
Method and path	**Permission**
POST /api/questionnaires, /api/documents, /api/claims, /api/outside-in	intake
POST /api/documents/{id}/assess	assess
POST /api/quote; GET /api/quote/{id} (customer: trimmed view)	quote / read
POST /api/policies/bind	bind
GET /api/policies/{id}, /warranties, /warranty-events (customer: own policy only)	read
PUT /api/policies/{id}/coverages	quote
GET/PUT /api/admin/config, /api/admin/controls/{id}, /api/admin/sectors; GET /api/admin/pricing-params; /api/admin/collectors	admin
POST /api/collector/rollup	collector HMAC signature
POST /api/agent/report	legacy agent bearer token (to be disabled)
GET /api/demo/tokens	demonstration only (to be removed)

Appendix C: List of Components

Beretta has no hardware components beyond ordinary computers (Section 3.2). Its software components are listed below.

Table C.1 Platform (Part 1) modules

Module	Lines	Purpose
app/api.py	819	HTTP routes, permission checks, customer scoping, static files
app/pricing_v2.py	546	Engine B, model V2: classification, perils, payout, controls, claims, premium, gate, referral, what-if, reproduce
app/database.py	369	SQLite schema, migrations, seed companies
app/underwriting.py	276	V1 model (frozen), binding, referral approval
app/catalogue.py	216	Controls, sectors, coverage heads, default configuration
app/observations.py	199	Engine A: observations, best-evidence resolution, documents, signals
app/fleet.py	193	Collector registration and links, roll-up verification, AGENT_FLEET evidence
app/coverage.py	160	Warranty state machine and coverage consequences
app/security.py	89	Tokens, role permissions, rate limiting, input validation
app/audit.py	41	Append-only audit log
app/pricing_v2_params.json	—	Every V2 parameter with value, unit, source, locator and tag
frontend/ (index.html, app.js, styles.css)	1,654	Platform dashboard
scripts/ (derive_sigma, derive_credit_x, derive_alpha, worked_example_v2, sensitivity_v2, run_demo)	468	Model derivations, worked example and sensitivity

Table C.2 Monitoring (Part 2) components

Component	Purpose
agent/kyh_agent/checks/ (windows.py, linux.py, macos.py, parsers.py, verdicts.py)	Twelve read-only checks per OS, parsers and decision rules
agent/kyh_agent/ (runner, policy, security, spool, transport, simulate, config)	Guarded runner, policy file, signing, outbox, HTTPS client, labelled simulation
agent/kyh_agent/policy.json	Which checks run, how often, and every threshold
agent/install/	Installers for Windows, Linux and macOS
collector/kyh_collector/ (server, core, evaluate, db, settings)	Enrolment, ingestion gate, roll-up, liveness, standalone warranties, API
collector/kyh_collector/platform_link.py	Mapping to platform controls, change detection, signing and pushing roll-ups

collector/kyh_collector/static/	Collector dashboard under a strict Content-SecurityPolicy
run_linked_demo.py; integration/test_linked_system.py	Linked demonstration; end-to-end test of both systems
Component	**Purpose**
docs/diagrams/, docs/report/make_report_charts.py	Standard-library generators for every UML diagram and chart in this report

Appendix D: Source Code

This appendix lists the source code of the parts that carry the project's contributions: on the platform, Engine A (evidence), Engine B (the V2 pricing model), the warranty state machine, the verification of signed fleet roll-ups and the role permissions; on the endpoint agent, the reporting loop, report signing, the decision rules and the Windows checks; and on the collector, the ingestion gate, the fleet roll-up with the silence rule, and the signed link to the platform. Each file is printed in full, exactly as it is in the project folder. The other files (the HTTP routes, the two dashboards, the database schemas, the Linux and macOS checks, the installers, the command-line tools, the model scripts, the demonstration and the tests) are listed in Appendix C and are in the project repository. Lines longer than the page continue on the next line with an indent.

E.1 Platform (Part 1)

`kyh-cyber-insurance/app/observations.py` 199 lines

Engine A: observations, evidence strength and best-evidence resolution (Section 3.3.3).

```
"""Engine A — Information Gathering.

Every source writes immutable rows into `observations`. Resolution answers
"what is the current best-evidenced state of each control, and where did that
value come from?"
""" import json   from . import
audit, catalogue from .database
import new_id, now_iso

_STRENGTH_RANK = {s: i for i, s in enumerate(catalogue.EVIDENCE_STRENGTHS)}
VALID_VALUES = {"true", "false", "partial"}
    class ValidationError(Exception):     def
__init__(self, code, message, field=None):
        super().__init__(message)         self.code, self.message,
self.field = code, message, field

def record_observation(conn, company_id, control_id, value, source_type,
source_id=None, source_locator=None, evidence_strength=None,
confidence=1.0, dims=None, observed_at=None, notes=None,
actor="system"):
    """Append one observation. Observations are never updated in place —
corrections are new rows (history is preserved)."""     value =
str(value).lower()     if value in ("1", "yes", "pass"):
        value = "true"     if value
in ("0", "no", "fail"):
        value = "false"     if
value not in VALID_VALUES:
        raise ValidationError("INVALID_CONTROL_VALUE",
                              "Expected boolean value (true/false/partial)", control_id)
    if source_type not in catalogue.SOURCE_TYPES:
        raise ValidationError("INVALID_SOURCE_TYPE", f"Unknown source type {source_type}")
ctrl = conn.execute("SELECT control_id FROM controls WHERE control_id=?",
                        (control_id,)).fetchone()
if not ctrl:
```

```
        raise ValidationError("UNKNOWN_CONTROL", f"Unknown control {control_id}", control_id)
strength = evidence_strength or catalogue.SOURCE_DEFAULT_EVIDENCE[source_type]     if
strength not in _STRENGTH_RANK:
        raise ValidationError("INVALID_EVIDENCE_STRENGTH", f"Unknown strength {strength}")
# Existence vs currency vs tested: an untested/stale artefact cannot carry
    # TESTED strength, and dims themselves can downgrade the value.
if dims is not None:         if not isinstance(dims, dict):
            raise ValidationError("INVALID_DIMS", "dims must be an object")
if dims.get("tested") is False and strength == "TESTED":
            strength = "DOCUMENTED"         if
dims.get("current") is False and value == "true":
            value = "partial"
if dims.get("exists") is False:
            value = "false"     oid =
new_id("OBS")     ts = observed_at or
now_iso()     conn.execute("""INSERT INTO
observations
        (observation_id,company_id,control_id,value,dims,source_type,source_id,
observed_at,received_at,source_locator,evidence_strength,confidence,status,notes)
        VALUES (?,?,?,?,?,?,?,?,?,?,?,?, 'ACTIVE', ?)""",
        (oid, company_id, control_id, value,          json.dumps(dims) if dims else
None, source_type, source_id, ts, now_iso(),          source_locator, strength,
float(confidence), notes))     audit.log(conn, actor=actor,
event_type="CONTROL_OBSERVATION_CREATED",               entity_type="OBSERVATION",
entity_id=oid,               new_value={"company_id": company_id, "control_id":
control_id,
                        "value": value, "source_type": source_type,
                        "evidence_strength": strength,
"source_locator": source_locator})     return oid
    def resolve_posture(conn,
company_id):
    """For each control (15 priced + monitoring-only) pick the current best observation:
highest evidence strength wins; ties broken by most recent observed_at.     Returns
{control_id: resolved dict} with provenance and conflict flags."""     controls =
conn.execute("SELECT * FROM controls ORDER BY tier, control_id").fetchall()     posture =
{}     for c in controls:
        rows = conn.execute("""SELECT rowid AS _rid, * FROM observations
                            WHERE company_id=? AND control_id=? AND status='ACTIVE'
                            ORDER BY observed_at DESC, _rid DESC""",
(company_id, c["control_id"])).fetchall()         if not rows:
            posture[c["control_id"]] = dict(
                control_id=c["control_id"], name=c["name"], tier=c["tier"],
is_warranty=bool(c["is_warranty"]), monitoring_only=bool(c["monitoring_only"]),
status="UNKNOWN", value=None,                 evidence_strength=None, source_type=None,
source_locator=None,                observation_id=None, observed_at=None, dims=None,
conflict=False,                sources=[])             continue
        best = max(rows, key=lambda r: (_STRENGTH_RANK[r["evidence_strength"]],
r["observed_at"], r["received_at"], r["_rid"]))         values = {r["value"] for r in
rows}         posture[c["control_id"]] = dict(
            control_id=c["control_id"], name=c["name"], tier=c["tier"],
is_warranty=bool(c["is_warranty"]), monitoring_only=bool(c["monitoring_only"]),             status={"true":
"PASS", "false": "FAIL", "partial": "PARTIAL"}[best["value"]],             value=best["value"],
evidence_strength=best["evidence_strength"],             source_type=best["source_type"],
source_locator=best["source_locator"],             observation_id=best["observation_id"],
observed_at=best["observed_at"],             dims=json.loads(best["dims"]) if best["dims"] else None,
            conflict=len(values) > 1,
sources=[dict(observation_id=r["observation_id"], value=r["value"],
source_type=r["source_type"], source_locator=r["source_locator"],
evidence_strength=r["evidence_strength"],
observed_at=r["observed_at"], notes=r["notes"]) for r in rows])     return posture


# Outside-in scan findings are evidence for a control (pricing V2 has no separate exposure modifier).
# signal type -> (control, value). An explicit related control on the signal wins; its value is "false"
# for an adverse finding.
SIGNAL_EVIDENCE = {
    "EXPOSED_RDP": ("CTRL-SECCONFIG", "false"),
    "EXPOSED_SMB": ("CTRL-SECCONFIG", "false"),
    "TLS_WEAK": ("CTRL-SECCONFIG", "false"),
```

```
    "EOL_SOFTWARE": ("CTRL-SECCONFIG", "false"),
    "UNPATCHED_CRITICAL_CVE": ("CTRL-PATCH-CRIT", "false"),
    "DMARC_MISSING": ("CTRL-EMAILSEC", "false"),
    "DMARC_PRESENT": ("CTRL-EMAILSEC", "true"),
}
    def record_signal(conn, company_id, signal_type, detail=None, related_control_id=None,
actor="system"):
    """Store an imported outside-in signal and, where it speaks to a control, the observation it
     implies."""
    signal_type = str(signal_type).strip().upper()     mapped = SIGNAL_EVIDENCE.get(signal_type)
control, value = (related_control_id, "false") if related_control_id else (mapped or (None, None))
severity = "POSITIVE" if value == "true" else "ADVERSE"     sid = new_id("SIG")     ts = now_iso()
conn.execute("INSERT INTO outside_in_signals VALUES (?,?,?,?,?,?,?,?)",                  (sid,
company_id, signal_type, detail, severity, control, ts, ts))     audit.log(conn, actor=actor,
event_type="OUTSIDE_IN_SIGNAL_RECORDED", entity_type="SIGNAL", entity_id=sid,
             new_value={"company_id": company_id, "signal_type": signal_type, "related_control_id":
     control})
    oid = None
if control:
        oid = record_observation(conn, company_id, control, value, source_type="OUTSIDE_IN_SCAN",
     source_id=sid,
                                 source_locator=f"Outside-in signal {signal_type}", notes=detail,
     actor=actor)
    return sid, oid


DOCUMENT_CATEGORIES = {
    "CAIQ": ("STANDARDIZED_QUESTIONNAIRE", "DOCUMENTED"),
    "SIG_LITE": ("STANDARDIZED_QUESTIONNAIRE", "DOCUMENTED"),
    "SOC2_TYPE2": ("AUDIT_ARTIFACT", "DOCUMENTED"),
    "ISO27001_SOA": ("AUDIT_ARTIFACT", "DOCUMENTED"),
    "VAPT_REPORT": ("AUDIT_ARTIFACT", "DOCUMENTED"),
    "SCAN_EXPORT": ("TECHNICAL_ARTIFACT", "DOCUMENTED"),
    "CONFIG_REPORT": ("TECHNICAL_ARTIFACT", "DOCUMENTED"),
    "ENDPOINT_REPORT": ("TECHNICAL_ARTIFACT", "DOCUMENTED"),
    "IDP_MFA_EXPORT": ("TECHNICAL_ARTIFACT", "DOCUMENTED"),   # identity-provider MFA enforcement export
    "BACKUP_TEST_REPORT": ("TESTING_EVIDENCE", "TESTED"),
    "IR_EXERCISE_EVIDENCE": ("TESTING_EVIDENCE", "TESTED"),
    "PENTEST_EVIDENCE": ("TESTING_EVIDENCE", "TESTED"),
    "NARRATIVE_POLICY": ("NARRATIVE", "DECLARED"),
    "SECURITY_DESCRIPTION": ("NARRATIVE", "DECLARED"),
}
    def register_document(conn, company_id, document_type, filename, actor):
"""Documents are metadata + assessed fields; uploads are never executed and
the prototype does not pretend to run AI extraction — assessed fields are
entered through a transparent workflow with locators."""     if document_type
not in DOCUMENT_CATEGORIES:
        raise ValidationError("INVALID_DOCUMENT_TYPE",
                              f"Unsupported document_type {document_type}", "document_type")
safe_name = "".join(ch for ch in filename if ch.isalnum() or ch in "._- ")[:120] or "document"
category, strength = DOCUMENT_CATEGORIES[document_type]     did = new_id("DOC")
conn.execute("INSERT INTO documents VALUES (?,?,?,?,?,?,?,?)",
                (did, company_id, document_type, category, safe_name, now_iso(),
                 "PENDING", strength))     audit.log(conn, actor=actor,
event_type="DOCUMENT_UPLOADED", entity_type="DOCUMENT",               entity_id=did,
new_value={"company_id": company_id,
                                       "document_type": document_type,
                                       "filename": safe_name, "category": category})
return did
    def assess_document_field(conn, document_id, control_id, value,
source_locator,                           dims=None, evidence_strength=None,
actor="underwriter"):
    doc = conn.execute("SELECT * FROM documents WHERE document_id=?", (document_id,)).fetchone()
if not doc:
        raise ValidationError("UNKNOWN_DOCUMENT", f"No document {document_id}")
if not source_locator:
        raise ValidationError("MISSING_SOURCE_LOCATOR",
                              "Every extracted value needs a source locator", "source_locator")
strength = evidence_strength or doc["default_evidence_strength"]     oid =
```

```
record_observation(conn, company_id=doc["company_id"], control_id=control_id,
value=value, source_type="DOCUMENT", source_id=document_id,
source_locator=source_locator, evidence_strength=strength,
dims=dims, actor=actor)     conn.execute("UPDATE documents SET assessment_status='ASSESSED'
WHERE document_id=?",
                (document_id,))     return oid
```

kyh-cyber-insurance/app/pricing_v2.py 546 lines

Engine B, model V2: classification, perils, lognormal payout, controls, claims credibility, premium, gate, referral, what-if and reproduction (Section 3.3.4).

```
"""Engine B, UNDERWRITING_MODEL_V2: a frequency-severity premium model calibrated to published claims data.
    For each peril p:
      lambda_p = lambda_p,global x R_turnover_freq x R_sector_freq x F_india,p x M_controls_freq,p x
     M_claims
      X_p      ~ Lognormal(mean = m_p x R_turnover_sev x R_sector_sev x S_india, sigma)
      payout_p = E[ min( sum_h s_p,h x M_sev,p,h x (X_p - d)+ , L_total ) ]      (no per-cover sub-limits)
    EL        = sum_p lambda_p x payout_p
Technical = EL x (1 + trend) / TLR
    Customer  = Technical x (1 + GST)

`price(inputs, params)` is a pure function: the same inputs and parameters always give the same result, and
the result carries every intermediate value. `calculate_premium(conn, policy_id)` collects the inputs from
the database, prices them, and stores the inputs and the parameters as the snapshot, so any V2 quote can be
reproduced later with `price(snapshot["inputs"], snapshot["params"])`.

Every number comes from pricing_v2_params.json, where each one carries its source, locator and tag.
""" import copy import json import math
import os from datetime import
datetime, timezone from functools
import lru_cache from statistics import
NormalDist

from . import audit, observations
from .database import get_config, new_id, now_iso

MODEL_VERSION = "UNDERWRITING_MODEL_V2"
PERILS = ["RANSOMWARE", "BEC", "FTF", "MISC_FIRST_PARTY", "THIRD_PARTY"]
STAGES = ["CREDENTIAL", "WEAKNESS", "EXECUTION"]
STATUSES = ["VERIFIED", "STATED", "NOT_WORKING", "UNKNOWN"]
PARAMS_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), "pricing_v2_params.json")
ROUND = 2
    class EngineError(Exception):
def __init__(self, code, message):
        super().__init__(message)
self.code, self.message = code, message
    def load_params():     with open(PARAMS_PATH,
encoding="utf-8") as fh:
        return json.load(fh)
    def
v(entry):
    """The value of a parameter entry (entries are {value, unit, source, locator, tag})."""
return entry["value"] if isinstance(entry, dict) and "value" in entry else entry


# ============================================================== classification def
nic_to_sector(nic_code, prm):
    """NIC 2008 division (2 digits) -> rating sector. Unknown or missing -> global average."""
try:
        div = int(str(nic_code).strip()[:2])
except (TypeError, ValueError):
        return prm["sector"]["default_sector"], False     for
lo, hi, sector in prm["sector"]["nic_map"]["ranges"]:
if lo <= div <= hi:             return sector, True
return prm["sector"]["default_sector"], False
    def banded(r_usd, bands,
zone):
```

```
    """Band value at turnover r_usd, with log-linear transition zones [b/zone, b*zone] at every boundary b.
Returns (value, zone_boundary or None)."""     values = [v(b) for b in bands]     bounds = [b["upper_usd"]
for b in bands[:-1]]     for i, b in enumerate(bounds):
        lo_edge, hi_edge = b / zone, b * zone
if lo_edge <= r_usd <= hi_edge:
            f_lo, f_hi = values[i], values[i + 1]             t =
math.log(r_usd / lo_edge) / math.log(zone * zone)
return f_lo * (f_hi / f_lo) ** t, b     for i, b in
enumerate(bounds):         if r_usd < b:
            return values[i], None
return values[-1], None
    def classify(inputs,
prm):
    usd_inr = v(prm["currency"]["usd_inr"])
r_usd = float(inputs["turnover_inr"]) / usd_inr
zone = v(prm["turnover"]["zone_factor"])
    f_turn, f_zone = banded(r_usd, prm["turnover"]["freq_bands"], zone)     s_turn, s_zone =
banded(r_usd, prm["turnover"]["sev_bands"], zone)     g_freq, g_sev =
v(prm["global"]["claims_frequency"]), v(prm["global"]["average_loss_usd"])     sector, matched
= nic_to_sector(inputs.get("nic_code"), prm)     sp = prm["sector"]     if sector in
sp["frequency"]:
        sec_f = v(sp["frequency"][sector]) / g_freq
elif sector in sp["frequency_relativity_direct"]:
        sec_f = v(sp["frequency_relativity_direct"][sector])
else:
        sec_f = 1.0     if sector in
sp["severity_usd"]:
        sec_s = v(sp["severity_usd"][sector]) / g_sev
elif sector in sp["severity_relativity_direct"]: sec_s =
v(sp["severity_relativity_direct"][sector])
    else:
        sec_s = 1.0     return dict(nic_code=inputs.get("nic_code"), nic_matched=matched,
sector=sector,                 turnover_inr=float(inputs["turnover_inr"]), turnover_usd=r_usd,
usd_inr=usd_inr,                   turnover_frequency=f_turn, turnover_freq_relativity=f_turn /
g_freq,                 turnover_severity_usd=s_turn, turnover_sev_relativity=s_turn / g_sev,
frequency_zone=f_zone, severity_zone=s_zone,                 sector_freq_relativity=sec_f,
sector_sev_relativity=sec_s)


# ================================================================ controls def
m_has_lacks(S, p):
    """Centred multipliers for a control with relative risk S and adoption p:
p x m_has + (1 - p) x m_lacks = 1."""     denom = p + (1 - p) * S     return
1.0 / denom, S / denom
    def control_multiplier(S, p, status,
prm):
    lo, hi = v(prm["controls"]["multiplier_cap"])     has,
lacks = m_has_lacks(S, p)     has, lacks = min(max(has, lo),
hi), min(max(lacks, lo), hi)     if status == "VERIFIED":
        return has     if
status == "STATED":
        return 1 - v(prm["controls"]["credit_x"]) * (1 - has)
return lacks  # NOT_WORKING or UNKNOWN
    def combine(mults,
beta):
    """Overlapping controls: sort by |ln m| descending, ln M = sum_k beta^k ln m_k."""
logs = sorted((math.log(m) for m in mults), key=abs, reverse=True)     return
math.exp(sum((beta ** k) * lm for k, lm in enumerate(logs)))
    def evidence_status(entry, document_types,
prm):
    """Map Engine A's resolved control (status + evidence strength + source) to a V2 evidence status."""
if entry is None or entry.get("status") in (None, "UNKNOWN"):
        return "UNKNOWN"
st = entry["status"]     if
st == "FAIL":
        return "NOT_WORKING"
if st == "PARTIAL":
return "STATED"
```

```
    strength = entry.get("evidence_strength")     if
strength in ("TECHNICALLY_OBSERVED", "TESTED"):
        return "VERIFIED"     if
strength == "DOCUMENTED":
        doc_type = document_types.get(entry.get("source_id") or "")
if doc_type in v(prm["controls"]["verified_document_types"]):
            return "VERIFIED"
return "STATED"     return
"STATED"  # DECLARED
    def control_effects(statuses, prm,
overrides=None):
    """Combined frequency multiplier per peril and severity multiplier per peril x head, with every
individual control multiplier for the breakdown."""     st = dict(statuses)     st.update(overrides
or {})     cp = prm["controls"]     beta = v(cp["beta"])     floor, ceil = v(cp["combined_bounds"])
heads = prm["heads"]["order"]     freq, freq_detail = {}, {}     for peril in PERILS:
        stages, detail = {}, []         for
cid, c in cp["frequency"].items():
if peril not in c["perils"]:
                continue             m = control_multiplier(c["S"], c["p"], st.get(cid,
"UNKNOWN"), prm)             stages.setdefault(c["stage"], []).append(m)
detail.append(dict(control_id=cid, stage=c["stage"], status=st.get(cid, "UNKNOWN"),
multiplier=m, source=c["source"], tag=c["tag"]))         per_stage = {s: combine(ms, beta) for
s, ms in stages.items()}         raw = math.prod(per_stage.values()) if per_stage else 1.0
clamped = min(max(raw, floor), ceil)         freq[peril] = clamped         freq_detail[peril] =
dict(controls=detail, per_stage=per_stage, raw=raw, value=clamped,
hit_ceiling=raw >= ceil, hit_floor=raw <= floor)     sev, sev_detail = {}, {}     for peril in
PERILS:         for h in heads:
            ms, detail = [], []             for
cid, c in cp["severity"].items():
                t = c["targets"]                 if t !=
"ALL" and h not in t.get(peril, []):
                    continue                 m = control_multiplier(c["S"], c["p"], st.get(cid,
"UNKNOWN"), prm)                 ms.append(m)
detail.append(dict(control_id=cid, status=st.get(cid, "UNKNOWN"), multiplier=m))
raw = combine(ms, beta) if ms else 1.0             sev[(peril, h)] = min(max(raw, floor), ceil)
sev_detail[f"{peril}|{h}"] = dict(controls=detail, value=sev[(peril, h)])     return freq,
freq_detail, sev, sev_detail


# ============================================================== severity and payout
@lru_cache(maxsize=8) def
_z_grid(n_main, n_tail):
    """Standard-normal quantiles on a mid-point grid: n_main cells over [0, 0.99), n_tail over [0.99, 1).
Returns (z values, probability weights)."""     nd = NormalDist()     zs, ws = [], []     w = 0.99 /
n_main     zs += [nd.inv_cdf((i + 0.5) * w) for i in range(n_main)]     ws += [w] * n_main     w2 = 0.01
/ n_tail     zs += [nd.inv_cdf(0.99 + (i + 0.5) * w2) for i in range(n_tail)]     ws += [w2] * n_tail
return tuple(zs), tuple(ws)
    def lev(u, mu,
sigma):
    """Closed-form limited expected value E[min(X, u)] of a lognormal."""
if u <= 0:         return 0.0     nd = NormalDist()     lu = math.log(u)
    return (math.exp(mu + sigma * sigma / 2) * nd.cdf((lu - mu - sigma * sigma) / sigma)
+ u * (1 - nd.cdf((lu - mu) / sigma)))
    def expected_payout(mu, sigma, retention, factors, limit_total,
prm):
    """E[ min( sum_h f_h x (X - d)+, L_total ) ] by integration over the lognormal on a quantile grid.
factors are per head (f_h = s_p,h x M_sev,p,h). There are no per-cover sub-limits: the overall limit
is the only cap. Returns (total, per-head expected payouts)."""     zs, ws =
_z_grid(int(v(prm["severity_shape"]["quantile_points"])),
int(v(prm["severity_shape"]["tail_points"])))     heads = [(h, f) for h, f in factors.items() if f > 0]
total, per_head = 0.0, {h: 0.0 for h, _ in heads}     for z, w in zip(zs, ws):
        x = math.exp(mu + sigma * z) - retention
if x <= 0:             continue         paid =
[(h, f * x) for h, f in heads]         s = sum(p
for _, p in paid)
        scale = 1.0 if s <= limit_total else limit_total / s
        total += w * min(s, limit_total)         for h, p in paid:             per_head[h] +=
w * p * scale   # the aggregate cap is shared pro rata across heads     return total, per_head
```

```
# ============================================================ the model def
price(inputs, prm, overrides=None, with_what_if=True):
    """Price one policy. inputs: nic_code, turnover_inr, years_in_operation, regulated_rbi_sebi,
material_incidents_3y, limit_total_inr, retention_inr, statuses {control_id: VERIFIED|STATED|...},
evidence (optional, for display). overrides: statuses to replace
    (used by what-if). Returns the full breakdown."""
statuses = dict(inputs["statuses"])
statuses.update(overrides or {})     cls =
classify(inputs, prm)     sigma =
v(prm["severity_shape"]["sigma"])     f_india =
v(prm["india"]["F_india"])     s_india =
v(prm["india"]["S_india"])     usd_inr =
cls["usd_inr"]
    L_total = float(inputs["limit_total_inr"])
d = float(inputs["retention_inr"])     heads =
prm["heads"]["order"]
      freq_m, freq_detail, sev_m, sev_detail = control_effects(statuses,
prm)

    # frequency before claims history
peril_rows = {}     for p in PERILS:
        base = v(prm["perils"][p]["frequency"])         lam = (base *
cls["turnover_freq_relativity"] * cls["sector_freq_relativity"]
              * f_india[p] * freq_m[p])
peril_rows[p] = dict(peril=p, base_frequency=base,
turnover_relativity=cls["turnover_freq_relativity"],
                             sector_relativity=cls["sector_freq_relativity"], india_factor=f_india[p],
control_multiplier=freq_m[p], control_detail=freq_detail[p],
frequency_before_claims=lam)     lam_firm = sum(r["frequency_before_claims"] for r in
peril_rows.values())

    # claims history (credibility)     alpha =
v(prm["claims"]["alpha"])     lookback =
v(prm["claims"]["lookback_years"])     years =
inputs.get("years_in_operation")
    T = min(lookback, float(years)) if years not in (None, "") else float(lookback)
    N = int(inputs.get("material_incidents_3y") or 0)
    M_claims = (alpha + N) / (alpha + lam_firm * T)
      el = 0.0     for p, row in
peril_rows.items():
        lam = row["frequency_before_claims"] * M_claims
        mean_usd = (v(prm["perils"][p]["mean_loss_usd"]) * cls["turnover_sev_relativity"]
                    * cls["sector_sev_relativity"] * s_india)
mean_inr = mean_usd * usd_inr         mu = math.log(mean_inr)
- sigma * sigma / 2         shares =
v(prm["heads"]["shares"][p])
        factors = {h: shares[h] * sev_m[(p, h)] for h in heads}
        payout, per_head = _cached_payout(mu, sigma, d, tuple(sorted(factors.items())), L_total, prm)
row.update(claims_multiplier=M_claims, frequency=lam, severity_mean_inr=mean_inr,
severity_mean_usd=mean_usd, mu=mu, sigma=sigma,
                   severity_multipliers={h: sev_m[(p, h)] for h in heads if shares[h] > 0},
payout=payout, payout_per_head=per_head, expected_loss=lam * payout)         el += lam *
payout
      trend = v(prm["loadings"]["trend"])     tlr =
v(prm["loadings"]["tlr"])     expense =
v(prm["loadings"]["expense_ratio"])     gst_rate =
v(prm["loadings"]["gst"])     technical = round(el *
(1 + trend) / tlr, ROUND)     gst = round(technical
* gst_rate, ROUND)
    total = round(technical + gst, ROUND)
      gating = gate(statuses, inputs.get("gate_mode", "BLOCK"),
prm)     referral = refer(cls, freq_detail, N, inputs, prm)
      result = dict(         model_version=MODEL_VERSION,         classification=cls,
policy_structure=dict(limit_total_inr=L_total, retention_inr=d),         statuses=statuses,
perils=[peril_rows[p] for p in PERILS],         severity_controls=sev_detail,
claims=dict(alpha=alpha, incidents=N, years_used=T, lambda_firm=lam_firm, multiplier=M_claims),
expected_loss=round(el, ROUND),         loadings=dict(trend=trend, tlr=tlr, expense_ratio=expense,
margin_ratio=round(1 - tlr - expense,
```

```
    4),                       split_inr=dict(expected_loss=round(el * (1 +
trend), ROUND),                                    expenses=round(technical *
expense, ROUND),
                                    profit_risk_reinsurance=round(technical * (1 - tlr - expense),
    ROUND))),         technical_premium=technical, gst_rate=gst_rate, gst=gst,
total_premium=total,         eligibility=gating, referral=referral,
assumptions=assumptions_used(prm))     if with_what_if:         result["what_if"] =
what_if(dict(inputs, statuses=statuses), prm, statuses, technical)     return result


_payout_cache = {}
    def _cached_payout(mu, sigma, d, factors, L_total,
prm):
    """Payouts depend only on severity inputs, so what-if runs that change only frequency controls reuse
     them."""
    key = (round(mu, 12), sigma, d, factors, L_total,
v(prm["severity_shape"]["quantile_points"]), v(prm["severity_shape"]["tail_points"]))     if key
not in _payout_cache:         if len(_payout_cache) > 2000:
            _payout_cache.clear()
        _payout_cache[key] = expected_payout(mu, sigma, d, dict(factors), L_total, prm)
return _payout_cache[key]
    def
priced_controls(prm):
    return sorted(set(prm["controls"]["frequency"]) | set(prm["controls"]["severity"]))
    def what_if(inputs, prm, statuses,
technical):
    """For each priced control not already VERIFIED: the premium with that control VERIFIED, and the
     saving."""
    out = []     for cid in
priced_controls(prm):         if
statuses.get(cid) == "VERIFIED":
            continue         r = price(inputs, prm, overrides={cid: "VERIFIED"},
with_what_if=False)         out.append(dict(control_id=cid,
current_status=statuses.get(cid, "UNKNOWN"),
premium_if_verified=r["technical_premium"],
saving_inr=round(technical - r["technical_premium"], ROUND)))     out.sort(key=lambda
x: x["saving_inr"], reverse=True)     return out


# ============================================================== gating and referral def
gate(statuses, gate_mode, prm):
    g = prm["gating"]
stated, failing = [], []
for cid in g["warranties"]:
        s = statuses.get(cid, "UNKNOWN")         if s == "VERIFIED" or (s ==
"STATED" and cid in g["stated_passes"]):             continue
        (stated if s == "STATED" else failing).append(cid)
if failing:
        status = "INELIGIBLE" if gate_mode == "BLOCK" else "CONDITIONALLY_ELIGIBLE"
reason = ("Warranty not met or not evidenced: " + ", ".join(failing) +
                  (". Binding is blocked." if gate_mode == "BLOCK"
else ". Needs remediation or an underwriter override before bind."))     elif stated:
        status = "CONDITIONALLY_ELIGIBLE"         reason = "Warranty stated but not verified: " + ",
".join(stated) + ". Evidence is needed before bind."
    else:
        status, reason = "ELIGIBLE", "Every warranty is verified (MFA may be stated)."     return
dict(status=status, reason=reason, failing_warranties=failing, stated_warranties=stated,
gate_mode=gate_mode, note="Gating never changes the price.")
    def refer(cls, freq_detail, N, inputs,
prm):
    r = prm["referral"]     reasons = []     if
cls["turnover_usd"] > v(r["turnover_usd_max"]):
        reasons.append(f"Turnover above USD {v(r['turnover_usd_max']) / 1e9:.0f}B.")
hit = [p for p, dct in freq_detail.items() if dct["hit_ceiling"]]     if hit:
        reasons.append("Control multiplier at the 3.0 ceiling for: " + ", ".join(hit) + ".")
if N >= v(r["incidents_min"]):
        reasons.append(f"{N} material incidents in the last 3 years.")
if inputs.get("regulated_rbi_sebi"):
```

```
        reasons.append("RBI- or SEBI-regulated entity.")
return dict(required=bool(reasons), reasons=reasons)
    def
assumptions_used(prm):
    """Every [A] parameter in the file, with its sensitivity range."""
out = []
      def walk(node, path):         if isinstance(node, dict):
if node.get("tag") == "A" or node.get("p_tag") == "A":
                out.append(dict(parameter=path, value=node.get("value", {k: node.get(k) for k in ("S", "p")
     if k in node}),
                                locator=node.get("locator"), sensitivity=node.get("sensitivity")))
for k, sub in node.items():                 if isinstance(sub, (dict, list)) and k not in
("value",):
                    walk(sub, f"{path}.{k}" if path else k)
elif isinstance(node, list):             for i, sub in
enumerate(node):                 if isinstance(sub, dict):
walk(sub, f"{path}[{i}]")     walk(prm, "")     return out


# ============================================================ database layer def
collect_inputs(conn, policy_id):
    pol = conn.execute("SELECT * FROM policies WHERE policy_id=?", (policy_id,)).fetchone()
if not pol:
        raise EngineError("UNKNOWN_POLICY", f"No policy {policy_id}")
    co = conn.execute("SELECT * FROM companies WHERE company_id=?", (pol["company_id"],)).fetchone()
prm = load_params()     posture = observations.resolve_posture(conn, co["company_id"])     src_ids =
{}
    for r in conn.execute("SELECT observation_id, source_id FROM observations WHERE company_id=?",
                          (co["company_id"],)).fetchall():
        src_ids[r["observation_id"]] = r["source_id"]
    doc_types = {r["document_id"]: r["document_type"] for r in conn.execute(
        "SELECT document_id, document_type FROM documents WHERE company_id=?",
     (co["company_id"],)).fetchall()}
statuses, evidence = {}, {}
    for cid in priced_controls(prm) + list(prm["gating"]["warranties"]):
        e = posture.get(cid)
if e is not None:
            e = dict(e, source_id=src_ids.get(e.get("observation_id")))         statuses[cid] =
evidence_status(e, doc_types, prm)         evidence[cid] = dict(name=(e or {}).get("name", cid),
resolved_status=(e or {}).get("status"),                                evidence_strength=(e or
{}).get("evidence_strength"),                                source_type=(e or {}).get("source_type"),
source_locator=(e or {}).get("source_locator"),                                observation_id=(e or
{}).get("observation_id"),                                v2_status=statuses[cid])     covs =
conn.execute("SELECT * FROM policy_coverages WHERE policy_id=?", (policy_id,)).fetchall()
limit_total = pol["limit_total_inr"] or max([c["limit_inr"] for c in covs] or [0])     retention =
pol["retention_inr"] if pol["retention_inr"] is not None else \         max([c["deductible_inr"] or 0
for c in covs] or [0])     years = co["years_in_operation"]     n_material = material_incidents(conn,
co["company_id"], retention)     return pol, co, prm, posture, dict(         nic_code=co["nic_code"],
turnover_inr=co["annual_revenue_inr"] or 0,         years_in_operation=years,
regulated_rbi_sebi=bool(co["regulated_rbi_sebi"]),         material_incidents_3y=n_material,
limit_total_inr=limit_total, retention_inr=retention,         statuses=statuses, evidence=evidence,
gate_mode=get_config(conn, "warranty_gate_mode") or "BLOCK")
    def material_incidents(conn, company_id,
retention):
    """Material incidents in the last 3 years: a loss above the retention, a DPDP/CERT-In notification,
or a ransom decision. An explicit `material` flag on the claim wins; otherwise it is inferred."""
cutoff = datetime.now(timezone.utc).year - 3     n = 0     for c in conn.execute("SELECT * FROM claims
WHERE company_id=?", (company_id,)).fetchall():         try:             if
int(str(c["incident_date"])[:4]) < cutoff:
                continue
except ValueError:
            continue         flag = c["material"] if "material" in c.keys() else None         if flag
is not None:             n += int(bool(flag))         elif (c["loss_amount_inr"] or 0) > retention or
"ransom" in (c["incident_type"] or "").lower():
            n += 1
return n
    def calculate_premium(conn, policy_id, actor="underwriter",
persist=True):
```

```
    pol, co, prm, posture, inputs = collect_inputs(conn, policy_id)
if pol["status"] == "BOUND":
        raise EngineError("POLICY_ALREADY_BOUND",
                          "This policy is bound. Post-bind observations evaluate warranties; "
                          "they do not re-price the policy.")
if not inputs["limit_total_inr"]:
        raise EngineError("NO_LIMIT", "Set an overall limit for the policy before quoting.")
r = price(inputs, prm)     r["company"] = dict(company_id=co["company_id"], name=co["name"],
nic_code=co["nic_code"],                          annual_revenue_inr=co["annual_revenue_inr"],
years_in_operation=co["years_in_operation"],
                        regulated_rbi_sebi=bool(co["regulated_rbi_sebi"]), geography=co["geography"])
r["evidence"] = inputs["evidence"]     r["explanation"] = explain(r)
    r["assumptions_note"] = ("Calibrated to published claims data (mostly US); several parameters are "
                             "assumptions, listed in this quote. Not a filed rate.")
calc_id = new_id("CALC")
    r["calculation_id"], r["calculated_at"] = calc_id, now_iso()
if persist:
        snapshot = dict(inputs=inputs, params=prm, posture=posture)
conn.execute("INSERT INTO calculations VALUES (?,?,?,?,?,?,?,?,?)",
                     (calc_id, co["company_id"], policy_id, r["calculated_at"], MODEL_VERSION,
json.dumps(snapshot), json.dumps(r), r["technical_premium"], r["eligibility"]["status"]))
        conn.execute("UPDATE policies SET status='QUOTED' WHERE policy_id=? AND status='DRAFT'",
     (policy_id,))         audit.log(conn, actor=actor, event_type="PREMIUM_CALCULATED",
entity_type="CALCULATION",                    entity_id=calc_id,
new_value=dict(policy_id=policy_id, technical_premium=r["technical_premium"],
                                                total_premium=r["total_premium"],
eligibility=r["eligibility"]["status"],
referral=r["referral"]["required"], model_version=MODEL_VERSION))
        audit.log(conn, actor=actor, event_type="POLICY_QUOTED", entity_type="POLICY", entity_id=policy_id,
new_value=dict(calculation_id=calc_id, technical_premium=r["technical_premium"]))     return r
    def
reproduce(snapshot):
    """Re-run a stored V2 quote from its snapshot (inputs + parameters as they were)."""
return price(snapshot["inputs"], snapshot["params"])


PERIL_NAMES = {"RANSOMWARE": "ransomware", "BEC": "business email compromise", "FTF": "funds transfer
     fraud",
               "MISC_FIRST_PARTY": "other first-party incidents", "THIRD_PARTY": "third-party liability"}
    def
explain(r):
    c = r["classification"]     top = sorted(r["perils"], key=lambda p: p["expected_loss"],
reverse=True)[0]     zone = " (inside a turnover transition zone)" if c["frequency_zone"] or
c["severity_zone"] else ""     return (f"Classified as {c['sector']} with turnover USD {c['turnover_usd'] /
1e6:,.1f}M{zone}. "              f"Expected annual loss is ₹{r['expected_loss']:,.0f}, mostly from
{PERIL_NAMES[top['peril']]} "             f"(₹{top['expected_loss']:,.0f}). Dividing by the 50% target loss
ratio gives a technical premium of "
            f"₹{r['technical_premium']:,.0f}; with 18% GST the customer pays ₹{r['total_premium']:,.0f}.")
    def with_params(prm, path,
value):
    """Copy of the parameters with one value replaced (for sensitivity analysis). path like
     'controls.beta'."""     out = copy.deepcopy(prm)     node = out
keys = path.split(".")     for k in keys[:-1]:         node = node[k]
if isinstance(node[keys[-1]], dict) and "value" in node[keys[-1]]:
        node[keys[-1]]["value"] = value
else:
        node[keys[-1]] = value     return out
```

kyh-cyber-insurance/app/coverage.py 160 lines

The warranty state machine and coverage consequences (Section 3.3.6).

```
"""Coverage engine — post-bind warranty monitoring.
State machine per (bound policy, warranty control):

    MET --control fails--> IN_GRACE --grace expires--> BREACHED
      ^                       |       +---- REMEDIATED <------+  (control
passes again during grace or breach)
```

```
Post-bind observations NEVER re-price the policy; they only drive this state
machine and its configured coverage consequences.

Prototype coverage-condition evaluation; final claim determination remains subject
to policy wording and authorized claims handling.
"""
from datetime import datetime, timedelta, timezone
  from . import audit,
observations from .database
import new_id, now_iso

STATES = ["MET", "IN_GRACE", "BREACHED", "REMEDIATED", "BREACH_CONFIRMED"]
COVERAGE_EVAL_LABEL = ("Prototype coverage-condition evaluation; final claim determination "
                       "remains subject to policy wording and authorized claims handling.")
    def
_parse(ts):
    return datetime.fromisoformat(ts)
    def initialize_warranties(conn, policy_id,
actor="system"):
    ts = now_iso()     for ctrl in conn.execute("SELECT * FROM controls WHERE
is_warranty=1").fetchall():
        wid = new_id("WTY")
conn.execute("""INSERT OR IGNORE INTO warranties
           (warranty_id,policy_id,control_id,state,last_observed_at,grace_period_days,
coverage_effect,updated_at) VALUES (?,?,?,?,?,?,?,?)""",             (wid, policy_id,
ctrl["control_id"], "MET", None,              ctrl["grace_period_days"],
ctrl["coverage_effect"], ts))         _event(conn, wid, policy_id, ctrl["control_id"],
None, "MET", ts,                note="Warranty initialized at bind")
    # Evaluate immediately against current posture so a control that is already
    # failing at bind is tracked from day one.     pol =
conn.execute("SELECT company_id FROM policies WHERE policy_id=?",
                       (policy_id,)).fetchone()
evaluate_policy(conn, policy_id, pol["company_id"], actor=actor)
    def _event(conn, warranty_id, policy_id, control_id, frm, to, at, obs_id=None,
note=None):
    conn.execute("INSERT INTO warranty_events VALUES (?,?,?,?,?,?,?,?,?)",
(new_id("WEV"), warranty_id, policy_id, control_id, frm, to, at,
obs_id, note))
    def _transition(conn, w, to_state, actor, obs_id=None, note=None,
extra_sql=None,                 extra_args=()):
    frm = w["state"]     if frm == to_state:         return False     at = now_iso()     sql =
"UPDATE warranties SET state=?, updated_at=?" + (", " + extra_sql if extra_sql else "")
conn.execute(sql + " WHERE warranty_id=?", (to_state, at, *extra_args, w["warranty_id"]))
_event(conn, w["warranty_id"], w["policy_id"], w["control_id"], frm, to_state, at,
obs_id, note)
    ev = {"IN_GRACE": "WARRANTY_BREACH_DETECTED", "BREACHED": "WARRANTY_BREACH_CONFIRMED",
          "REMEDIATED": "WARRANTY_REMEDIATED", "MET": "WARRANTY_MET"}.get(to_state,
                                                                          "WARRANTY_STATE_CHANGED")
audit.log(conn, actor=actor, event_type=ev, entity_type="WARRANTY",
entity_id=w["warranty_id"],
             old_value={"state": frm}, new_value={"state": to_state, "note": note,
"control_id": w["control_id"],                                          "policy_id":
w["policy_id"]})     return True
    def evaluate_policy(conn, policy_id, company_id,
actor="system"):
    """Re-evaluate every warranty against current resolved posture, and expire     grace periods.
Deterministic given the observation set and the clock."""     posture =
observations.resolve_posture(conn, company_id)     now = datetime.now(timezone.utc)     results =
[]     for w in conn.execute("SELECT * FROM warranties WHERE policy_id=?",
(policy_id,)).fetchall():
        p = posture.get(w["control_id"], {})
status = p.get("status")         obs_id =
p.get("observation_id")         observed_at
= p.get("observed_at")         if
observed_at:
           conn.execute("UPDATE warranties SET last_observed_at=? WHERE warranty_id=?",
(observed_at, w["warranty_id"]))         if status == "FAIL":             if w["state"] in ("MET",
"REMEDIATED"):                 grace_days = w["grace_period_days"]                 detected =
```

```
now.isoformat(timespec="microseconds")                expires = (now +
timedelta(days=grace_days)).isoformat(timespec="microseconds")
               _transition(conn, w, "IN_GRACE", actor, obs_id,
note=f"Failing observation; grace period {grace_days} days",
extra_sql="breach_detected_at=?, grace_expires_at=?",
extra_args=(detected, expires))            elif w["state"] == "IN_GRACE" and
w["grace_expires_at"] \                    and now >= _parse(w["grace_expires_at"]):
_transition(conn, w, "BREACHED", actor, obs_id,                          note="Grace
period expired while control still failing")         elif status in ("PASS",
"PARTIAL"):             if w["state"] in ("IN_GRACE", "BREACHED"):
_transition(conn, w, "REMEDIATED", actor, obs_id,
note="Passing observation received",
extra_sql="breach_detected_at=NULL, grace_expires_at=NULL",
extra_args=())                 w2 = conn.execute("SELECT * FROM warranties WHERE
warranty_id=?",
                                  (w["warranty_id"],)).fetchone()
               _transition(conn, w2, "MET", actor, obs_id, note="Warranty restored")
# UNKNOWN: no change — absence of a report is not a breach in this prototype.
        results.append(warranty_view(conn, w["warranty_id"]))
return results
    def expire_grace_periods(conn, actor="scheduler"):     """Lazy expiry pass (also
callable on a schedule)."""     now = datetime.now(timezone.utc)     for w in
conn.execute("SELECT * FROM warranties WHERE state='IN_GRACE'").fetchall():         if
w["grace_expires_at"] and now >= _parse(w["grace_expires_at"]):
           _transition(conn, w, "BREACHED", actor, note="Grace period expired")
    def warranty_view(conn,
warranty_id):
    w = conn.execute("SELECT w.*, c.name AS control_name FROM warranties w "
"JOIN controls c ON c.control_id=w.control_id "                    "WHERE
w.warranty_id=?", (warranty_id,)).fetchone()     d = dict(w)
d["coverage_consequence"] = coverage_consequence(d)     return d
    def
coverage_consequence(w):
    """Illustrative policy-condition evaluation for the current state."""
if w["state"] == "MET" or w["state"] == "REMEDIATED":
        return dict(applies=False, text="None — warranty met.", label=COVERAGE_EVAL_LABEL)
if w["state"] == "IN_GRACE":         return dict(applies=False,
                   text=(f"Warranty breached on {w['breach_detected_at']}. Grace period "
f"expires on {w['grace_expires_at']}. If the warranty remains "
f"breached when a covered incident occurs after grace expiry, the "
f"configured coverage consequence may apply: "
f"{w['coverage_effect'] or 'per policy wording'}."),
label=COVERAGE_EVAL_LABEL)     return dict(applies=True,
               text=(f"Warranty breach confirmed. Configured coverage consequence: "
f"{w['coverage_effect'] or 'per policy wording'}."),                label=COVERAGE_EVAL_LABEL)
    def policy_warranties(conn,
policy_id):
    expire_grace_periods(conn)
    rows = conn.execute("SELECT warranty_id FROM warranties WHERE policy_id=?",
                        (policy_id,)).fetchall()     return
[warranty_view(conn, r["warranty_id"]) for r in rows]   def
policy_warranty_events(conn, policy_id, limit=200):
    return [dict(r) for r in conn.execute(
        "SELECT * FROM warranty_events WHERE policy_id=? ORDER BY at DESC LIMIT ?",
        (policy_id, min(int(limit), 1000))).fetchall()]
```

kyh-cyber-insurance/app/fleet.py

193 lines

Verification of signed collector roll-ups and AGENT_FLEET evidence (Section 3.3.9).

```
"""Link to the KYH live-monitoring collector (Part 2).

The collector receives signed reports from the agents on every insured machine and
rolls them up into one result per control for the whole company. It sends that roll-up
here, signed. This module:

1. authenticates the collector (HMAC over the exact body, increasing sequence number,      timestamp
   within 5 minutes, company must be linked to that collector);
```

```
2. stores the roll-up unchanged (append-only);
3. records each control result as an AGENT_FLEET observation with      TECHNICALLY_OBSERVED strength
   (Engine A), skipping exact repeats;
4. lets the existing coverage engine evaluate warranties.

It never prices. Before bind the observations feed the next quote like any other evidence;
after bind they reach only the warranty state machine.
""" import hashlib import hmac import json import
secrets from datetime import datetime, timedelta,
timezone
  from . import audit, coverage,
observations from .database import new_id,
now_iso

SCHEMA_ID = "kyh-fleet-rollup/1"
MAX_SKEW = timedelta(minutes=5)
REFRESH_HOURS = 6          # re-record an unchanged result at most this often
MAX_CONTROLS, MAX_HOSTS = 64, 5000
VALID = {("PASS", "true"), ("PASS", "partial"), ("FAIL", "false")}
    class
FleetError(Exception):
    def __init__(self, code, message, status=400):
        super().__init__(message)
        self.code, self.message, self.status = code, message, status
    def
_parse(ts):
    dt = datetime.fromisoformat(str(ts).replace("Z", "+00:00"))
return dt if dt.tzinfo else dt.replace(tzinfo=timezone.utc)


# ------------------------------------------------------------------ administration def
register_collector(conn, name, dashboard_url, actor):
    cid = new_id("COL")     secret
= secrets.token_hex(32)
    conn.execute("INSERT INTO collectors (collector_id,name,secret,dashboard_url,created_at) "
"VALUES (?,?,?,?,?)", (cid, name, secret, dashboard_url, now_iso()))     audit.log(conn,
actor, "COLLECTOR_REGISTERED", "COLLECTOR", cid,               new_value=dict(name=name,
dashboard_url=dashboard_url))     return dict(collector_id=cid, secret=secret,
               note="Store the secret in the collector (kyh_collector link-platform). It is shown once.")
    def link_company(conn, collector_id, company_id, collector_company_id,
actor):
    if not conn.execute("SELECT 1 FROM collectors WHERE collector_id=?", (collector_id,)).fetchone():
        raise FleetError("UNKNOWN_COLLECTOR", "No such collector", 404)     if not
conn.execute("SELECT 1 FROM companies WHERE company_id=?", (company_id,)).fetchone():
        raise FleetError("UNKNOWN_COMPANY", "No such company", 404)
    conn.execute("INSERT OR REPLACE INTO collector_links VALUES (?,?,?,?)",
                 (collector_id, company_id, str(collector_company_id)[:80], now_iso()))
audit.log(conn, actor, "COLLECTOR_LINKED", "COMPANY", company_id,
new_value=dict(collector_id=collector_id, collector_company_id=collector_company_id))     return
dict(ok=True)
    def
list_collectors(conn):
    out = []     for c in conn.execute("SELECT
collector_id,name,dashboard_url,last_seq,last_seen,created_at "
                          "FROM collectors ORDER BY created_at").fetchall():
d = dict(c)         d["links"] = [dict(r) for r in conn.execute(
            "SELECT l.company_id, l.collector_company_id, co.name FROM collector_links l "
            "JOIN companies co ON co.company_id=l.company_id WHERE l.collector_id=?",
            (c["collector_id"],)).fetchall()]
out.append(d)     return out


# ------------------------------------------------------------------ authentication def
authenticate(conn, collector_id, signature, raw):
    row = conn.execute("SELECT * FROM collectors WHERE collector_id=?", (collector_id or "",)).fetchone()
if row is None:
        raise FleetError("UNKNOWN_COLLECTOR", "Collector not registered", 401)     expected =
"v1=" + hmac.new(bytes.fromhex(row["secret"]), raw, hashlib.sha256).hexdigest()     if not
signature or not hmac.compare_digest(expected, signature):         audit.log(conn,
```

```
f"collector:{collector_id}", "COLLECTOR_ROLLUP_REJECTED", "COLLECTOR",
collector_id, new_value={"reason": "bad signature"})         raise FleetError("BAD_SIGNATURE",
"Signature does not match body", 401)     return dict(row)


# ------------------------------------------------------------------ ingestion def
ingest_rollup(conn, collector, raw, now=None):
    now = now or datetime.now(timezone.utc)
try:
        body = json.loads(raw.decode("utf-8"))
except (ValueError, UnicodeDecodeError):
        raise FleetError("INVALID_JSON", "Body must be JSON")     if
not isinstance(body, dict) or body.get("schema") != SCHEMA_ID:
raise FleetError("SCHEMA", f"schema must be {SCHEMA_ID}")     if
body.get("collector_id") != collector["collector_id"]:
        raise FleetError("COLLECTOR_MISMATCH", "collector_id in body differs from header", 403)
seq = body.get("seq")     if not isinstance(seq, int) or seq <= collector["last_seq"]:
        audit.log(conn, f"collector:{collector['collector_id']}", "COLLECTOR_ROLLUP_REJECTED",
                  "COLLECTOR", collector["collector_id"],
                  new_value={"reason": f"replayed sequence {seq} <= {collector['last_seq']}"})
raise FleetError("REPLAY", f"Sequence {seq} already received", 409)     try:
        sent = _parse(body["sent_at"])         evaluated =
_parse(body.get("evaluated_at") or body["sent_at"])     except
(KeyError, ValueError):
        raise FleetError("BAD_TIMESTAMP", "sent_at missing or invalid")
if abs(now - sent) > MAX_SKEW:
        raise FleetError("STALE_OR_FUTURE", "sent_at is more than 5 minutes from platform time")

    company_id = (body.get("company") or {}).get("platform_company_id")
    link = conn.execute("SELECT * FROM collector_links WHERE collector_id=? AND company_id=?",
                        (collector["collector_id"], company_id)).fetchone()
if link is None:
        raise FleetError("NOT_LINKED", "This collector is not linked to that company", 403)

    controls = body.get("controls") or []     hosts = (body.get("fleet") or {}).get("hosts") or
[]     if not isinstance(controls, list) or len(controls) > MAX_CONTROLS or len(hosts) >
MAX_HOSTS:
        raise FleetError("TOO_MANY_ITEMS", "controls/hosts list too long")
for c in controls:
        if not isinstance(c, dict) or (c.get("status"), c.get("value")) not in VALID:
            raise FleetError("INVALID_CONTROL_VALUE", "each control needs status PASS/FAIL and value
     true/partial/false")
        if not conn.execute("SELECT 1 FROM controls WHERE control_id=?",
     (c.get("platform_control_id"),)).fetchone():
            raise FleetError("UNKNOWN_CONTROL", f"Unknown control {c.get('platform_control_id')}")
      rollup_id = new_id("ROL")     conn.execute("INSERT INTO
collector_rollups VALUES (?,?,?,?,?,?)",
                 (rollup_id, collector["collector_id"], company_id, seq, now_iso(), raw.decode("utf-8")))
conn.execute("UPDATE collectors SET last_seq=?, last_seen=? WHERE collector_id=?",                  (seq,
now_iso(), collector["collector_id"]))
      recorded, unchanged = [],
[]     for c in controls:
        ctrl = c["platform_control_id"]         counts = f"{c.get('passing',
'?')}/{c.get('observed', '?')} hosts pass"         prev = conn.execute("""SELECT value,
observed_at, source_locator FROM observations                                WHERE company_id=?
AND control_id=? AND source_type='AGENT_FLEET'
                               ORDER BY observed_at DESC, rowid DESC LIMIT 1""", (company_id,
     ctrl)).fetchone()
        if prev and prev["value"] == c["value"] and (prev["source_locator"] or "").endswith(counts) and \
evaluated - _parse(prev["observed_at"]) < timedelta(hours=REFRESH_HOURS):
            unchanged.append(ctrl)             continue         failing = ", ".join(h.get("host", "?")
for h in (c.get("failing_hosts") or [])[:10])         sources = ", ".join(s.get("control", "?") for s
in c.get("sources") or [])         observations.record_observation(             conn,
company_id=company_id, control_id=ctrl, value=c["value"], source_type="AGENT_FLEET",
source_id=rollup_id,             source_locator=f"{collector['name']} roll-up #{seq} · {sources} ·
{counts}",             evidence_strength="TECHNICALLY_OBSERVED",
observed_at=evaluated.isoformat(timespec="microseconds"),
            notes=(str(c.get("reason", ""))[:300] + (f" Failing: {failing}" if failing else "")),
actor=f"collector:{collector['collector_id']}")         recorded.append(ctrl)
```

```
      snapshot = dict(collector_id=collector["collector_id"],
collector_name=collector["name"],
dashboard_url=collector["dashboard_url"], seq=seq, rollup_id=rollup_id,
evaluated_at=body.get("evaluated_at"), received_at=now_iso(),
collector_company_id=link["collector_company_id"],
company=body.get("company"), fleet=body.get("fleet") or {},
controls=controls, coverage=body.get("coverage"),
unmapped=body.get("unmapped") or [])
    conn.execute("INSERT OR REPLACE INTO fleet_snapshots VALUES (?,?,?,?,?)",
                 (company_id, collector["collector_id"], rollup_id, now_iso(), json.dumps(snapshot)))
audit.log(conn, f"collector:{collector['collector_id']}", "COLLECTOR_ROLLUP_RECEIVED",
"COMPANY", company_id, new_value=dict(rollup_id=rollup_id, seq=seq, recorded=recorded,
unchanged=unchanged))

    # Post-bind: warranty evaluation ONLY. Pre-bind: observations wait for the next quote.
pol = conn.execute("SELECT * FROM policies WHERE company_id=?", (company_id,)).fetchone()
warranties = []     if pol and pol["status"] == "BOUND":
        warranties = coverage.evaluate_policy(conn, pol["policy_id"], company_id,
actor=f"collector:{collector['collector_id']}")     return dict(rollup_id=rollup_id, seq=seq,
observations_recorded=recorded, unchanged=unchanged,                repricing_performed=False,
               policy=dict(policy_id=pol["policy_id"], status=pol["status"], bound_at=pol["bound_at"],
bound_premium=pol["bound_premium"]) if pol else None,                warranties=[{k: w[k] for k in
("control_id", "control_name", "state", "breach_detected_at",
                                             "grace_expires_at", "grace_period_days")}
                            | {"consequence": w["coverage_consequence"]["text"]} for w in warranties])
    def latest_snapshot(conn,
company_id):
    row = conn.execute("SELECT * FROM fleet_snapshots WHERE company_id=?", (company_id,)).fetchone()
return json.loads(row["snapshot_json"]) if row else None
```

kyh-cyber-insurance/app/security.py 89 lines

Role permissions, token checks, rate limiting and input validation (Section 3.3.10).

```
"""Authentication, role authorization and basic rate limiting."""
import hmac import threading import time   from .database import
get_config
    class AuthError(Exception):     def
__init__(self, code, message, status=401):
        super().__init__(message)         self.code, self.message,
self.status = code, message, status


_rate_lock = threading.Lock()
_rate_buckets = {}  # token -> [window_start, count]
    def rate_limit(conn, token):     limit =
get_config(conn, "rate_limit_per_minute") or 240     now =
int(time.time() // 60)     with _rate_lock:
        window, count = _rate_buckets.get(token, (now, 0))
if window != now:
            window, count = now, 0
count += 1
        _rate_buckets[token] = (window, count)
if count > limit:
        raise AuthError("RATE_LIMITED", "Too many requests; try again shortly.", 429)
    def authenticate_user(conn,
token):     if not token:
        raise AuthError("MISSING_TOKEN", "Provide X-Auth-Token header.")
for row in conn.execute("SELECT * FROM users").fetchall():         if
hmac.compare_digest(row["token"], token):
            rate_limit(conn, token)             return
dict(row)     raise AuthError("INVALID_TOKEN",
"Unknown token.")
    def authenticate_agent(conn,
token):
    if not token:
        raise AuthError("MISSING_TOKEN", "Agent must send Authorization: Bearer <token>.")
for row in conn.execute("SELECT * FROM agents").fetchall():
        if hmac.compare_digest(row["token"], token):
```

```
            rate_limit(conn, token)
return dict(row)
    raise AuthError("INVALID_AGENT_TOKEN", "Agent token not recognized.")


ROLE_PERMS = {
    "UNDERWRITER": {"read", "staff", "intake", "quote", "bind", "assess"},
    # The insured customer is read-only, and only for their own company (see api.Handler._own).
    "CUSTOMER": {"read"},
    "ADMIN": {"read", "staff", "intake", "quote", "bind", "assess", "admin"},
}
    def require(user,
perm):
    if perm not in ROLE_PERMS.get(user["role"], set()):
        raise AuthError("FORBIDDEN", f"Role {user['role']} cannot perform '{perm}'.", 403)
    def clean_str(value, field, max_len=200,
required=True):     if value is None or
str(value).strip() == "":
        if required:
            raise ValueError(f"{field} is required")
return None     s = str(value).strip()     if len(s)
> max_len:
        raise ValueError(f"{field} exceeds {max_len} characters")
return s
    def clean_number(value, field, minimum=None, maximum=None,
required=True):     if value is None:         if required:
raise ValueError(f"{field} is required")         return None     try:
        n = float(value)     except
(TypeError, ValueError):
        raise ValueError(f"{field} must be a number")
if minimum is not None and n < minimum:
        raise ValueError(f"{field} must be >= {minimum}")
if maximum is not None and n > maximum:
        raise ValueError(f"{field} must be <= {maximum}")
return n
```

E.2 Endpoint agent (Part 2)

kyh-agent/agent/kyh_agent/agent.py 286 lines

The agent loop: schedules checks, detects changes, builds signed envelopes, queues and sends them (Section 3.3.7).

```
"""The agent loop: schedule checks, detect changes, build signed envelopes, queue, send."""
  from __future__ import
annotations
  import hashlib import json
import os import platform as
_platform import random
import socket import time
from datetime import datetime, timedelta

from . import SCHEMA_VERSION, __version__ from .
import config as config_mod from .checks.base
import iso, utcnow from .policy import
canonical_json from .runner import Runner,
current_platform from .security import
build_hash, sign from .spool import Spool from
.transport import Transport, TransportError

MAX_BODY = 900_000  # stay under the collector's 1 MB cap
    def fingerprint(result: dict) ->
str:
    """What counts as a 'change'. Timestamps and durations are excluded on purpose."""
key = f"{result['status']}|{result.get('assurance')}|{result['summary']}"     return
hashlib.sha256(key.encode()).hexdigest()[:16]
```

```
    def os_label() ->
str:
    try:
        if current_platform() == "linux":
            with open("/etc/os-release", encoding="utf-8") as fh:
for line in fh:                    if
line.startswith("PRETTY_NAME="):
                        return line.split("=", 1)[1].strip().strip('"')         return
f"{_platform.system()} {_platform.release()} ({_platform.version()[:40]})"     except
OSError:
        return _platform.platform()


SCAN_REQUEST = "scan-now.json"
    def request_scan(state_dir: str, controls=None) ->
str:
    """Ask the running agent service to scan now (all checks, or only the named controls)."""
os.makedirs(state_dir, exist_ok=True)     path = os.path.join(state_dir, SCAN_REQUEST)
tmp = path + ".tmp"     with open(tmp, "w", encoding="utf-8") as fh:
        json.dump({"controls": [c.upper() for c in (controls or [])], "requested_at": iso(utcnow())}, fh)
os.replace(tmp, path)     return path
    class
Agent:
    def __init__(self, cfg, policy, state_dir: str, transport=None, clock=utcnow, registry=None,
simulate: str | None = None, platform: str | None = None, sleep=time.sleep, rng=None):
        self.cfg = cfg         self.policy = policy
self.state_dir = state_dir
os.makedirs(state_dir, exist_ok=True)
self.clock = clock         self.sleep = sleep
self.rng = rng or random.Random()
self.simulate = simulate         self.platform =
platform or current_platform()         if simulate:
            from .simulate import registry_for             registry = registry_for(simulate,
self.platform)         self.runner = Runner(policy, platform=self.platform,
registry=registry, clock=clock,                                salt=cfg.get("agent_id",
"unenrolled"))         self.spool = Spool(os.path.join(state_dir, "outbox.db"))
self.transport = transport         if self.transport is None and cfg.get("server_url"):
            self.transport = Transport(cfg["server_url"], cfg.get("ca_file"),
allow_insecure_http=bool(cfg.get("allow_insecure_http")))         self.build = build_hash()
self.hostname = cfg.get("hostname_override") or socket.gethostname()         self.state_path =
os.path.join(state_dir, "state.json")         self.state = self._load_state()
self.backoff = 0         self.started = time.monotonic()
      def close(self) ->
None:
        """Release the outbox database (Windows will not delete a state dir with it open)."""
self.spool.close()

    # ---------------------------------------------------------------------- state
def _load_state(self) -> dict:
        try:
            with open(self.state_path, encoding="utf-8") as fh:
                return json.load(fh)
except (OSError, ValueError):
            return {"next_due": {}, "fingerprints": {}, "next_full": None, "next_heartbeat": None,
"last_results": {}, "last_scan_at": None}
      def _save_state(self) -> None:         tmp =
self.state_path + ".tmp"         with open(tmp, "w",
encoding="utf-8") as fh:
            json.dump(self.state, fh, indent=1, sort_keys=True)
os.replace(tmp, self.state_path)
      def _jitter(self, seconds: float) ->
float: j =
float(self.policy.defaults.get("jitter_fraction
", 0.1))
        return seconds * (1 + self.rng.uniform(-j, j))

    # ---------------------------------------------------------------------- envelope
def envelope(self, kind: str, results: list, seq: int) -> dict:
        return {
```

```
            "schema_version": SCHEMA_VERSION,
            "kind": kind,                                  # full | delta | heartbeat
            "agent_id": self.cfg.get("agent_id"),
            "seq": seq,
            "created_at": iso(self.clock()),
            "agent": {"version": __version__, "build_hash": self.build, "platform": self.platform,
                      "hostname": self.hostname, "os": self.cfg.get("os_override") or os_label()},
            "policy": {"id": self.policy.id, "version": self.policy.version, "hash": self.policy.hash},
            "simulated": bool(self.simulate),
            "queue_depth": self.spool.depth(),
            "results": results,
        }       def enqueue(self, kind: str, results:
list) -> int:
        seq = self.spool.next_seq()         env = self.envelope(kind, results,
seq)         body = canonical_json(env)         if len(body) > MAX_BODY:  #
trim evidence rather than lose the verdicts             for r in
env["results"]:                 r["evidence"] = {"truncated": True}
body = canonical_json(env)         self.spool.put(seq, kind, env["created_at"],
body)         self._transparency(env)         return seq
      def _transparency(self, env: dict) ->
None:
        """Local, human-readable record of everything the agent sends (visible to the insured's staff)."""
line = {"seq": env["seq"], "kind": env["kind"], "created_at": env["created_at"],
"results": {r["check_id"]: r["status"] for r in env["results"]}}         path =
os.path.join(self.state_dir, "sent.jsonl")         if os.path.exists(path) and os.path.getsize(path) >
5_000_000:
            os.replace(path, path + ".1")
with open(path, "a", encoding="utf-8") as fh:
            fh.write(json.dumps(line) + "\n")

    # ------------------------------------------------------------------ sending
def flush(self) -> dict:
        stats = {"sent": 0, "rejected": 0, "remaining": 0, "error": None}
if not self.transport or not self.cfg.enrolled:
stats["remaining"] = self.spool.depth()             stats["error"] = "not
enrolled"             return stats         for seq, kind, created_at,
body in self.spool.pending(limit=200):
            headers = {"X-KYH-Agent-Id": self.cfg["agent_id"],
                       "X-KYH-Signature": sign(self.cfg["agent_secret"], body)}
try:
                status, resp = self.transport.post("/api/v1/report", body, headers)
except TransportError as exc:                 stats["error"] = str(exc)
                self.backoff = min(max(self.backoff * 2, 30), 900)                 break
if status in (200, 202) or (status == 409 and resp.get("code") == "REPLAY"):
                self.spool.ack(seq)       # 409 REPLAY = the collector already has it (lost ACK)
stats["sent"] += 1                 self.backoff = 0             elif status in (429,) or status
>= 500:
                stats["error"] = f"collector busy ({status})"
self.backoff = min(max(self.backoff * 2, 30), 900)                 break
            else:                           # 4xx: this report will never be accepted; park it
self.spool.dead_letter(seq, status, json.dumps(resp)[:500])
stats["rejected"] += 1         stats["remaining"] = self.spool.depth()
        return stats

    # ------------------------------------------------------------------ scanning     def
scan(self, check_ids=None, force_full: bool = False) -> dict:         now = self.clock()
due_full = force_full or self.state.get("next_full") is None or \             now >=
datetime.fromisoformat(self.state["next_full"].replace("Z", "+00:00"))         if
due_full:
            check_ids = [c["id"] for c in self.policy.checks]
elif check_ids is None:
            check_ids = [cid for cid in self.policy.by_id if self._is_due(cid, now)]
if not check_ids:
            return {"kind": None, "ran": 0, "sent": 0}         results = [r.to_dict() for r in
self.runner.run(check_ids)]         changed = [r for r in results if
self.state["fingerprints"].get(r["check_id"]) != fingerprint(r)]         for r in results:
            self.state["fingerprints"][r["check_id"]] = fingerprint(r)
self.state["last_results"][r["check_id"]] = r             nxt = now +
timedelta(seconds=self._jitter(self.policy.interval_seconds(r["check_id"])))
```

```
self.state["next_due"][r["check_id"]] = iso(nxt)         self.state["last_scan_at"] = iso(now)
kind = None         if due_full:
            kind = "full"             self.state["next_full"] =
iso(now +
timedelta(seconds=self._jitter(self.cfg.full_snapshot_seconds())))
            self.enqueue("full", results)
elif changed:
            kind = "delta"             self.enqueue("delta", changed)
self._save_state()         return {"kind": kind, "ran": len(results), "sent": len(results) if
kind == "full" else len(changed)}
      def _is_due(self, check_id: str, now: datetime) ->
bool:
        nd = self.state["next_due"].get(check_id)         return nd is None or now >=
datetime.fromisoformat(nd.replace("Z", "+00:00"))
      def heartbeat_if_due(self) ->
bool:
        now = self.clock()         nh = self.state.get("next_heartbeat")
if nh and now < datetime.fromisoformat(nh.replace("Z", "+00:00")):
            return False
self.enqueue("heartbeat", [])
        self.state["next_heartbeat"] = iso(now + timedelta(seconds=self.cfg.heartbeat_seconds()))
self._save_state()         return True
      def
take_scan_request(self):
        """Return the check ids asked for by `kyh-agent scan-now` ([] = everything), or None if no request.

        The request is a small file in the state directory, so the one running service stays the only
process that scans and sends; nothing else touches the outbox or the sequence numbers."""
path = os.path.join(self.state_dir, SCAN_REQUEST)         try:
            with open(path, encoding="utf-8") as fh:
                req = json.load(fh)
except FileNotFoundError:
            return None
except (OSError, ValueError):
            req = {}
try:
            os.remove(path)
except OSError:
            pass
        wanted = {str(x).upper() for x in (req.get("controls") or [])} | {str(x) for x in
    (req.get("checks") or [])}         return [c["id"] for c in self.policy.checks if c["control"].upper()
in wanted or c["id"] in wanted]   def tick(self) -> dict:
        """One iteration of the service loop."""
req = self.take_scan_request()         if req is
None:
            out = self.scan()
elif req:
            out = self.scan(check_ids=req)
else:
            out = self.scan(force_full=True)
self.heartbeat_if_due()         out["flush"]
= self.flush()         return out
      def run_forever(self, tick_seconds: int = 30, stop=None) ->
None:         while not (stop and stop()):             try:
              self.tick()             except Exception as exc:  # noqa: BLE001 - the service must
survive any single failure                 with open(os.path.join(self.state_dir, "agent-errors.log"),
"a", encoding="utf-8") as fh:
                    fh.write(f"{iso(self.clock())} {type(exc).__name__}: {exc}\n")
self.sleep(max(tick_seconds, self.backoff))


# ---------------------------------------------------------------------- enrollment def enroll(cfg,
server_url: str, token: str, ca_file: str | None, policy, allow_insecure_http=False,
transport=None, hostname: str | None = None) -> dict:
    t = transport or Transport(server_url, ca_file, allow_insecure_http=allow_insecure_http)
if hostname:
        cfg["hostname_override"] = hostname     body = json.dumps({"enrollment_token": token,
"hostname": hostname or socket.gethostname(),
                       "platform": current_platform(), "os": os_label(), "agent_version": __version__,
```

```
                    "build_hash": build_hash(), "policy_id": policy.id, "policy_hash":
     policy.hash}).encode()
    status, resp = t.post("/api/v1/enroll", body)
if status != 201:
        raise RuntimeError(f"Enrollment refused ({status}): {resp.get('error', resp)}")
cfg.update({"server_url": server_url, "ca_file": ca_file, "agent_id": resp["agent_id"],
"agent_secret": resp["agent_secret"], "company_id": resp["company_id"],
                "company_name": resp.get("company_name"), "enrolled_at": iso(utcnow()),
                "allow_insecure_http": bool(allow_insecure_http)})
config_mod.save(cfg)     skew = None     if
resp.get("server_time"):
        server_now = datetime.fromisoformat(resp["server_time"].replace("Z", "+00:00"))
skew = round(abs((utcnow() - server_now).total_seconds()), 1)
    return {"agent_id": resp["agent_id"], "company": resp.get("company_name"), "clock_skew_seconds": skew}
```

kyh-agent/agent/kyh_agent/security.py 43 lines

HMAC-SHA256 report signing, canonical JSON and the build hash.

```
"""Report signing and agent self-integrity."""

from __future__ import annotations
  import
hashlib import
hmac import os

SIG_PREFIX = "v1="
    def sign(secret_hex: str, body: bytes) ->
str:
    """HMAC-SHA256 over the exact bytes sent. The collector recomputes it with its copy of the secret."""
return SIG_PREFIX + hmac.new(bytes.fromhex(secret_hex), body, hashlib.sha256).hexdigest()
    def verify(secret_hex: str, body: bytes, header: str) ->
bool:
    if not header or not header.startswith(SIG_PREFIX):
        return False
    return hmac.compare_digest(sign(secret_hex, body), header)
    def build_hash(package_dir: str | None = None) ->
str:
    """SHA-256 over every .py file and the policy in the agent package, in sorted path order.

    The collector keeps a list of approved release hashes. A modified agent produces a
different hash, so its reports are stored but not trusted as TECHNICALLY_OBSERVED.
    """     package_dir = package_dir or
os.path.dirname(os.path.abspath(__file__))     h = hashlib.sha256()
files = []     for root, dirs, names in os.walk(package_dir):
        dirs[:] = sorted(d for d in dirs if d != "__pycache__")         for n in names:
if n.endswith(".py") or n == "policy.json":                 files.append(os.path.join(root, n))
for path in sorted(files, key=lambda p: os.path.relpath(p, package_dir).replace(os.sep, "/")):
        rel = os.path.relpath(path, package_dir).replace(os.sep, "/")
        h.update(rel.encode() + b"\0")
with open(path, "rb") as fh:
            h.update(fh.read().replace(b"\r\n", b"\n"))
        h.update(b"\0")                 return    h.hexdigest()
```

kyh-agent/agent/kyh_agent/checks/verdicts.py 239 lines

Decision rules: the status and assurance level of each check.

```
"""Verdicts: turn observed facts + policy thresholds into a PASS/FAIL outcome.

Each function is pure (facts, params, now) -> Outcome, so the decision rules can be
read, tested and cited independently of how the facts were collected.
"""   from __future__ import
annotations
  from datetime import date,
datetime
```

```
from .base import (ABSENT, FAIL, NOT_APPLICABLE, NOT_SUPPORTED, OPERATING, PASS, PRESENT,
RUNNING, Outcome, iso) from .parsers import host_matches, is_loopback
    def _days(now: datetime, then: datetime |
None):
    return None if then is None else round((now - then).total_seconds() / 86400, 1)


# ---------------------------------------------------------------- PATCH (warranty) def
patching(facts: dict, p: dict, now: datetime, method: str) -> Outcome:
    pending = facts.get("pending_security")     last = facts.get("last_update")
index_age = facts.get("index_age_days")     since = _days(now, last)     ev =
{"pending_security": pending, "pending_total": facts.get("pending_total"),
          "days_since_last_update": since, "package_index_age_days": index_age,
          "security_packages": facts.get("security_packages", []),
"max_patch_age_days": p["max_patch_age_days"]}     if index_age is
not None and index_age > p["max_index_age_days"]:
        return Outcome(FAIL, f"Package index is {index_age} days old; the pending-update count cannot be
     trusted",
                       PRESENT, ev, method)
if pending == 0:
        return Outcome(PASS, "No security updates pending", OPERATING, ev, method)     what = "Pending
updates could not be listed" if pending is None else f"{pending} security update(s) pending"
    if since is not None and since <= p["max_patch_age_days"]:
        return Outcome(PASS, f"{what}, but the machine was patched {since} days ago "
f"(within the {p['max_patch_age_days']}-day cycle)",                         OPERATING if
pending is not None else RUNNING, ev, method)
    age = "never observed" if since is None else f"{since} days ago"
    return Outcome(FAIL, f"{what}; last patching {age} (limit {p['max_patch_age_days']} days)", PRESENT,
     ev, method)


# ---------------------------------------------------------------- EDR (warranty) def edr(found: dict,
defender: dict | None, p: dict, method: str, health: dict | None = None) -> Outcome:     """found =
{process: product} of known EDR/AV processes; defender = parsed Get-MpComputerStatus."""     ev =
{"edr_processes": found, "defender": defender, "health": health,           "max_signature_age_days":
p["max_signature_age_days"]}     third_party = {k: v for k, v in found.items() if not
v.startswith("Microsoft Defender")}     if health is not None and health.get("healthy") is False:
        return Outcome(FAIL, f"{health.get('product')} reports itself unhealthy", PRESENT, ev, method)
if health is not None and health.get("healthy") is True:
        return Outcome(PASS, f"{health.get('product')} running and reports healthy", OPERATING, ev, method)
if third_party:
        names = ", ".join(sorted(set(third_party.values())))         return Outcome(PASS, f"EDR
running: {names} (process observed; product health not queryable locally)",
                       RUNNING, ev, method)
if defender is not None:
        age = defender.get("signature_age_days")         mode = (defender.get("running_mode") or
"Normal").lower()         active = defender["service_enabled"] and defender["antivirus_enabled"] and
"passive" not in mode         if active and defender["realtime_enabled"] and age is not None and age <=
p["max_signature_age_days"]:
            return Outcome(PASS, f"Microsoft Defender active, real-time protection on, signatures {age}
     day(s) old",
                           OPERATING, ev, method)
problems = []         if not active:
            problems.append(f"not the active engine (mode: {defender.get('running_mode')})")
if not defender["realtime_enabled"]:
            problems.append("real-time protection off")
if age is None or age > p["max_signature_age_days"]:
            problems.append(f"signatures {age} days old (limit {p['max_signature_age_days']})")
return Outcome(FAIL, "Microsoft Defender present but " + "; ".join(problems), PRESENT, ev, method)     if
found:  # Defender for Endpoint on Linux/macOS, no health query available         names = ",
".join(sorted(set(found.values())))         return Outcome(PASS, f"EDR running: {names} (process
observed)", RUNNING, ev, method)     return Outcome(FAIL, "No known EDR/anti-malware process is running",
ABSENT, ev, method)


# ---------------------------------------------------------------- BACKUP (warranty) def
backup(last_success: datetime | None, source: str | None, agents: list, p: dict, now: datetime,
method: str) -> Outcome:     age = _days(now, last_success)
    ev = {"last_success": iso(last_success), "age_days": age, "source": source, "backup_software": agents,
```

```
          "max_backup_age_days": p["max_backup_age_days"], "restore_test_verified": False}     note = "
Restore testing cannot be verified from the endpoint; supply a dated restore-test record."     if
last_success is not None:
        if age <= p["max_backup_age_days"]:
            return Outcome(PASS, f"Last successful backup {age} days ago ({source}).{note}", OPERATING, ev,
    method)
        return Outcome(FAIL, f"Last successful backup {age} days ago (limit {p['max_backup_age_days']})
    ({source})",
                       PRESENT, ev, method)
if agents:
        return Outcome(NOT_SUPPORTED, f"Backup software detected ({', '.join(agents)}) but its last-success
    time is "
                                    f"not readable locally. Provide the backup job report, or configure
    the "
                                    f"job to write the KYH backup marker.", PRESENT, ev, method)
return Outcome(NOT_SUPPORTED, "No local backup evidence found. Backups may run centrally; provide the
backup "
                                "job report as documentary evidence.", ABSENT, ev, method)


# ---------------------------------------------------------------- MFA (warranty)
def mfa(method: str = "not observable from an endpoint") -> Outcome:
    return Outcome(NOT_SUPPORTED, "MFA is enforced by the identity provider, not the endpoint. Evidence
     must come "
                                "from an IdP export or connector (e.g. Entra ID / Google Workspace signin
     policy).",
                  None, {"guidance": "Upload the IdP conditional-access / 2-step verification policy
     export."}, method)


# ---------------------------------------------------------------- disk encryption def
disk_encryption(encrypted: bool, detail: dict, method: str, suspended: bool = False) -> Outcome:
if encrypted and not suspended:
        return Outcome(PASS, "System drive is encrypted", OPERATING, detail, method)
if encrypted and suspended:
        return Outcome(FAIL, "System drive is encrypted but protection is suspended", PRESENT, detail,
     method)
    return Outcome(FAIL, "System drive is not encrypted", ABSENT, detail, method)


# ---------------------------------------------------------------- firewall
def firewall(enabled: bool, detail: dict, method: str) -> Outcome:     if
enabled:
        return Outcome(PASS, "Host firewall is enabled", OPERATING, detail, method)     return
Outcome(FAIL, "Host firewall is disabled or has no filtering rules", ABSENT, detail, method)


# ---------------------------------------------------------------- exposed services def
exposed_services(listeners: list, p: dict, method: str) -> Outcome:
    critical = {int(k): v for k, v in p["critical_ports"].items()}     review = {int(k): v for k, v
in p["review_ports"].items()}     allow = set(int(x) for x in p.get("allow_ports", []))     exposed =
sorted({port for ip, port in listeners if not is_loopback(ip)})     crit = [{"port": x, "service":
critical[x]} for x in exposed if x in critical and x not in allow]     rev = [{"port": x, "service":
review[x]} for x in exposed if x in review and x not in allow]     ev =
{"listening_ports_non_loopback": exposed, "critical": crit, "review": rev,
          "note": "Local view only. Whether a port is reachable from the internet needs an outside-in
     scan."}
    if crit:
        names = ", ".join(f"{c['service']} ({c['port']})" for c in crit)         return Outcome(FAIL,
f"High-risk service listening on a network interface: {names}", None, ev, method)
    if rev:
        names = ", ".join(f"{c['service']} ({c['port']})" for c in rev)         return
Outcome(PASS, f"No high-risk listeners. Review: {names}", None, ev, method)     return
Outcome(PASS, "No high-risk services listening on network interfaces", None, ev, method)


# ---------------------------------------------------------------- remote access def
ssh_hardening(cfg: dict, p: dict, method: str) -> Outcome:
```

```
    root = cfg.get("permitrootlogin", "prohibit-password")   # OpenSSH >= 7.0 default     pw =
cfg.get("passwordauthentication", "yes")             # OpenSSH default     empty =
cfg.get("permitemptypasswords", "no")     ev = {"permitrootlogin": root,
"passwordauthentication": pw, "permitemptypasswords": empty}     problems = []     if root ==
"yes":
        problems.append("root can log in with a password (PermitRootLogin yes)")
if empty == "yes":
        problems.append("empty passwords allowed")
if pw == "yes" and p.get("require_key_auth", True):
        problems.append("password authentication enabled (keys required by policy)")
if problems:
        return Outcome(FAIL, "SSH server: " + "; ".join(problems), None, ev, method)
    return Outcome(PASS, "SSH server hardened (no root password login, key-based auth)", None, ev, method)
    def rdp_hardening(rdp_enabled: bool, nla: bool, method: str) ->
Outcome:
    ev = {"rdp_enabled": rdp_enabled, "nla_required": nla}
if not rdp_enabled:
        return Outcome(NOT_APPLICABLE, "Remote Desktop is disabled", None, ev, method)
    if nla:
        return Outcome(PASS, "Remote Desktop enabled with Network Level Authentication", None, ev, method)
return Outcome(FAIL, "Remote Desktop enabled WITHOUT Network Level Authentication", None, ev, method)


# ---------------------------------------------------------------- admin accounts
def admin_accounts(count: int, hashed: list, extra_uid0: int, guest_enabled: bool, p: dict, method: str) ->
     Outcome:
    ev = {"admin_count": count, "admin_ids_hashed": hashed, "extra_uid0_accounts": extra_uid0,
          "guest_enabled": guest_enabled, "max_local_admins": p["max_local_admins"],
          "privacy": "Account names are salted-hashed on the endpoint; only counts leave the machine in
     clear."}
    problems = []
if extra_uid0:
        problems.append(f"{extra_uid0} extra account(s) with UID 0")
if guest_enabled:
        problems.append("Guest account enabled")
if count > p["max_local_admins"]:
        problems.append(f"{count} local administrators (limit {p['max_local_admins']})")
if problems:
        return Outcome(FAIL, "; ".join(problems), None, ev, method)     return Outcome(PASS, f"{count}
local administrator(s); no guest or hidden root accounts", None, ev, method)


# ---------------------------------------------------------------- OS support
def os_support(label: str, eol: str | None, p: dict, now: datetime, method: str, known: bool = True) ->
     Outcome:
    ev = {"os": label, "end_of_support": eol, "table_as_of": p.get("as_of"),
          "note": "Lifecycle dates are policy data; verify against vendor lifecycle pages."}
if not known:
        return Outcome(NOT_SUPPORTED, f"No lifecycle data for {label}; add it to the policy table", None,
     ev, method)
    if eol is None:
        return Outcome(PASS, f"{label} is supported", None, ev, method)
days_left = (date.fromisoformat(eol) - now.date()).days
ev["days_to_end_of_support"] = days_left     if days_left < 0:
        return Outcome(FAIL, f"{label} reached end of vendor support on {eol}", None, ev, method)
if days_left <= p.get("warn_days", 90):
        return Outcome(PASS, f"{label} supported, but support ends on {eol} ({days_left} days)", None, ev,
     method)
    return Outcome(PASS, f"{label} supported until {eol}", None, ev, method)


# ---------------------------------------------------------------- logging (CERT-In) def
log_retention(active: bool, forwarder: str | None, retained_days: float | None, p: dict, method: str) -
     > Outcome:
    need = p["required_retention_days"]     ev = {"logging_active": active, "forwarder":
forwarder, "local_retained_days": retained_days,
          "required_retention_days": need,
          "basis": "CERT-In Directions (28 Apr 2022): enable ICT logs and keep them 180 days within India"}
if not active:
```

```
        return Outcome(FAIL, "System event logging is not active", ABSENT, ev, method)
if forwarder:
        return Outcome(PASS, f"Logs forwarded off-host by {forwarder}; the {need}-day retention must be
     shown "
                           f"at the central log store", RUNNING, ev, method)
if retained_days is not None and retained_days >= need:
        return Outcome(PASS, f"Local logs cover {retained_days} days (>= {need})", OPERATING, ev, method)
have = "unknown" if retained_days is None else f"{retained_days} days"
    return Outcome(FAIL, f"Local logs cover {have} (< {need}) and no log forwarder was detected", PRESENT,
     ev, method)


# ---------------------------------------------------------------- time sync (CERT-In)
def time_sync(synchronized: bool | None, sources: list, p: dict, method: str, hierarchy: bool = False) ->
     Outcome:
    approved = p["approved_sources"]
    ok = [s for s in sources if host_matches(s, approved)]     ev = {"synchronized":
synchronized, "sources": sources, "approved_sources": approved,
          "domain_hierarchy": hierarchy,
          "basis": "CERT-In Directions (28 Apr 2022): sync ICT clocks to NIC/NPL NTP or sources traceable
     to them"}
    if synchronized is False:
        return Outcome(FAIL, "System clock is not synchronised to any time source", ABSENT, ev, method)
if hierarchy:
        return Outcome(PASS, "Clock follows the Windows domain hierarchy; confirm the domain's PDC emulator
     syncs "
                           "to NIC/NPL", RUNNING, ev, method)
if ok:
        level = OPERATING if synchronized else RUNNING         return Outcome(PASS, f"Clock
synchronised to approved source {ok[0]}", level, ev, method)     if not sources:
        return Outcome(FAIL, "No NTP source configured", ABSENT, ev, method)
if p.get("require_nic_npl_source", True):
        return Outcome(FAIL, f"Clock synchronised to {', '.join(sources[:3])}, not a NIC/NPL server. Switch
     to "
                           f"samay1.nic.in / time.nplindia.org or document traceability.", PRESENT, ev,
     method)
    return Outcome(PASS, f"Clock synchronised to {', '.join(sources[:3])}", RUNNING, ev, method)
```

kyh-agent/agent/kyh_agent/checks/windows.py 230 lines

The Windows checks (read-only PowerShell and registry queries).

```
"""Windows collectors. PowerShell snippets are read-only (Get-* cmdlets and COM searches only); registry
reads use the standard-library winreg module so no process is spawned."""
  from __future__ import
annotations
  import json   from . import parsers as P from . import verdicts as V from .base import
(ERROR, NOT_SUPPORTED, NeedsPrivilege, Outcome, looks_like_access_denied,
read_text, run_cmd, run_powershell)

PS_PROCESSES = "@(Get-Process | Select-Object -ExpandProperty ProcessName)"
    def _reg(hive_path: str, name:
str):
    import winreg  # Windows-only stdlib module
root, _, sub = hive_path.partition("\\")     hive =
{"HKLM": winreg.HKEY_LOCAL_MACHINE}[root]     try:
        with winreg.OpenKey(hive, sub) as k:
            return winreg.QueryValueEx(k, name)[0]
except FileNotFoundError:
        return None
    def _ps_json(script: str, timeout: float =
60):
    rc, out, err = run_powershell(script, timeout=timeout)
if rc != 0 or not out.strip():
        if looks_like_access_denied(err):
            raise NeedsPrivilege(err.strip()[:200])         raise
RuntimeError(err.strip()[:300] or f"PowerShell exited {rc}")     return
json.loads(out)
```

```
# ---------------------------------------------------------------------------- PATCH
PS_PATCH = r"""
$ErrorActionPreference='Stop'
$s=New-Object -ComObject Microsoft.Update.Session
$r=$s.CreateUpdateSearcher().Search("IsInstalled=0 and IsHidden=0 and Type='Software'") $u=@(foreach($x
in
     $r.Updates){[pscustomobject]@{Title=$x.Title;Severity=$x.MsrcSeverity;Categories=@(foreach($c in
     $x.Categories){$c.Name})}})
$h=Get-HotFix | Where-Object {$_.InstalledOn} | Sort-Object InstalledOn -Descending | Select-Object -First
     1
[pscustomobject]@{Pending=$u;LastHotfix=$(if($h){$h.InstalledOn.ToString('o')}else{$null})} | ConvertTo-
     Json -Depth 4 -Compress
"""
PS_LAST_HOTFIX = r"""
$h=Get-HotFix | Where-Object {$_.InstalledOn} | Sort-Object InstalledOn -Descending | Select-Object -First
     1
[pscustomobject]@{Pending=$null;LastHotfix=$(if($h){$h.InstalledOn.ToString('o')}else{$null})} | ConvertTo-
     Json -Compress
"""
    def patch(p, ctx) ->
Outcome:
    method = "Windows Update Agent search + Get-HotFix"     rc, out, err = run_powershell(PS_PATCH,
timeout=p.get("timeout", 300))     if rc != 0 or not out.strip():  # offline / WSUS unreachable:
fall back to last-install age only         rc, out, err = run_powershell(PS_LAST_HOTFIX, timeout=60)
method = "Get-HotFix (update search unavailable)"         if rc != 0:
            return Outcome(ERROR, f"Could not query updates: {err.strip()[:200]}", method=method)
facts = P.parse_windows_updates(out)         facts["pending_security"] = None     else:
        facts = P.parse_windows_updates(out)
facts["index_age_days"] = None     return
V.patching(facts, p, ctx.now, method)


# ---------------------------------------------------------------------------- EDR
PS_EDR = r"""
$ErrorActionPreference='SilentlyContinue'
$d=Get-MpComputerStatus | Select-Object
     AMServiceEnabled,AntivirusEnabled,RealTimeProtectionEnabled,AntivirusSignatureAge,AMRunningMode
[pscustomobject]@{Processes=%s;Defender=$d} | ConvertTo-Json -Depth 3 -Compress
""" % PS_PROCESSES
    def edr(p, ctx) -> Outcome:     data = _ps_json(PS_EDR)     found =
P.match_known_processes(data.get("Processes") or [], p["known_processes"])     defender =
P.parse_defender_status(json.dumps(data["Defender"])) if data.get("Defender") else None     return
V.edr(found, defender, p, "Get-Process + Get-MpComputerStatus")


# ---------------------------------------------------------------------------- BACKUP
PS_BACKUP = r"""
$ErrorActionPreference='SilentlyContinue'
$wb=$null if(Get-Command Get-WBSummary){ $s=Get-WBSummary; if($s -and
$s.LastSuccessfulBackupTime -gt
     [datetime]'2000-01-01'){ $wb=$s.LastSuccessfulBackupTime.ToUniversalTime().ToString('o') } }
[pscustomobject]@{WB=$wb;Processes=%s} | ConvertTo-Json -Compress
""" % PS_PROCESSES
    def backup(p, ctx) -> Outcome:     for path in
p["marker_paths"].get("windows", []):
        txt = read_text(path)
if txt:
            ts = P.parse_iso(txt.strip().splitlines()[0])
if ts:
                return V.backup(ts, f"marker {path}", [], p, ctx.now, "KYH backup marker file")
data = _ps_json(PS_BACKUP)     if data.get("WB"):
        return V.backup(P.parse_iso(data["WB"]), "Windows Server Backup", [], p, ctx.now, "Get-WBSummary")
agents = P.match_process_keywords(data.get("Processes") or [], p["process_keywords"])     return
V.backup(None, None, agents, p, ctx.now, "marker file + Get-WBSummary + process table")


# ---------------------------------------------------------------------------- DISK
```

```
PS_BITLOCKER = r"""
$v=Get-BitLockerVolume -MountPoint $env:SystemDrive -ErrorAction Stop
[pscustomobject]@{MountPoint=$v.MountPoint;VolumeStatus=$v.VolumeStatus.ToString();ProtectionStatus=$v.Prot
    ectionStatus.ToString();EncryptionPercentage=$v.EncryptionPercentage} | ConvertTo-Json -Compress
"""
    def disk_encryption(p, ctx) ->
Outcome:
    rc, out, err = run_powershell(PS_BITLOCKER, timeout=60)
if rc == 0 and out.strip():
        info, method = P.parse_bitlocker(out), "Get-BitLockerVolume"
else:         if looks_like_access_denied(err):
            raise NeedsPrivilege("BitLocker status requires administrator rights")
rc, out, err = run_cmd(["manage-bde", "-status", "C:"], timeout=60)         if rc
!= 0:             if looks_like_access_denied(out + err):
                raise NeedsPrivilege("manage-bde requires administrator rights")             return
Outcome(NOT_SUPPORTED, "BitLocker is not available on this edition", method="managebde")
        info, method = P.parse_manage_bde(out), "manage-bde -status"     return
V.disk_encryption(info["fully_encrypted"] or info["protection_on"], info, method,
suspended=info["fully_encrypted"] and not info["protection_on"])


# ---------------------------------------------------------------------------- FIREWALL
def firewall(p, ctx) -> Outcome:     rc, out, err = run_powershell(
        "Get-NetFirewallProfile | Select-Object Name,@{n='Enabled';e={[string]$_.Enabled}} | ConvertTo-Json
    -Compress")
if rc != 0:
        return Outcome(ERROR, f"Get-NetFirewallProfile failed: {err.strip()[:200]}", method="Get-
    NetFirewallProfile")     info = P.parse_windows_firewall(out)     return
V.firewall(info["all_enabled"], info, "Get-NetFirewallProfile (Domain/Private/Public)")


# ---------------------------------------------------------------------------- LISTENERS def
exposed_services(p, ctx) -> Outcome:
    rc, out, err = run_powershell(
        "Get-NetTCPConnection -State Listen | Select-Object LocalAddress,LocalPort | ConvertTo-Json -
    Compress")
if rc != 0:
        return Outcome(ERROR, f"Get-NetTCPConnection failed: {err.strip()[:200]}", method="Get-
    NetTCPConnection")     return V.exposed_services(P.parse_windows_listeners(out), p, "Get-
NetTCPConnection -State Listen")


# ---------------------------------------------------------------------------- REMOTE ACCESS def
remote_access(p, ctx) -> Outcome:
    base = r"HKLM\SYSTEM\CurrentControlSet\Control\Terminal Server"     deny = _reg(base,
"fDenyTSConnections")     nla = _reg(base + r"\WinStations\RDP-Tcp", "UserAuthentication")     return
V.rdp_hardening(rdp_enabled=(deny == 0), nla=(nla == 1), method="registry: Terminal Server settings")


# ---------------------------------------------------------------------------- ADMINS
PS_ADMINS = r"""
$ErrorActionPreference='Stop'
$a=@(Get-LocalGroupMember -SID 'S-1-5-32-544' | ForEach-Object { $_.Name })
$g=Get-LocalUser | Where-Object { $_.SID.Value -like '*-501' } | Select-Object -First 1
[pscustomobject]@{Admins=$a;GuestEnabled=[bool]($g -and $g.Enabled)} | ConvertTo-Json -Compress
"""
    def admin_accounts(p, ctx) ->
Outcome:
    info = P.parse_windows_admins(json.dumps(_ps_json(PS_ADMINS)))
    hashed = [ctx.hash_id(a) for a in info["admins"]]     return
V.admin_accounts(len(info["admins"]), hashed, 0, info["guest_enabled"], p,
                            "Get-LocalGroupMember S-1-5-32-544 + Get-LocalUser (RID 501)")


# ---------------------------------------------------------------------------- OS SUPPORT def
os_support(p, ctx) -> Outcome:
    cv = r"HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion"     build
= str(_reg(cv, "CurrentBuild") or "")     display = _reg(cv,
"DisplayVersion") or _reg(cv, "ReleaseId") or ""     kind = (_reg(cv,
```

```
"InstallationType") or "Client").lower()     edition = (_reg(cv,
"EditionID") or "").lower()     tables = p["eol"]["windows"]     if
"server" in kind:
        table, label = tables["server"], f"Windows Server (build {build})"
else:
        name = "Windows 11" if build.isdigit() and int(build) >= 22000 else "Windows 10"         label
= f"{name} {display} (build {build})"         table = tables["client_enterprise"] if ("enterprise" in
edition or "education" in edition) else tables["client"]
    if build not in table:
        return V.os_support(label, None, p, ctx.now, "registry: CurrentVersion", known=False)
return V.os_support(label, table[build], p, ctx.now, "registry: CurrentVersion")


# ---------------------------------------------------------------------------- LOGGING
PS_LOGS = r"""
$ErrorActionPreference='SilentlyContinue'
$log='Security'; $e=Get-WinEvent -LogName Security -MaxEvents 1 -Oldest if(-not
$e){ $log='System'; $e=Get-WinEvent -LogName System -MaxEvents 1 -Oldest }
$l=Get-WinEvent -ListLog $log
[pscustomobject]@{Log=$log;Oldest=$(if($e){$e.TimeCreated.ToUniversalTime().ToString('o')}else{$null});Enab
led=[bool]$l.IsEnabled;MaxSizeBytes=$l.MaximumSizeInBytes;Processes=%s} | ConvertTo-Json -Compress """ %
PS_PROCESSES
    def log_retention(p, ctx) -> Outcome:     data = _ps_json(PS_LOGS, timeout=90)     fwd =
P.match_known_processes(data.get("Processes") or [], p["forwarder_processes"])     oldest =
P.parse_iso(data.get("Oldest")) if data.get("Oldest") else None     retained = round((ctx.now -
oldest).total_seconds() / 86400, 1) if oldest else None     out =
V.log_retention(bool(data.get("Enabled")), ", ".join(sorted(set(fwd.values()))) or None, retained,
p,
                           f"Get-WinEvent ({data.get('Log')} log, oldest record) + process table")
out.evidence["log_name"] = data.get("Log")     out.evidence["max_size_bytes"] =
data.get("MaxSizeBytes")     return out


# ---------------------------------------------------------------------------- TIME SYNC def
time_sync(p, ctx) -> Outcome:
    params = r"HKLM\SYSTEM\CurrentControlSet\Services\W32Time\Parameters"     ttype =
(_reg(params, "Type") or "").upper()     configured = [s.split(",")[0].lower() for s in
(_reg(params, "NtpServer") or "").split() if s]     synced, sources = None, []     try:
        rc, out, _ = run_cmd(["w32tm", "/query", "/status"], timeout=20)
if rc == 0:
            st = P.parse_w32tm_status(out)
synced = st["synchronized"]             if
st["source"]:
                sources.append(st["source"])
    except Exception:  # noqa: BLE001 - w32tm missing or service stopped; registry still tells us the
     config
        pass
    hierarchy = ttype in ("NT5DS", "ALLSYNC") and not any(P.host_matches(s, p["approved_sources"]) for s in
     sources)
    sources = list(dict.fromkeys(sources + configured))     return V.time_sync(synced, sources, p, "w32tm
/query /status + registry W32Time", hierarchy=hierarchy)
```

E.3 Collector (Part 2)

`kyh-agent/collector/kyh_collector/core.py` 230 lines

Enrolment and the ingestion gate for signed reports (Section 3.3.8).

```
"""Enrollment, report ingestion and admin operations (transport-independent, so tests call them
    directly)."""
  from __future__ import
annotations
  import hashlib import hmac import json import re
import secrets import uuid from datetime import
datetime, timedelta, timezone
  from . import db as
dbm
```

```
STRENGTH_OBSERVED = "TECHNICALLY_OBSERVED"
STRENGTH_SIMULATED = "SIMULATED"
STRENGTH_UNTRUSTED_BUILD = "UNTRUSTED_BUILD"
STRENGTH_UNTRUSTED_POLICY = "UNTRUSTED_POLICY"
STATUSES = {"PASS", "FAIL", "NOT_APPLICABLE", "NOT_SUPPORTED", "ERROR"}
_ID = re.compile(r"^[A-Za-z0-9_.:-]{1,80}$")
    def iso(dt: datetime) ->
str:
    return dt.astimezone(timezone.utc).isoformat(timespec="seconds").replace("+00:00", "Z")
    def parse(ts: str) ->
datetime:
    dt = datetime.fromisoformat(ts.replace("Z", "+00:00"))
return dt if dt.tzinfo else dt.replace(tzinfo=timezone.utc)
    def sha(s) ->
str:
    return hashlib.sha256(s if isinstance(s, bytes) else s.encode()).hexdigest()
    class
ApiError(Exception):
    def __init__(self, status: int, code: str, message: str):
        super().__init__(message)
        self.status, self.code, self.message = status, code, message


# ============================================================ admin operations def
create_admin_token(conn, now) -> str:
    token = "kyh_admin_" + secrets.token_urlsafe(24)
    conn.execute("INSERT INTO admin_tokens VALUES (?,?)", (sha(token), iso(now)))
dbm.audit(conn, iso(now), "cli", "ADMIN_TOKEN_CREATED")     return token  #
shown once; only its hash is stored
    def check_admin(conn, token: str | None) ->
bool:
    if not token:         return False     h = sha(token)     for
row in conn.execute("SELECT token_hash FROM admin_tokens"):
        if hmac.compare_digest(row["token_hash"], h):
            return True
return False

def add_company(conn, now, name: str, declared_endpoints: int, sector: str = "", demo: bool = False,
company_id: str | None = None) -> str:
    cid = company_id or "co_" + uuid.uuid4().hex[:10]     conn.execute("INSERT INTO companies (id,
name, sector, declared_endpoints, demo_mode, created_at) "
                 "VALUES (?,?,?,?,?,?)",
                 (cid, name, sector, int(declared_endpoints), int(demo), iso(now)))
conn.execute("INSERT INTO policies (company_id, status) VALUES (?, 'QUOTED')", (cid,))
dbm.audit(conn, iso(now), "admin", "COMPANY_CREATED", cid, f"{name}; declared={declared_endpoints};
demo={demo}")
    return cid
    def issue_enrollment_token(conn, now, company_id: str, max_uses: int = 50, days: int = 7) ->
str:     if not conn.execute("SELECT 1 FROM companies WHERE id=?", (company_id,)).fetchone():
        raise ApiError(404, "NO_COMPANY", "unknown company")     token = "kyh_enr_" +
secrets.token_urlsafe(18)     conn.execute("INSERT INTO enrollment_tokens VALUES (?,?,?,?,?,0)",
(sha(token), company_id, iso(now), iso(now + timedelta(days=days)), int(max_uses)))
dbm.audit(conn, iso(now), "admin", "ENROLLMENT_TOKEN_ISSUED", company_id, f"max_uses={max_uses};
days={days}")
    return token
    def bind_policy(conn, now, company_id: str, premium_inr: int, settings:
dict,                 model_id: str = "UNDERWRITING_MODEL_V1") -> None:
    """Records the premium that Engine B calculated at inception. Nothing in the collector ever changes
     it."""
    row = conn.execute("SELECT status FROM policies WHERE company_id=?", (company_id,)).fetchone()
if row is None:
        raise ApiError(404, "NO_COMPANY", "unknown company")     linked = conn.execute("SELECT
platform_company_id FROM companies WHERE id=?", (company_id,)).fetchone()     if linked and
linked["platform_company_id"]:
        raise ApiError(409, "LINKED_TO_PLATFORM", "this company is linked to the Beretta platform, which
     binds "
                                               "the policy and owns its warranties")
if row["status"] == "BOUND":
```

```
        raise ApiError(409, "POLICY_ALREADY_BOUND", "policy already bound; the premium is fixed")
conn.execute("UPDATE policies SET status='BOUND', model_id=?, bound_premium_inr=?, bound_at=? WHERE
company_id=?",
                 (model_id, int(premium_inr), iso(now), company_id))
for control in settings["warranty_controls"]:
        conn.execute("INSERT OR REPLACE INTO warranties (company_id, control, state, since) VALUES
    (?,?,'MET',?)",
                     (company_id, control, iso(now)))         conn.execute("INSERT INTO warranty_events
(company_id, control, from_state, to_state, at, reason) "                      "VALUES
(?,?,NULL,'MET',?,'Policy bound')", (company_id, control, iso(now)))     dbm.audit(conn, iso(now), "admin",
"POLICY_BOUND", company_id, f"premium_inr={premium_inr}; model={model_id}")
    def approve(conn, now, table: str, h: str, label: str) ->
None:     assert table in ("approved_builds",
"approved_policies")
    conn.execute(f"INSERT OR REPLACE INTO {table} VALUES (?,?,?)", (h, label, iso(now)))
conn.execute(f"UPDATE agents SET {'build_trusted' if table == 'approved_builds' else
     'policy_trusted'}=1 "                  f"WHERE {'build_hash' if table == 'approved_builds'
else 'policy_hash'}=?", (h,))     dbm.audit(conn, iso(now), "admin", table.upper().rstrip("S") +
"_ADDED", h, label)
    def retire_agent(conn, now, agent_id: str, reason: str) ->
None:
    row = conn.execute("SELECT company_id FROM agents WHERE agent_id=?", (agent_id,)).fetchone()
if row is None:
        raise ApiError(404, "UNKNOWN_AGENT", "unknown agent")
    conn.execute("UPDATE agents SET retired=1 WHERE agent_id=?", (agent_id,))
dbm.audit(conn, iso(now), "admin", "AGENT_RETIRED", agent_id, reason)
    def _approved(conn, table: str, h: str) ->
bool:
    return bool(h) and conn.execute(f"SELECT 1 FROM {table} WHERE hash=?", (h,)).fetchone() is not None


# ============================================================ enrollment def enroll(conn, now, body:
dict) -> dict:     token = body.get("enrollment_token") or ""     row = conn.execute("SELECT * FROM
enrollment_tokens WHERE token_hash=?", (sha(token),)).fetchone()     if row is None:
        dbm.audit(conn, iso(now), "anonymous", "ENROLL_REJECTED", "", "unknown token")
raise ApiError(401, "BAD_TOKEN", "enrollment token not recognised")     if now >
parse(row["expires_at"]):
        raise ApiError(401, "TOKEN_EXPIRED", "enrollment token expired")
if row["uses"] >= row["max_uses"]:
        raise ApiError(401, "TOKEN_EXHAUSTED", "enrollment token has no uses left")     company =
conn.execute("SELECT * FROM companies WHERE id=?", (row["company_id"],)).fetchone()     agent_id
= "ag_" + uuid.uuid4().hex[:16]     secret = secrets.token_hex(32)     build, pol =
str(body.get("build_hash", ""))[:64], str(body.get("policy_hash", ""))[:64]     conn.execute(
        "INSERT INTO agents (agent_id, company_id, hostname, platform, os, secret, enrolled_at,
     agent_version, "
        "build_hash, build_trusted, policy_hash, policy_trusted) VALUES (?,?,?,?,?,?,?,?,?,?,?,?)",
(agent_id, company["id"], str(body.get("hostname", ""))[:128], str(body.get("platform", ""))[:16],
str(body.get("os", ""))[:128], secret, iso(now), str(body.get("agent_version", ""))[:32],          build,
int(_approved(conn, "approved_builds", build)), pol, int(_approved(conn,
    "approved_policies", pol))))     conn.execute("UPDATE enrollment_tokens SET uses=uses+1 WHERE
token_hash=?", (row["token_hash"],))     dbm.audit(conn, iso(now), agent_id, "AGENT_ENROLLED",
company["id"], f"host={body.get('hostname')}")     return {"agent_id": agent_id, "agent_secret": secret,
"company_id": company["id"],             "company_name": company["name"], "server_time": iso(now)}


# ============================================================ ingestion def ingest(conn,
now, agent_id: str, signature: str, raw: bytes, settings: dict) -> dict:
    """Validate and store one signed report. Order of checks matters: authenticate before parsing."""
if len(raw) > settings["max_body_bytes"]:
        raise ApiError(413, "TOO_LARGE", "report exceeds size limit")     agent =
conn.execute("SELECT * FROM agents WHERE agent_id=?", (agent_id or "",)).fetchone()     if
agent is None or agent["retired"]:
        raise ApiError(401, "UNKNOWN_AGENT", "agent not enrolled")     expected = "v1=" +
hmac.new(bytes.fromhex(agent["secret"]), raw, hashlib.sha256).hexdigest()     if not signature
or not hmac.compare_digest(expected, signature):
        dbm.audit(conn, iso(now), agent_id, "REPORT_REJECTED", agent["company_id"], "bad signature")
raise ApiError(401, "BAD_SIGNATURE", "signature does not match report body")     try:
        env = json.loads(raw.decode("utf-8"))
except (ValueError, UnicodeDecodeError):
```

```
        raise ApiError(400, "BAD_JSON", "body is not valid JSON")     if
env.get("schema_version") not in settings["supported_schema_versions"]:
        raise ApiError(400, "SCHEMA_VERSION", f"unsupported schema_version {env.get('schema_version')}")
if env.get("agent_id") != agent_id:
        raise ApiError(400, "AGENT_MISMATCH", "envelope agent_id differs from header")
seq = env.get("seq")     if not isinstance(seq, int) or seq < 1:
        raise ApiError(400, "BAD_SEQ", "seq must be a positive integer")
if seq <= agent["last_seq"]:
        dbm.audit(conn, iso(now), agent_id, "REPORT_REJECTED", agent["company_id"],
f"replay/out-of-order seq {seq} <= {agent['last_seq']}")         raise
ApiError(409, "REPLAY", f"sequence {seq} already received")     try:
        created = parse(env["created_at"])
except (KeyError, ValueError, AttributeError):
        raise ApiError(400, "BAD_TIMESTAMP", "created_at missing or invalid")
if created > now + timedelta(minutes=settings["max_clock_skew_minutes"]):
        raise ApiError(400, "FUTURE_TIMESTAMP", "created_at is in the future beyond allowed clock skew")
if created < now - timedelta(days=settings["max_report_age_days"]):
        raise ApiError(400, "TOO_OLD", "report older than the accepted replay window")
kind = env.get("kind")     if kind not in ("full", "delta", "heartbeat"):
        raise ApiError(400, "BAD_KIND", "kind must be full, delta or heartbeat")
results = env.get("results") or []     for r in results:
        if not isinstance(r, dict) or r.get("status") not in STATUSES or not
    _ID.match(str(r.get("check_id", ""))) \
or not _ID.match(str(r.get("control", ""))):
            raise ApiError(400, "BAD_RESULT", "result entry failed validation")
      ag = env.get("agent") or {}     build = str(ag.get("build_hash", ""))[:64]     pol =
str((env.get("policy") or {}).get("hash", ""))[:64]     build_ok, pol_ok = _approved(conn,
"approved_builds", build), _approved(conn, "approved_policies", pol)     simulated =
bool(env.get("simulated"))     strength = (STRENGTH_SIMULATED if simulated else STRENGTH_UNTRUSTED_BUILD if
not build_ok                 else STRENGTH_UNTRUSTED_POLICY if not pol_ok else STRENGTH_OBSERVED)
      cur =
conn.execute(
        "INSERT INTO reports (agent_id, seq, kind, created_at, received_at, body_sha256, body, simulated, "
        "evidence_strength) VALUES (?,?,?,?,?,?,?,?,?)",
        (agent_id, seq, kind, iso(created), iso(now), sha(raw), raw, int(simulated), strength))
report_id = cur.lastrowid     conn.execute(
        "UPDATE agents SET last_seq=?, last_seen=?, build_hash=?, build_trusted=?, policy_hash=?,
    policy_trusted=?, "
        "last_simulated=?, queue_depth=?, hostname=COALESCE(?, hostname), os=COALESCE(?, os), "
        "agent_version=COALESCE(?, agent_version) WHERE agent_id=?",
        (seq, iso(now), build, int(build_ok), pol, int(pol_ok), int(simulated), env.get("queue_depth"),
ag.get("hostname"), ag.get("os"), ag.get("version"), agent_id))     if kind == "full":
        conn.execute("UPDATE agents SET last_full_at=? WHERE agent_id=?", (iso(now), agent_id))
for r in results:
        observed = str(r.get("observed_at") or iso(created))         vals = (agent_id,
r["check_id"], r["control"], str(r.get("tier", "")), r["status"], r.get("assurance"),
               str(r.get("summary", ""))[:500], json.dumps(r.get("evidence", {}))[:20000],
str(r.get("method", ""))[:200], observed, strength)         conn.execute("INSERT INTO observations
(report_id, agent_id, company_id, check_id, control, tier, status, "
                     "assurance, summary, evidence, method, observed_at, evidence_strength) "
                     "VALUES (?,?,?,?,?,?,?,?,?,?,?,?)",
                     (report_id, vals[0], agent["company_id"]) + vals[1:])         prev =
conn.execute("SELECT observed_at FROM results_current WHERE agent_id=? AND check_id=?",
                             (agent_id, r["check_id"])).fetchone()         if prev is None or
parse(observed) >= parse(prev["observed_at"]):  # late-arriving old reports don't win
            conn.execute("INSERT OR REPLACE INTO results_current VALUES (?,?,?,?,?,?,?,?,?,?,?,?)",
vals + (report_id,))
    return {"accepted": True, "report_id": report_id, "seq": seq, "evidence_strength": strength,
            "company_id": agent["company_id"], "results": len(results)}
```

kyh-agent/collector/kyh_collector/evaluate.py 291 lines

Liveness, the silence rule, the 95% fleet roll-up and stand-alone warranties.

```
"""From host results to company warranty states.

  host result  ->  host control status  ->  company control status  ->  warranty state machine
```

```
This module decides coverage conditions only. It has no access to, and no code path into,
premium calculation: the bound premium is read here for display and never written.
"""
from __future__ import annotations
  import
json import
math
from datetime import timedelta

from .core import STRENGTH_OBSERVED, STRENGTH_SIMULATED, iso, parse from
.settings import LABEL

ONLINE, UNVERIFIED, SILENT = "ONLINE", "UNVERIFIED", "SILENT"
NOT_OBSERVABLE = "NOT_OBSERVABLE"
SILENT_PASS = "SILENT_PASS"   # silent host whose last result passed: left out of the roll-up for a while
MET, IN_GRACE, BREACHED, REMEDIATED = "MET", "IN_GRACE", "BREACHED", "REMEDIATED"
TIER_ORDER = {"warranty": 0, "regulatory": 1, "important": 2}


# ---------------------------------------------------------------------- 1. host liveness def
liveness(agent, now, s) -> dict:
    ref = agent["last_seen"] or agent["enrolled_at"]     hours = (now -
parse(ref)).total_seconds() / 3600     state = ONLINE if hours <=
s["unverified_after_hours"] else UNVERIFIED if hours <= s["silent_after_hours"] \
        else SILENT     return {"state": state, "hours_quiet": round(hours, 1), "never_reported":
agent["last_seen"] is None}


# ---------------------------------------------------------------------- 2. host -> control def
usable(row, demo: bool) -> bool:
    return row["evidence_strength"] == STRENGTH_OBSERVED or (demo and row["evidence_strength"] ==
     STRENGTH_SIMULATED)
    def host_control(rows, live: dict, demo: bool, s: dict | None = None) ->
tuple:
    """rows = this host's current results for one control. Returns (status, reason).

    Silence rule, judged on the host's last known result for this control:
* last result FAIL (or no trusted result at all): FAIL. Going quiet can never hide a failure.       *
  last result PASS, quiet for up to silent_pass_exclusion_days: SILENT_PASS, left out of the
  fleet ratio (staff on leave, office closed for a holiday).
* last result PASS, quiet for longer: FAIL.
    Controls the host could never observe (e.g. MFA) are left alone.
    """     rows = [r for r in rows if usable(r, demo)]
statuses = {r["status"] for r in rows}     observable =
bool(statuses & {"PASS", "FAIL"})     if live["state"] ==
SILENT and (observable or not rows):
        hours = live["hours_quiet"]
if not rows:
            return "FAIL", f"host silent for {hours} h with no trusted result"
if "FAIL" in statuses:
            return "FAIL", f"host silent for {hours} h; last result was FAIL"
cap_days = (s or {}).get("silent_pass_exclusion_days", 0)         if hours <=
cap_days * 24:
            return SILENT_PASS, (f"host silent for {hours} h; last result was PASS, so it is left out of
     the "
                                 f"roll-up until it has been quiet for {cap_days} days")        return
"FAIL", f"host silent for {hours} h, more than {cap_days} days; last result was PASS"     if not rows:
        return "NO_DATA", "no trusted result"     statuses = [r["status"]
for r in rows]     for st in ("FAIL", "PASS", "ERROR", "NOT_SUPPORTED",
"NOT_APPLICABLE"):         if st in statuses:
            return st, next(r["summary"] for r in rows if r["status"] == st)
return statuses[0], rows[0]["summary"]


# ---------------------------------------------------------------------- 3. fleet roll-up def
company_control(control: str, host_statuses: list, s) -> dict:
    """host_statuses = [(hostname, status, reason)]. Only PASS/FAIL hosts are 'observed'."""
observed = [h for h in host_statuses if h[1] in ("PASS", "FAIL")]     passing = [h for h in
```

```
observed if h[1] == "PASS"]     failing = [{"host": h[0], "reason": h[2]} for h in observed
if h[1] == "FAIL"]     quiet = sum(1 for h in host_statuses if h[1] == SILENT_PASS)
    note = f"; {quiet} silent host(s) whose last result passed left out" if quiet else ""
if not observed:
        other = sorted({h[1] for h in host_statuses}) or ["NO_DATA"]
        return {"control": control, "status": NOT_OBSERVABLE, "observed": 0, "passing": 0, "ratio": None,
                "failing_hosts": [], "silent_excluded": quiet,
                "reason": f"No host could observe this control ({', '.join(other)})"}
ratio = len(passing) / len(observed)
    status = "PASS" if ratio >= s["warranty_pass_ratio"] else "FAIL"     return {"control": control,
"status": status, "observed": len(observed), "passing": len(passing),
            "ratio": round(ratio, 4), "failing_hosts": failing, "silent_excluded": quiet,
"reason": f"{len(passing)}/{len(observed)} observed hosts pass ({ratio:.0%}; "
f"threshold {s['warranty_pass_ratio']:.0%}){note}"}
    def coverage(company, lives: list, s, onboarding_until=None, now=None) ->
dict:
    declared = company["declared_endpoints"]     reporting = sum(1 for lv in lives if
lv["state"] != SILENT and not lv["never_reported"])     if onboarding_until is not None and
now is not None and now < onboarding_until:
        return {"control": "MONITORING_COVERAGE", "status": NOT_OBSERVABLE, "declared": declared,
                "enrolled": len(lives), "reporting": reporting,
                "ratio": round(reporting / declared, 4) if declared else None,
                "reason": f"Agent roll-out window until {iso(onboarding_until)}; {reporting} of {declared}
    "                           f"declared endpoints reporting
so far"}     if not declared:
        return {"control": "MONITORING_COVERAGE", "status": NOT_OBSERVABLE, "declared": 0, "enrolled":
    len(lives),
                "reporting": reporting, "ratio": None, "reason": "No declared endpoint count on the
    proposal"}
    ratio = reporting / declared     need = math.ceil(declared * s["coverage_min_ratio"])     return
{"control": "MONITORING_COVERAGE", "status": "PASS" if ratio >= s["coverage_min_ratio"] else "FAIL",
            "declared": declared, "enrolled": len(lives), "reporting": reporting, "ratio": round(ratio, 4),
"reason": f"{reporting} of {declared} declared endpoints reporting ({ratio:.0%}; need {need}, "
f"i.e. {s['coverage_min_ratio']:.0%})"}


# ------------------------------------------------------------------ 4. warranty state machine def
step(w: dict, observed: str, reason: str, now, s) -> tuple:
    """Return (new_state_dict, event or None). Pure: no database access.

    MET --fail--> IN_GRACE --grace expires--> BREACHED
    IN_GRACE/BREACHED --pass--> REMEDIATED --pass held N h--> MET
    REMEDIATED --fail--> IN_GRACE (new grace period)
    NOT_OBSERVABLE never moves a warranty: absence of evidence is not evidence of breach.
    """     control, state = w["control"], w["state"]     new
= dict(w, last_observed=observed, last_reason=reason)
grace = timedelta(days=s["grace_days"].get(control, 7))
effect = s["coverage_effects"].get(control, "")     failing =
observed == "FAIL"     to, why, consequence = None, reason,
None
      if observed ==
NOT_OBSERVABLE:
        pass     elif state == MET
and failing:
        to = IN_GRACE
        new.update(breach_detected_at=iso(now), grace_expires_at=iso(now + grace), remediated_at=None)
consequence = f"If not fixed by {iso(now + grace)}: {effect}"     elif state == IN_GRACE and not
failing:         to = REMEDIATED
        new.update(remediated_at=iso(now))     elif state ==
IN_GRACE and now >= parse(w["grace_expires_at"]):
        to, consequence = BREACHED, effect
        why = f"Grace period expired at {w['grace_expires_at']}; still failing: {reason}"
elif state == BREACHED and not failing:         to = REMEDIATED
        new.update(remediated_at=iso(now))
elif state == REMEDIATED and failing:
        to = IN_GRACE
        new.update(breach_detected_at=iso(now), grace_expires_at=iso(now + grace), remediated_at=None)
consequence = f"Regression. If not fixed by {iso(now + grace)}: {effect}"     elif state ==
REMEDIATED and now - parse(w["remediated_at"]) >= timedelta(hours=s["remediation_confirm_hours"]):
```

```
        to = MET
        why = f"Fix held for {s['remediation_confirm_hours']} h"
new.update(breach_detected_at=None, grace_expires_at=None)
      if to is
None:
        return new, None     new.update(state=to, since=iso(now))     return new, {"control": control,
"from_state": state, "to_state": to, "at": iso(now), "reason": why,                   "consequence":
consequence}


# ---------------------------------------------------------------- orchestration def
evaluate_company(conn, company_id: str, now, s, persist: bool = True) -> dict:
    company = conn.execute("SELECT * FROM companies WHERE id=?", (company_id,)).fetchone()
if company is None:
        return {}     demo = bool(company["demo_mode"])     policy = conn.execute("SELECT * FROM
policies WHERE company_id=?", (company_id,)).fetchone()     agents = conn.execute("SELECT * FROM
agents WHERE company_id=? AND retired=0 ORDER BY hostname",
                          (company_id,)).fetchall()     rows = conn.execute("SELECT rc.* FROM
results_current rc JOIN agents a ON a.agent_id=rc.agent_id "
                       "WHERE a.company_id=? AND a.retired=0", (company_id,)).fetchall()
by_agent = {}     for r in rows:
        by_agent.setdefault(r["agent_id"], []).append(r)
controls = {}  # control -> tier     for r in rows:
        controls.setdefault(r["control"], r["tier"])
for c in s["warranty_controls"]:         if c !=
"MONITORING_COVERAGE":
            controls.setdefault(c, "warranty")
      hosts, lives = [], []     per_control =
{c: [] for c in controls}     for a in agents:
lv = liveness(a, now, s)
lives.append(lv)         mine =
by_agent.get(a["agent_id"], [])         cells
= {}         for c in controls:
            st, why = host_control([r for r in mine if r["control"] == c], lv, demo, s)
cells[c] = {"status": st, "reason": why}             per_control[c].append((a["hostname"] or
a["agent_id"], st, why))         hosts.append({"agent_id": a["agent_id"], "hostname": a["hostname"],
"platform": a["platform"],                       "os": a["os"], "last_seen": a["last_seen"],
"liveness": lv, "controls": cells,
                     "build_trusted": bool(a["build_trusted"]), "policy_trusted":
     bool(a["policy_trusted"]),
                     "simulated": bool(a["last_simulated"]), "agent_version": a["agent_version"],
                     "queue_depth": a["queue_depth"]})
      rollup = [dict(company_control(c, per_control[c], s), tier=controls[c]) for c in
controls]     rollup.sort(key=lambda x: (TIER_ORDER.get(x["tier"], 9), x["control"]))
      linked = bool(company["platform_company_id"]) if "platform_company_id" in company.keys() else
False     platform = None     if linked:
        # Linked mode: the Beretta platform is the only warranty authority. We read back
        # what it last told us and never run our own state machine for this company.         ps =
conn.execute("SELECT * FROM platform_state WHERE company_id=?", (company_id,)).fetchone()
resp = json.loads(ps["response_json"]) if ps and ps["response_json"] else {}
        platform = {"platform_company_id": company["platform_company_id"], "state": dict(ps) if ps else
    None,
                    "policy": resp.get("policy"), "warranties": resp.get("warranties") or []}
bound_at = (resp.get("policy") or {}).get("bound_at")         start = bound_at or
company["platform_linked_at"]
        onboarding_until = parse(start) + timedelta(days=s.get("coverage_onboarding_days", 0)) if start
     else None
    else:
        onboarding_until = None
if policy and policy["bound_at"]:
onboarding_until =
parse(policy["bound_at"]) +
timedelta(days=s.get("coverage_onboarding
_days",
     0))     cov = coverage(company, lives, s, onboarding_until, now)
observed = {x["control"]: (x["status"], x["reason"]) for x in rollup}
observed["MONITORING_COVERAGE"] = (cov["status"], cov["reason"])
```

```
      warranties, changes = [], []
if linked:         for w in
platform["warranties"]:
           warranties.append({"control": w.get("control_name") or w["control_id"],
"platform_control_id": w["control_id"], "state": w["state"],
                              "since": None, "grace_expires_at": w.get("grace_expires_at"),
                              "last_reason": "State held by the Beretta platform",
                              "consequence": w.get("consequence") if w["state"] in (IN_GRACE, BREACHED)
    else None,
                              "from_platform": True})
elif policy and policy["status"] == "BOUND":
       for w in conn.execute("SELECT * FROM warranties WHERE company_id=? ORDER BY control",
    (company_id,)):
           st, why = observed.get(w["control"], (NOT_OBSERVABLE, "no evidence"))
new, event = step(dict(w), st, why, now, s)             if persist:
               conn.execute("UPDATE warranties SET state=?, since=?, breach_detected_at=?,
    grace_expires_at=?, "
                            "remediated_at=?, last_observed=?, last_reason=? WHERE company_id=? AND
    control=?",
                            (new["state"], new["since"], new.get("breach_detected_at"),
    new.get("grace_expires_at"),
                             new.get("remediated_at"), new["last_observed"], new["last_reason"],
    company_id,
                             w["control"]))
if event:
                   conn.execute("INSERT INTO warranty_events (company_id, control, from_state, to_state,
    at, "
                                "reason, consequence) VALUES (?,?,?,?,?,?,?)",
                                (company_id, event["control"], event["from_state"], event["to_state"],
    event["at"],
                                 event["reason"], event["consequence"]))
if event:
               changes.append(event)             active = new["state"] in (IN_GRACE, BREACHED)
new["consequence"] = (s["coverage_effects"].get(w["control"]) if new["state"] == BREACHED else
f"Applies if still failing at {new.get('grace_expires_at')}: "
f"{s['coverage_effects'].get(w['control'])}" if active else None)             warranties.append(new)
     live_counts = {k: sum(1 for lv in lives if lv["state"] == k) for k in (ONLINE, UNVERIFIED,
SILENT)}     if linked:
       pp = platform["policy"] or {}         policy_view = {"status": pp.get("status", "UNKNOWN"),
"bound_premium_inr": pp.get("bound_premium"),
                     "bound_at": pp.get("bound_at"), "model_id": "Beretta platform (Part 1)"}
else:         policy_view = dict(policy) if policy else None     return {
       "company": dict(company),
       "linked": linked,
       "platform": platform,
       "policy": policy_view,
       "premium_note": "Premium fixed at bind. Monitoring changes warranty state only; it never
    reprices.",
       "coverage": cov,
       "liveness_counts": live_counts,
       "controls": rollup,
       "warranties": warranties,
       "warranty_changes": changes,
       "hosts": hosts,
       "label": LABEL,
       "evaluated_at": iso(now),
   }
   def host_detail(conn, agent_id: str) -> dict |
None:
   a = conn.execute("SELECT * FROM agents WHERE agent_id=?", (agent_id,)).fetchone()     if a
is None:         return None     res = conn.execute("SELECT * FROM results_current WHERE
agent_id=? ORDER BY tier, control",
    (agent_id,))     reports = conn.execute("SELECT seq, kind, created_at, received_at,
evidence_strength, body_sha256 FROM reports "
                          "WHERE agent_id=? ORDER BY seq DESC LIMIT 25", (agent_id,))
agent = {k: a[k] for k in a.keys() if k != "secret"}  # never expose the signing secret
results = []        for r in res:              d = dict(r)              d["evidence"] =
json.loads(d["evidence"] or "{}")         results.append(d)     results.sort(key=lambda x:
(TIER_ORDER.get(x["tier"], 9), x["control"]))    return {"agent": agent, "results": results,
```

"reports": [dict(x) for x in reports]} **kyh-agent/collector/kyh_collector/platform_link.py** 268 lines

Mapping to platform controls and the signed roll-ups sent to the platform.

```
"""Linked mode: feed the Beretta platform (Part 1) with fleet roll-ups.

The collector keeps everything that is about endpoints: signed agent reports, replay
protection, the silence rule and the 95% fleet roll-up. The platform keeps everything that
is about the policy: pricing, and the one warranty state machine.

For each linked company the link worker periodically:
1. evaluates the company (same code as the dashboard);
2. maps collector controls onto platform controls (several can feed one);
3. sends one signed roll-up if anything meaningful changed, or as a heartbeat;
4. stores the platform's answer (policy status, warranty states) for display.
"""   from __future__ import
annotations
  import hashlib
import hmac import
json import os import
ssl import threading
import urllib.error
import urllib.request

from . import evaluate as ev from
.core import ApiError, iso, parse

SCHEMA_ID = "kyh-fleet-rollup/1"
LINK_FILE = "platform_link.json"


# ---------------------------------------------------------------------- configuration
def load_link(data_dir: str) -> dict | None:     path = os.path.join(data_dir,
LINK_FILE)     if not os.path.exists(path):
        return None     with open(path,
encoding="utf-8") as fh:
        return json.load(fh)
    def save_link(data_dir: str, url: str, collector_id: str, secret: str, ca_file: str | None =
None,               dashboard_url: str | None = None, name: str = "Beretta collector") -> str:
    os.makedirs(data_dir, exist_ok=True)
path = os.path.join(data_dir, LINK_FILE)
    fd = os.open(path + ".tmp", os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o600)
with os.fdopen(fd, "w", encoding="utf-8") as fh:
        json.dump({"url": url.rstrip("/"), "collector_id": collector_id, "secret": secret, "ca_file":
     ca_file,
                   "dashboard_url": dashboard_url, "name": name}, fh, indent=2)
os.replace(path + ".tmp", path)     return path
    def link_company(conn, now, company_id: str, platform_company_id: str) -> None:
if not conn.execute("SELECT 1 FROM companies WHERE id=?", (company_id,)).fetchone():
        raise ApiError(404, "NO_COMPANY", "unknown company")     conn.execute("UPDATE companies
SET platform_company_id=?, platform_linked_at=? WHERE id=?",
                (platform_company_id, iso(now), company_id))     conn.execute("INSERT INTO
audit_log (at, actor, action, entity, detail) VALUES (?,?,?,?,?)",
                (iso(now), "admin", "PLATFORM_LINKED", company_id,
     f"platform_company_id={platform_company_id}"))


# ---------------------------------------------------------------------- mapping def
build_controls(snapshot: dict, s: dict) -> tuple[list, list]:
    """Group collector controls by platform control.

    FAIL if any source control fails; PASS if at least one passes and none fails;
nothing is sent when every source is NOT_OBSERVABLE (absence of evidence is not sent
as evidence). Returns (controls_to_send, unmapped_or_unobservable).
    """       cmap, partial = s["platform_control_map"], set(s["platform_partial_on_pass"])
rows    =    list(snapshot["controls"])    +    [dict(snapshot["coverage"],    tier="warranty",
passing=snapshot["coverage"].get("reporting"),
```

```
observed=snapshot["coverage"].get("declared"),
failing_hosts=[])]     groups, skipped = {}, []     for r in rows:
        target = cmap.get(r["control"])         if not target:
skipped.append({"control": r["control"], "why": "no platform mapping"})
continue         if r["status"] not in ("PASS", "FAIL"):
            skipped.append({"control": r["control"], "why": r["status"], "platform_control_id": target,
                            "reason": r.get("reason", "")})
continue         groups.setdefault(target, []).append(r)
    # A source no host could observe is listed under the control it would have fed, if that control was
     sent.
    not_observed = {}     for u in [u for u in skipped if
u.get("platform_control_id") in groups]:
not_observed.setdefault(u["platform_control_id"], []).append(u["control"])
skipped = [u for u in skipped if u.get("platform_control_id") not in groups]
out = []     for target, srcs in sorted(groups.items()):         failing = [r for
r in srcs if r["status"] == "FAIL"]         status = "FAIL" if failing else
"PASS"
        value = "false" if failing else ("partial" if any(r["control"] in partial for r in srcs) else
     "true")
def why(r):
            hosts = "; ".join(f"{h['host']}: {h['reason']}" for h in (r.get("failing_hosts") or [])[:5])
return f"{r['control']}: {r['reason']}" + (f" (failing: {hosts})" if hosts else "")         reason = ";
".join(why(r) for r in (failing or srcs))         if value == "partial":
            reason += " (sent as partial: restore testing is not observable from endpoints)"
hosts = []         for r in failing:
            hosts += r.get("failing_hosts") or []         out.append({"platform_control_id":
target, "status": status, "value": value, "reason":
     reason[:600],
                    "observed": min(r["observed"] or 0 for r in srcs), "passing": min(r["passing"] or 0 for
     r in srcs),
                    "failing_hosts": hosts[:50],
                    "sources": [{"control": r["control"], "status": r["status"], "passing": r["passing"],
                                 "observed": r["observed"]} for r in srcs],
                    "not_observed": not_observed.get(target, [])})
return out, skipped
    def fleet_view(snapshot: dict) ->
dict:
    return {"declared": snapshot["coverage"].get("declared"), "enrolled":
     snapshot["coverage"].get("enrolled"),
            "reporting": snapshot["coverage"].get("reporting"), "liveness_counts":
     snapshot["liveness_counts"],
            "collector_controls": [{k: c[k] for k in ("control", "tier", "status", "passing", "observed",
     "reason")}                                     for c in
snapshot["controls"]],
            "hosts": [{"agent_id": h["agent_id"], "host": h["hostname"], "os": h["os"],
                       "liveness": h["liveness"]["state"], "last_seen": h["last_seen"],
                       "simulated": h["simulated"], "build_trusted": h["build_trusted"],
"controls": {c: v["status"] for c, v in h["controls"].items()}}                       for
h in snapshot["hosts"]]}
    def change_hash(controls: list, fleet: dict, coverage: dict) ->
str:
    key = {"c": [(c["platform_control_id"], c["value"]) for c in controls],
           "cov": coverage.get("status"),
           "h": sorted((h["agent_id"], h["liveness"], tuple(sorted(h["controls"].items()))) for h in
     fleet["hosts"])}
    return hashlib.sha256(json.dumps(key, sort_keys=True).encode()).hexdigest()
    def next_seq(conn) ->
int:
    row = conn.execute("SELECT v FROM platform_meta WHERE k='seq'").fetchone()     seq =
(int(row["v"]) if row else 0) + 1     conn.execute("INSERT OR REPLACE INTO platform_meta (k,
v) VALUES ('seq', ?)", (str(seq),))     return seq


# ---------------------------------------------------------------- transport def
sign(secret_hex: str, body: bytes) -> str:
    return "v1=" + hmac.new(bytes.fromhex(secret_hex), body, hashlib.sha256).hexdigest()
    def _open(req, timeout,
ctx):
```

```
    """Go direct (no system proxy) to localhost and private addresses, where a proxy cannot reach."""
import ipaddress     host = urllib.request.urlparse(req.full_url).hostname or ""     try:
        direct = host == "localhost" or ipaddress.ip_address(host).is_private or
ipaddress.ip_address(host).is_loopback     except ValueError:         direct =
False     if direct:
        return urllib.request.build_opener(urllib.request.ProxyHandler({}),
                                           urllib.request.HTTPSHandler(context=ctx)).open(req,
    timeout=timeout)
    return urllib.request.urlopen(req, timeout=timeout, context=ctx)
    def http_send(link: dict, body: bytes, timeout: float =
15):
    req = urllib.request.Request(link["url"] + "/api/collector/rollup", data=body, method="POST")
req.add_header("Content-Type", "application/json")     req.add_header("X-KYH-Collector-Id",
link["collector_id"])     req.add_header("X-KYH-Signature", sign(link["secret"], body))     ctx =
None     if link["url"].startswith("https"):
        ctx = ssl.create_default_context(cafile=link.get("ca_file")) if link.get("ca_file") \
else ssl.create_default_context()     try:
        with _open(req, timeout, ctx) as resp:
            return resp.status, json.loads(resp.read() or b"{}")
except urllib.error.HTTPError as exc:         try:
            return exc.code, json.loads(exc.read() or b"{}")
except ValueError:
            return exc.code, {}     except
(urllib.error.URLError, OSError) as exc:
        return None, {"error": "UNREACHABLE", "message": str(getattr(exc, "reason", exc))}


# ---------------------------------------------------------------- one sync step def
sync_company(app, company_id: str, link: dict, force: bool = False, sender=http_send) -> dict:
"""Evaluate, decide, push, store. The HTTP call happens outside the database lock."""     s, now =
app.settings, app.clock()     with app.lock:
        conn = app.conn()
try:
            conn.execute("BEGIN IMMEDIATE")             snap =
ev.evaluate_company(conn, company_id, now, s)             if not
snap or not snap.get("linked"):
                conn.execute("ROLLBACK")                 return {"pushed": False, "why": "not linked"}
controls, skipped = build_controls(snap, s)             fleet = fleet_view(snap)             h =
change_hash(controls, fleet, snap["coverage"])             ps = conn.execute("SELECT * FROM
platform_state WHERE company_id=?", (company_id,)).fetchone()             due = force or ps is None or
not ps["last_ok"] or ps["last_hash"] != h or not ps["last_push_at"] or \
                (now - parse(ps["last_push_at"])).total_seconds() >= 60 * s["push_heartbeat_minutes"]
if not due:
                conn.execute("ROLLBACK")
return {"pushed": False, "why": "no change"}
seq = next_seq(conn)             conn.execute("COMMIT")
finally:
            conn.close()
      company = snap["company"]     payload = {"schema": SCHEMA_ID, "collector_id":
link["collector_id"], "seq": seq, "sent_at":
     iso(app.clock()),
               "evaluated_at": snap["evaluated_at"],
               "collector_dashboard_url": link.get("dashboard_url"),
               "company": {"collector_company_id": company["id"], "platform_company_id":
     company["platform_company_id"],
                           "name": company["name"], "declared_endpoints": company["declared_endpoints"]},
"coverage": snap["coverage"], "controls": controls, "unmapped": skipped, "fleet": fleet}     body =
json.dumps(payload, sort_keys=True, default=str).encode()     status, resp = sender(link, body)     ok =
status == 200
      with
app.lock:
        conn = app.conn()
try:
            conn.execute("BEGIN IMMEDIATE")
            prev = conn.execute("SELECT response_json FROM platform_state WHERE company_id=?",
    (company_id,)).fetchone()
conn.execute(
                "INSERT INTO platform_state (company_id, last_attempt_at, last_push_at, last_ok, last_http,
    last_error, "
```

```
                "last_hash, last_seq, sent_json, response_json) VALUES (?,?,?,?,?,?,?,?,?,?) "
                "ON CONFLICT(company_id) DO UPDATE SET last_attempt_at=excluded.last_attempt_at, "
                "last_push_at=COALESCE(excluded.last_push_at, platform_state.last_push_at),
    last_ok=excluded.last_ok, "
                "last_http=excluded.last_http, last_error=excluded.last_error, "
                "last_hash=COALESCE(excluded.last_hash, platform_state.last_hash),
    last_seq=excluded.last_seq, "
                "sent_json=excluded.sent_json, "
                "response_json=COALESCE(excluded.response_json, platform_state.response_json)",
                (company_id, iso(now), iso(now) if ok else None, int(ok), status,
                 None if ok else f"{resp.get('error', 'HTTP ' + str(status))}: {resp.get('message',
    '')}"[:300],                  h if ok else None, seq, json.dumps({"controls": controls,
"unmapped": skipped}),                 json.dumps(resp) if ok else (prev["response_json"] if
prev else None)))             conn.execute("COMMIT")         finally:
            conn.close()     return {"pushed": True, "ok": ok, "status": status, "seq": seq, "response":
resp, "controls": controls}


# ------------------------------------------------------------------ background worker class
LinkWorker(threading.Thread):
    """Re-evaluates linked companies every push_interval_seconds, or right away when a report arrives.
    Periodic evaluation is also what turns a quiet host into SILENT without any new report."""
      def __init__(self, app, link: dict):
super().__init__(daemon=True, name="kyh-platform-link")
self.app, self.link = app, link         self.wake =
threading.Event()         self.stopped = threading.Event()
self.last = {}
      def run(self):         while not
self.stopped.is_set():
            self.wake.wait(self.app.settings["push_interval_seconds"])
self.wake.clear()             try:
                conn = self.app.conn()
                ids = [r["id"] for r in conn.execute("SELECT id FROM companies WHERE platform_company_id IS
    NOT NULL")]
conn.close()
for cid in ids:
                    self.last[cid] = sync_company(self.app, cid, self.link)             except
Exception as exc:  # noqa: BLE001 - the link must never take the collector down
self.last["_error"] = repr(exc)
      def stop(self):
self.stopped.set()
self.wake.set()
```